



Willian Dihanster Gomes de Oliveira

***Data Augmentation via Generative Adversarial
Networks* aplicado em classificação de imagens**

São José dos Campos, SP

Willian Dihanster Gomes de Oliveira

***Data Augmentation via Generative Adversarial
Networks* aplicado em classificação de imagens**

Trabalho de conclusão de curso apresentado ao
Instituto de Ciência e Tecnologia – UNIFESP,
como parte das atividades para obtenção do tí-
tulo de Bacharel em Ciência da Computação.

Universidade Federal de São Paulo – UNIFESP

Instituto de Ciência e Tecnologia

Bacharelado em Ciência da Computação

Orientadora: Prof. Dra. Lilian Berton

São José dos Campos, SP

Dezembro de 2019

Na qualidade de titular dos direitos autorais, em consonância com a Lei de direitos autorais nº 9610/98, autorizo a publicação livre e gratuita desse trabalho no Repositório Institucional da UNIFESP ou em outro meio eletrônico da instituição, sem qualquer ressarcimento dos direitos autorais para leitura, impressão e/ou download em meio eletrônico para fins de divulgação intelectual, desde que citada a fonte.

Ficha catalográfica gerada automaticamente com dados fornecidos pelo(a) autor(a)

Gomes de Oliveira, Willian Dihanster

Data Augmentation via Generative Adversarial Networks aplicado em classificação de imagens/ Willian Dihanster Gomes de Oliveira

Orientação: Lilian Berton-São José dos Campos, 2019.

113 p.

Data Augmentation via Generative Adversarial Networks applied in image classification

Trabalho de Conclusão de Curso-Bacharelado em Ciência da Computação-
Universidade Federal de São Paulo-Instituto de Ciência e Tecnologia, 2019.

1. aprendizado de máquina. 2. classificação de imagens. 3. data augmentation. 4.
generative adversarial networks. 5. convolutional neural networks. I. Berton, Lilian

Willian Dihanster Gomes de Oliveira

***Data Augmentation via Generative Adversarial Networks* aplicado em classificação de imagens**

Trabalho de conclusão de curso apresentado ao Instituto de Ciência e Tecnologia – UNIFESP, como parte das atividades para obtenção do título de Bacharel em Ciência da Computação.

Trabalho aprovado em 05 de Dezembro de 2019:

Prof. Dra. Lilian Berton

Orientadora

Prof. Dr. Reginaldo Massanobu Kuroshu

Convidado 1

Dr. Didier A. Vega-Oliveros

Convidado 2

São José dos Campos, SP

Dezembro de 2019

*Este trabalho é dedicado aos meus pais que sempre fizeram de tudo
e me apoiaram incondicionalmente para que eu chegasse até aqui.*

Agradecimentos

Agradeço primeiramente aos meus pais, que sempre me apoiaram incondicionalmente e fizeram de tudo para que eu pudesse realizar o sonho de fazer e concluir o curso de Ciência da Computação em uma universidade pública e de qualidade.

Também agradeço aos meus amigos, principalmente os da graduação, que estiveram comigo nos melhores e piores momentos desses últimos 4 anos. Sem eles, eu não conseguiria chegar até aqui.

À Profa. Lilian Berton que, além de orientadora, foi uma amiga, sempre me apoiando, incentivando e orientando. Parte muito importante da minha formação acadêmica, profissional e conquistas.

À Profa. Daniela Leal Musa, atual coordenadora do curso, por avaliar o meu trabalho e por me iniciar na computação, sendo minha professora de Lógica de Programação quando ingressei na faculdade, e agora presente também, na conclusão do curso.

Aos membros da banca, Reginaldo Massanobu Kuroshu e Didier A. Vega-Oliveros, que aceitaram e se dispuseram a avaliar e oferecer suas considerações ao meu trabalho.

*“Never a failure,
always a lesson”
(Rihanna, 2009)*

Resumo

A classificação de imagens é uma das grandes áreas de aplicação do Aprendizado de Máquina. No entanto, para um bom desempenho dos algoritmos, faz-se necessária uma grande quantidade de dados, especialmente, rotulados. Diversas técnicas foram propostas para se obter mais dados via *data augmentation* (DA). Uma abordagem comum é aplicar Transformação de Imagens (TIs) no conjunto, gerando novas imagens a partir das imagens originais com aplicações de efeitos como corte, rotação, ruídos, *etc.* Uma nova abordagem é o uso das *Generative Adversarial Networks* (GANs) que são redes capazes de sintetizar dados artificiais a partir dos dados originais, sob um processo adversarial de duas redes neurais. Além disso, para se trabalhar com imagens, pode ser necessário representá-las por um vetor de atributos, sendo possível utilizar Redes Neurais Convolucionais (CNNs) para extrair suas características. Portanto, o objetivo deste trabalho é analisar essas duas abordagens de *data augmentation* aplicadas na classificação de imagens e avaliar diferentes CNNs como extratores de características.

Palavras-chaves: aprendizado de máquina. classificação de imagens. *data augmentation*. *convolutional neural networks*. *generative adversarial networks*.

Abstract

Image classification is one of the major areas of application for Machine Learning. However, for a good performance of the algorithms, it needs a large amount of data, especially labeled. Several techniques have been proposed to produce more data, called data augmentation. A common approach is to apply Image Transformation to the dataset, generating new images from the original images with effect such as cropping, rotation, noise and others. A new approach is the use of Generative Adversarial Networks, which are networks capable of synthesizing artificial data from the original data, under the adverse process of two neural networks. Also, to work with images, it may be necessary to represent them as an attribute vector, possibly by using Convolutional Neural Networks (CNNs) to extract their characteristics. Therefore, this work aims to analyze these two approaches of *data augmentation* applied to image classification evaluating different CNNs as feature extractors.

Key-words: machine learning. classification of images. data augmentation. convolutional neural networks. generative adversarial networks.

Lista de Ilustrações

Figura 1 – Exemplos de aplicação Supervisionado x Não Supervisionado.	34
Figura 2 – Categorias do Aprendizado Indutivo.	35
Figura 3 – Exemplos de problemas de predição.	38
Figura 4 – Exemplo prático da extração de características.	39
Figura 5 – Exemplo do cenário de um SVM.	40
Figura 6 – Exemplo de problema não linear.	44
Figura 7 – Neurônio biológico.	46
Figura 8 – Modelo de um neurônio.	47
Figura 9 – Gráficos das funções de ativação.	49
Figura 10 – Hiperplano gerado por um perceptron para UM problema de duas variáveis.	50
Figura 11 – Modelo de uma rede MLP.	52
Figura 12 – Exemplo de arquitetura da CNN LeNet-5.	54
Figura 13 – Visualização da Arquitetura das VGGs.	58
Figura 14 – Esquema da técnica de <i>residual learning</i>	59
Figura 15 – Visualização da Arquitetura da VGG19, uma CNN com 34 camadas e a ResNet34.	60
Figura 16 – Visualização da Arquitetura da Xception.	61
Figura 17 – Exemplo de funcionamento das GANs.	63
Figura 18 – Exemplos de métodos de amostragem.	67
Figura 19 – Exemplo de funcionamento de DA.	69
Figura 20 – Exemplo de funcionamento das transformações de imagens fotométricas.	70
Figura 21 – Exemplo de funcionamento das transformações de imagens geométricas.	70
Figura 22 – Exemplo de aplicação das GANs na reconstrução de rostos.	71
Figura 23 – Exemplos de imagens rotuladas como <i>coffee</i>	79
Figura 24 – Exemplos de imagens rotuladas como <i>non-coffee</i>	80
Figura 25 – Exemplos de imagens da base CIFAR-10.	80
Figura 26 – Exemplos de imagens rotuladas como <i>Dog</i>	81
Figura 27 – Exemplos de imagens rotuladas como <i>Cat</i>	81
Figura 28 – Exemplos de imagens contidas na MNIST.	82
Figura 29 – Exemplos de imagens contidas na EMNIST.	82
Figura 30 – Exemplos de imagens da base MNIST-Fashion.	83
Figura 31 – Esquema da metodologia empregada.	85
Figura 32 – Esquema da Avaliação dos Resultados.	87
Figura 33 – Exemplos de imagens contidas no <i>dataset</i> original.	89
Figura 34 – Exemplos de imagens geradas por Transformação de Imagens.	90

Figura 35 – Exemplos de imagens geradas por <i>Generative Adversarial Networks</i>	90
Figura 36 – Exemplos de imagens contidas no <i>dataset</i> original.	92
Figura 37 – Exemplos de imagens geradas por Transformação de Imagens.	93
Figura 38 – Exemplos de imagens geradas por <i>Generative Adversarial Networks</i>	93
Figura 39 – Exemplos de imagens contidas no <i>dataset</i> original.	96
Figura 40 – Exemplos de imagens geradas por Transformação de Imagens.	96
Figura 41 – Exemplos de imagens geradas por <i>Generative Adversarial Networks</i>	97
Figura 42 – Exemplos de imagens contidas no <i>dataset</i> original.	100
Figura 43 – Exemplos de imagens geradas por <i>Generative Adversarial Networks</i>	100
Figura 44 – Exemplos de imagens contidas no <i>dataset</i> original.	101
Figura 45 – Exemplos de imagens geradas por <i>Generative Adversarial Networks</i>	102
Figura 46 – Exemplos de imagens contidas no <i>dataset</i> original.	103
Figura 47 – Exemplos de imagens geradas por <i>Generative Adversarial Networks</i>	103

Lista de Tabelas

Tabela 1 – Conjunto de Exemplos.	36
Tabela 2 – Tabela de funções de <i>kernel</i>	44
Tabela 3 – Exemplo de Matriz de Confusão.	64
Tabela 4 – Resumo dos Trabalhos Relacionados.	77
Tabela 5 – Resultados para Base 1 sem DA.	90
Tabela 6 – Resultados para Base 1 com TIs.	91
Tabela 7 – Resultados para Base 1 com GANs.	91
Tabela 8 – Resumo dos resultados para Base 1.	91
Tabela 9 – Resultados para Base 2 sem DA.	94
Tabela 10 – Resultados para Base 2 com TIs.	94
Tabela 11 – Resultados para Base 2 com GANs.	94
Tabela 12 – Resumo dos resultados para Base 2.	94
Tabela 13 – Resultados para Base 3 sem DA.	98
Tabela 14 – Resultados para Base 3 com TIs.	98
Tabela 15 – Resultados para Base 3 com GANs.	98
Tabela 16 – Resumo dos resultados para Base 3.	99
Tabela 17 – Resultado médios de acurácia das CNNs para os <i>datasets</i>	104
Tabela 18 – Comparação das CNNs.	104
Tabela 19 – Comparação estatística das abordagens.	106

Lista de Abreviaturas e Siglas

AM	Aprendizado de Máquina
API	<i>Application Programming Interface</i>
ADASYN	<i>Adaptive synthetic sampling approach for imbalanced learning</i>
AVG	<i>Average</i>
AVGPool	<i>Average Pooling</i>
BIC	<i>Border Interior Classification</i>
CIFAR	<i>Canadian Institute for Advanced Research</i>
CNN	<i>Convolutional Neural Network</i>
Conv	Convolução
ConvNet	<i>Convolutional Network</i>
CPU	<i>Central Processing Unit</i>
DA	<i>Data Augmentation</i>
DAGAN	<i>Data Augmentation Generative Adversarial Network</i>
DDSM	<i>Digital Database for Screening Mammography</i>
EMNIST	<i>Extend MNIST</i>
Exp	Exponencial
FC	<i>Full Connected</i>
FN	Falso Negativo
FP	Falso Positivo
Fortran	<i>IBM Mathematical FORMula TRANslation System</i>
GAN	<i>Generative Adversarial Network</i>
GCH	<i>Global Color Histogram</i>
GPU	<i>Graphics Processing Unit</i>

HD	<i>Hard Disk</i>
IA	Inteligência Artificial
ILSVR	<i>ImageNet Large Scale Visual Recognition Challenge</i>
ITP	<i>Image Transformation Pursuit</i>
Log	Logarítmo
Max	Máximo
MaxPool	<i>Max Pooling</i>
Min	Minímo
MNIST	<i>Modified National Institute of Standards and Technology</i>
MLP	<i>Multilayer Perceptron</i>
RAM	<i>Random Access Memory</i>
RBF	<i>Radial Basis Function</i>
RGB	<i>Red, Green, Blue</i>
ReLU	<i>Rectified Linear Units</i>
ResNet	<i>Residual Neural Network</i>
RNA	Rede Neural Artificial
PCA	<i>Principal Component Analys</i>
SGD	<i>Stochastic gradient descent</i>
SMOTE	<i>Synthetic Minority Over-sampling Technique</i>
SV	<i>Support Vector</i>
SVM	<i>Support Vector Machine</i>
Tanh	Tangente Hiperbólico
TB	Terabyte
TCC I	Trabalho de Conclusão de Curso I
TCC II	Trabalho de Conclusão de Curso II
TIs	Transformação de Imagens

VGG	Visual Geometry Group
VN	Verdadeiro Negativo
VN	Verdadeiro Positivo
VOC	<i>Visual Object Classes</i>
WGAN	<i>Wasserstein</i> GAN
Xception	<i>Extreme Inception</i>

Lista de Símbolos

$\mathbb{R}^n$	Conjunto de n -uplas de números reais
$\mathfrak{S}$	Imaginário representa o Espaço de Características no mapeamento da SVM
α	Letra grega alfa utilizada como constante na função Sigmóide e usada como parâmetro do problema de otimização da SVM
δ	Letra grega delta usada como parâmetro multiplicador na função de <i>kernel</i> Polinomial
η	Letra grega eta se refere ao parâmetro de taxa de aprendizado de uma rede neural
κ	Letra grega capa usada como parâmetro de adição na função de <i>kernel</i> Polinomial
Φ	Letra grega fi é utilizada como o mapeamento do Espaço de Atributos para Espaço de Características na SVM
φ	Letra grega Fi em outra representação sendo utilizada para denotar as funções de ativações de uma rede neural
ρ	Letra grega rô representa o valor de margem de um hiperplano para a SVM
σ	Letra grega sigma é um parâmetro para a função de <i>kernel</i> Gaussiano
$\sum$	Letra grega Sigma indicando a função de Somatório
θ	Letra grega teta representa os parâmetros de uma rede MLP para as GANs
ξ	Letra grega xi representando as variáveis de folga da SVM
$\cdot$	Multiplicação (quando escalares) e Produto Escalar (quando vetores)
$\ \mathbf{x}\ $	Norma do vetor $\mathbf{x}$
$\geq$	Maior que
$\leq$	Menor que
$\ll$	Muito menor que
$\forall$	Para todo
$\in$	Pertence

$\times$ Vezes (Indicando dimensão 5×5)

$\%$ Porcentagem

$\sum_a^b$ Somatório de a até b

$\mapsto$ Seta de mapeamento

$\mathbf{x}$ Vetor $\mathbf{x}$

$\mathbf{x}^T$ Vetor $\mathbf{x}$ transposto

$\cup$ União

Sumário

1	Introdução	29
1.1	Contextualização e motivação	29
1.2	Justificativas	30
1.3	Objetivos	31
1.3.1	Objetivo Geral	31
1.3.2	Objetivos Específicos	31
1.4	Metodologia	32
1.5	Organização do Documento	32
2	Fundamentação Teórica	33
2.1	Aprendizado de Máquina	33
2.1.1	Conceitos de Aprendizado de Máquina	35
2.1.2	Algoritmos Preditivos	37
2.1.2.1	Classificação de Imagens	38
2.1.2.1.1	Extração de Características	39
2.1.2.2	<i>Support Vector Machine (SVM)</i>	39
2.1.2.2.1	SVM com Margens Rígidas	40
2.1.2.2.2	SVM com Margens Suaves	43
2.1.2.2.3	SVM Não-linear	43
2.1.2.2.4	SVM para Problemas Multi-classes	45
2.1.2.2.5	Vantagens e Desvantagens das SVMs	45
2.1.2.3	Redes Neurais Artificiais (RNAs)	45
2.1.2.3.1	Neurônio Biológico	46
2.1.2.3.2	Neurônio Artificial	47
2.1.2.3.3	Funções de Ativação	48
2.1.2.3.4	Perceptron	49
2.1.2.3.5	Treinamento de uma RNA	51
2.1.2.3.6	<i>Multilayer Perceptron</i>	51
2.1.2.3.7	Vantagens e Desvantagens das RNAs	53
2.1.3	<i>Deep Learning</i>	53
2.1.3.1	<i>Convolutional Neural Networks (CNNs)</i>	53
2.1.3.1.1	Convolução	54
2.1.3.1.2	<i>Pooling</i>	55
2.1.3.1.3	Camada Totalmente Conectada	56
2.1.3.2	Transferência de Aprendizado e <i>Finetuning</i>	56

2.1.3.3	Modelos de <i>Convolutional Neural Networks</i>	56
2.1.3.3.1	VGGNet	57
2.1.3.3.2	ResNet	58
2.1.3.3.3	Xception	60
2.1.3.4	<i>Generative Adversarial Network (GANs)</i>	61
2.1.4	Medidas de Avaliação de Algoritmos de AM	63
2.1.4.1	Taxa de Arro e Acurácia	63
2.1.4.2	Matriz de Confusão	64
2.1.4.3	Medidas de Desempenho de Algoritmos de AM	65
2.1.4.4	Métodos de Amostragem para Algoritmos de AM	65
2.1.4.5	Medidas de Comparação de Algoritmos de AM	67
2.2	<i>Data Augmentation</i>	68
2.2.1	Transformação de Imagens	69
2.2.2	<i>Generative Adversarial Network (GANs)</i> Como Técnica de DA	71
3	Revisão Bibliográfica	73
3.1	Trabalhos Relacionados	73
3.1.1	Aplicação de DA	73
3.1.2	Avaliação e Propostas de Novas Técnicas de DA Baseada em Transformação de Imagens	74
3.1.3	Métodos Gerativos como DA	75
3.2	Resumo dos Trabalhos Relacionados	76
3.3	Conclusões	78
4	Proposta de Trabalho	79
4.1	Bases de Dados	79
4.1.1	<i>Brazilian Coffee Scenes Dataset</i>	79
4.1.2	CIFAR-10	80
4.1.3	<i>Dogs vs Cats</i>	80
4.1.4	<i>MNIST DATABASE of handwritten digits</i>	81
4.1.5	EMNIST	82
4.1.6	MNIST-Fashion	82
4.2	Softwares e Bibliotecas	83
4.2.1	Python	83
4.2.2	TensorFlow	83
4.2.3	Keras	84
4.2.4	scikit-learn	84
4.2.5	NumPy	84
4.3	Método e Análise Proposta	84
4.3.1	Aplicação das Técnicas de DA	85

4.3.2	Extração de Características das Imagens	86
4.3.3	Avaliação dos Resultados	87
4.3.4	Ambiente Computacional	87
5	Resultados	89
5.1	Base 1: MNIST	89
5.1.1	Aplicação de DA	89
5.1.2	Resultados	90
5.1.3	Discussão	91
5.2	Base 2: EMNIST	92
5.2.1	Aplicação de DA	92
5.2.2	Resultados	93
5.2.3	Discussão	95
5.3	Base 3: MNIST-Fashion	95
5.3.1	Aplicação de DA	95
5.3.2	Resultados	98
5.3.3	Discussão	99
5.4	Base 4: <i>Brazilian Coffee Scenes Dataset</i>	99
5.5	Base 5: CIFAR-10	101
5.6	Base 6: Dogs vs Cats	102
5.7	Avaliação das CNNs	103
5.8	Discussão dos Resultados	105
6	Considerações Finais	107
	Referências	109

1 Introdução

Nesse capítulo é apresentada a contextualização do trabalho, os objetivos e breve descrição da metodologia a ser empregada.

1.1 Contextualização e motivação

O Aprendizado de Máquina (AM) é uma subárea da Inteligência Artificial, que se baseia em construir e modelar sistemas capazes de adquirir conhecimento e que melhorem automaticamente com a experiência (MITCHELL et al., 1990). No AM o aprendizado indutivo é o mais comum, ou seja, são utilizadas técnicas de indução para obter conclusões genéricas sobre um conjunto particular de exemplos (MONARD; BARANAUSKAS, 2003). Este tipo de aprendizado possui uma hierarquia, podendo ser dividido entre aprendizado supervisionado, não supervisionado e semissupervisionado.

No aprendizado supervisionado, onde a classificação é uma das tarefas mais conhecidas, a quantidade de exemplos rotulados disponíveis e os resultados estão relacionados: quanto maior a quantidade de exemplos, maior a capacidade de predição dos classificadores gerados. No entanto, em situações da vida real, possuir bases de dados com uma grande quantidade de exemplos rotulados nem sempre é uma tarefa fácil, e muitas vezes, custosa. Desse modo, podemos utilizar abordagens de *data augmentation* (DA), que é um conceito fundado por técnicas computacionais com o objetivo de aumentar a quantidade de exemplos rotulados em um conjunto de dados e assim, melhorar os resultados obtidos (TAYLOR; NITSCHKE, 2018).

Na literatura, diversas técnicas para obtenção de mais dados rotulados foram propostas. Uma abordagem simples e comum é expandir o conjunto de dados por meio de transformações de imagens (TIs), gerando novas imagens a partir de rotações, cortes, efeitos *blur* e *etc*, no conjunto original. Já uma nova abordagem são as *Generative Adversarial Network* (GANs) (GOODFELLOW et al., 2014), que são redes capazes de gerar dados artificiais, a partir dos dados originais, sob um processo adversarial de duas redes.

A classificação de imagens possui diversas aplicações, por exemplo: análise de dados de satélites (PENATTI K. NOGUEIRA, 2015), reconhecimento facial (VARGAS; PAES; VASCONCELOS, 2016), detecção de doenças (VEMURI et al., 2008), dentre outras tarefas. Porém, em aplicações como imagens aéreas e médicas, os dados são limitados e, dessa forma é evidente a necessidade de utilização de técnicas de DA, já que a classificação de imagens oferece uma boa quantidade de aplicações e utilidade ao ser humano.

Além disso, em algumas aplicações de classificação de imagens, é necessário repre-

sentar as imagens por um vetor de suas características e portanto, é necessário utilizar um bom descritor de imagens. Atualmente, as *Convolutional Neural Networks* (CNNs) estão sendo muito utilizadas para essa tarefa. As CNNs são redes neurais baseadas no processo biológico do processamento de dados visuais, possuem uma fase de extração de características e uma de classificação. Essas redes estão obtendo ótimos resultados na classificação e reconhecimento de imagens, pois aplicam filtros detectores de padrões e preservam a noção espacial das imagens (VARGAS; PAES; VASCONCELOS, 2016).

Sendo assim, neste trabalho, utilizamos diferentes técnicas de DA aplicadas na classificação de imagens, como a abordagem de Transformação de Imagens e o uso das *Generative Adversarial Networks* em bases de dados de diferentes aplicações. As imagens originais e as geradas por DA foram descritas por *Convolutional Neural Networks*, sendo elas a ResNet, VGGNet e Xception. Por fim, as bases geradas passaram por experimentos de classificação com o algoritmo *Support Vector Machine* (SVM).

A hipótese inicial, que DA seria capaz de aumentar a acurácia dos *datasets* simulados, não foi observada para todas as bases. Em algumas bases, as GANs não conseguiram convergir no tempo definido, devido sua complexidade. Com as bases em que foi possível aplicar as GANs, os experimentos demonstraram uma significância da aplicação de GANs em relação às TIs. Com TIs, não houve melhora na acurácia. Com as GANs, em dois de três *datasets*, a acurácia foi aumentada. Em relação à avaliação das CNNs, a ResNet foi responsável pelos melhores resultados de acurácia, menor tempo e gerou menos atributos, também possui relativamente menos parâmetros e é relativamente pequena (em espaço de armazenamento).

1.2 Justificativas

A classificação de imagens possui diversas aplicações no mundo real, como reconhecimento facial, detecção de doenças, entre outras. Mas para executá-la, é necessário obter uma grande quantidade de dados, o que muitas vezes não é uma tarefa fácil e geralmente, custosa. Por exemplo, dados médicos precisam que exames sejam realizados, o que toma tempo, recursos monetários e dependendo da condição médica, pode ser um caso raro. Além disso, nesse caso, é necessário a anotação de um especialista para rotular os dados.

Sendo assim, a geração de dados artificiais por meio de técnicas de *data augmentation* pode ser uma alternativa mais viável. Diversos trabalhos já aplicaram DA e obtiveram melhora em seus resultados. (HUSSAIN et al., 2017) aplicaram DA de imagens de mamografia e conseguiram aumentar a acurácia de validação de 84% para 88%. (WANG et al., 2018) também utilizou DA em dados médicos, mas dessa vez com imagens do cérebro para detecção de alcoolismo, os autores conseguiram uma acurácia de 97.04% e resultados superior a outras três abordagens do estado da arte. Já (YU et al., 2017) aplicaram DA em imagens aéreas (outra

área de difícil obtenção de dados) e conseguiram resultados significativamente melhores que os encontrados com o conjunto original disponível.

1.3 Objetivos

1.3.1 Objetivo Geral

O objetivo deste trabalho é estudar e analisar diferentes técnicas baseadas em DA aplicadas à classificação de imagens, comparando uma técnica tradicional (Transformação de Imagens) e uma técnica nova (*Generative Adversarial Network*). Assim, verificaremos os resultados, vantagens e desvantagens de cada técnica.

1.3.2 Objetivos Específicos

Os objetivos específicos são:

- estudar os principais conceitos de AM;
- estudar e selecionar técnicas de DA, como a Transformação de Imagens e *Generative Adversarial Network*;
- estudar diferentes tipos de redes neurais;
- estudar transferência de aprendizado e extração de características de imagens por meio de *Convolutional Neural Networks*;
- separar as bases de dados escolhidas em treino e teste. Além de k conjuntos diferentes para cada base e para cada técnica de DA;
- aplicar a técnica de Transformação de Imagens aos conjuntos de dados;
- realizar experimentos e avaliação sobre as bases geradas pelas Transformação de Imagens;
- aplicar a técnica de *Generative Adversarial Network* nos conjuntos de dados;
- realizar experimentos e avaliação sobre as bases geradas pelas *Generative Adversarial Network*;
- avaliar e comparar os resultados obtidos pelas duas técnicas.

1.4 Metodologia

Para alcançar os objetivos com a realização deste trabalho a metodologia pode ser dividida em 4 etapas principais.

Na primeira etapa, foi realizado um levantamento teórico sobre conceitos de Aprendizado de Máquina, técnicas de *data augmentation*, redes neurais e uma revisão bibliográfica sobre trabalhos relacionados por meio de livros e artigos científicos.

Na segunda etapa, foram pesquisados e definidos quais bases de dados, algoritmos e bibliotecas seriam utilizadas. Além disso, os algoritmos e *scripts* necessários foram devidamente implementados.

Na terceira etapa foram feitas as experimentações das técnicas de DA estudadas com os algoritmos e bases de dados escolhidas. Em seguida, foram feitas avaliações com medidas estatísticas para efeitos de comparação.

Por fim, na quarta e última etapa, foi realizada a documentação, com a escrita e organização dos conceitos, resultados e comparações.

1.5 Organização do Documento

Essa monografia está organizada em outros 5 capítulos como segue:

No Capítulo 2, a fundamentação teórica é apresentada, abordando conteúdos teóricos básicos de Aprendizado de Máquina, classificação de dados, algoritmo *Support Vector Machine*, Redes Neurais Artificiais, uma introdução ao *Deep Learning* e as *Convolutional Neural Networks*, *Generative Adversarial Network*, medidas de avaliação e comparação de algoritmos de AM e o conceito de *data augmentation*.

No Capítulo 3, a revisão bibliográfica é detalhada, com a apresentação de alguns trabalhos relacionados que utilizaram técnicas de *data augmentation*, divididos entre aplicações de DA, novas propostas de técnicas de DA e abordagens diferentes, como as GANs (métodos gerativos).

No Capítulo 4, temos a proposta do trabalho, onde são detalhadas as bases de dados utilizadas, os softwares e bibliotecas, algoritmos e a metodologia empregada.

Já no Capítulo 5, são apresentados os resultados obtidos com a aplicação das duas técnicas nas bases de dados.

Por fim, no Capítulo 6, temos as considerações finais, de acordo com os resultados obtidos.

2 Fundamentação Teórica

Neste capítulo é detalhado o embasamento teórico e a sustentação deste trabalho, descrevendo os principais conceitos de Aprendizado de Máquina, bem como alguns algoritmos, técnicas, métodos de avaliação e comparação.

2.1 Aprendizado de Máquina

O Aprendizado de Máquina (AM) é uma subárea da Inteligência Artificial, que se baseia em construir e modelar sistemas capazes de adquirir conhecimento de forma automática e que melhorem automaticamente com a experiência (MICHALSKI; TECUCI, 1994). Assim, o sistema de aprendizado é um software computacional que toma decisões baseadas nas experiências acumuladas de soluções de problemas anteriores (MONARD; BARANAUSKAS, 2003).

No AM a forma de adquirir conhecimento e prever eventos futuros é por meio do aprendizado indutivo. Isto é, são utilizadas técnicas de indução, que é uma forma de inferência lógica, que permite obter conclusões genéricas sobre um conjunto particular de exemplos. A indução é um raciocínio que se origina em um conceito específico e o generaliza (da parte para o todo) e sendo assim, o conhecimento é adquirido utilizando inferência indutiva sobre os exemplos que foram apresentados e portanto, não necessariamente preservam a verdade (MONARD; BARANAUSKAS, 2003). Ainda, esse aprendizado indutivo pode ser dividido em categorias, comumente, em supervisionado, não supervisionado e semissupervisionado.

No AM a forma de adquirir conhecimento e prever eventos futuros é por meio do aprendizado indutivo. Isto é, são utilizadas técnicas de indução, que é uma forma de inferência lógica, que permite obter conclusões genéricas sobre um conjunto particular de exemplos. A indução é um raciocínio que se origina em um conceito específico e o generaliza a partir de exemplos, e portanto, não necessariamente preservam a verdade (MONARD; BARANAUSKAS, 2003). Ainda, esse aprendizado indutivo pode ser dividido em categorias, comumente, em supervisionado, não supervisionado e semissupervisionado.

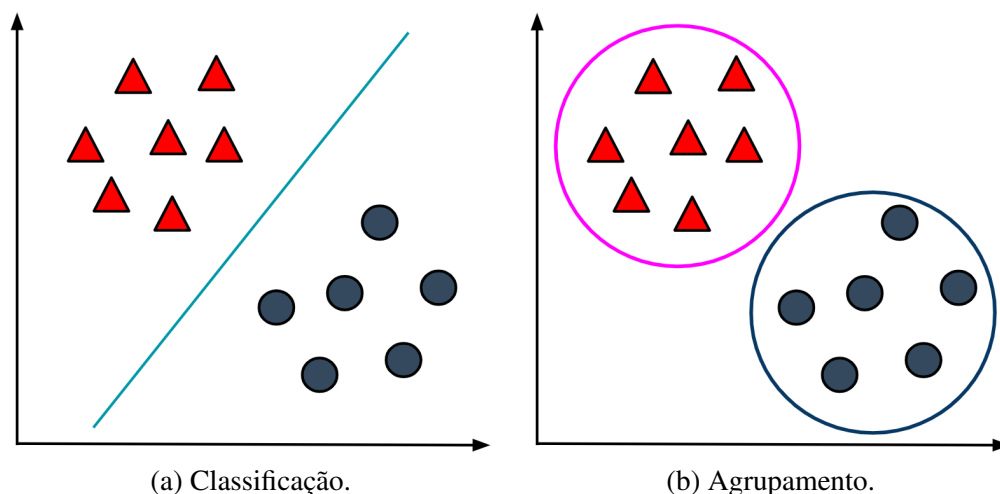
No aprendizado supervisionado, o algoritmo de aprendizado trabalha com um conjunto de treinamento onde o rótulo ou classe de cada exemplo é conhecido. Assim, o objetivo deste algoritmo é construir um estimador que possa determinar corretamente o rótulo de novos exemplos, que não faziam parte do conjunto de treino. Em problemas com rótulos discretos, o problema é conhecido como classificação, para rótulos contínuos, regressão (MONARD; BARANAUSKAS, 2003). O aprendizado supervisionado possui diversas aplicações na vida real. Alguns exemplos de problemas de classificação são detectar se determinado e-mail é *spam* ou não (CRAWFORD et al., 2015), detecção de fraudes em cartão de crédito (BAHNSEN et al.,

2014), detecção de doenças (VEMURI et al., 2008) e *etc.* Problemas de regressão podem ser exemplificados por problemas de predição de preços de imóveis (PARK; BAE, 2015), predição de preços de carros (PUDARUTH, 2014), dentre outras tarefas.

Já no aprendizado não-supervisionado, o algoritmo de aprendizado trabalha com um conjunto de treinamento onde o rótulo ou classe de cada exemplo não é conhecido. E então é comum a tarefa de agrupamento, onde o objetivo é analisar os exemplos fornecidos e determinar se alguns deles podem ser agrupados de alguma maneira, formando agrupamentos ou *clusters* (MONARD; BARANAUSKAS, 2003). Um exemplo de problema de agrupamento seria, dado um conjunto de exemplos de animais e suas características, como número de patas, *habitat* natural e *etc.*, separar esses animais em diferentes *clusters* a partir da similaridade. Após a geração dos agrupamentos poderia ser concluído, por exemplo, que tal *cluster* são de animais aquáticos e um outro *cluster* poderia representar animais não aquáticos.

Na Figura 1 são exemplificados dois cenários diferentes: uma aplicação de aprendizado supervisionado e uma de aprendizado não supervisionado. A Figura 1a apresenta um cenário de classificação, separando dois objetos diferentes. Já na Figura 1b há um cenário de agrupamento, onde foram encontrados dois *clusters* entre os objetos. Coincidentemente, os mesmos objetos ficaram separados entre classes e *clusters* diferentes, mas isso nem sempre acontece.

Figura 1: Exemplos de aplicação Supervisionado x Não Supervisionado.

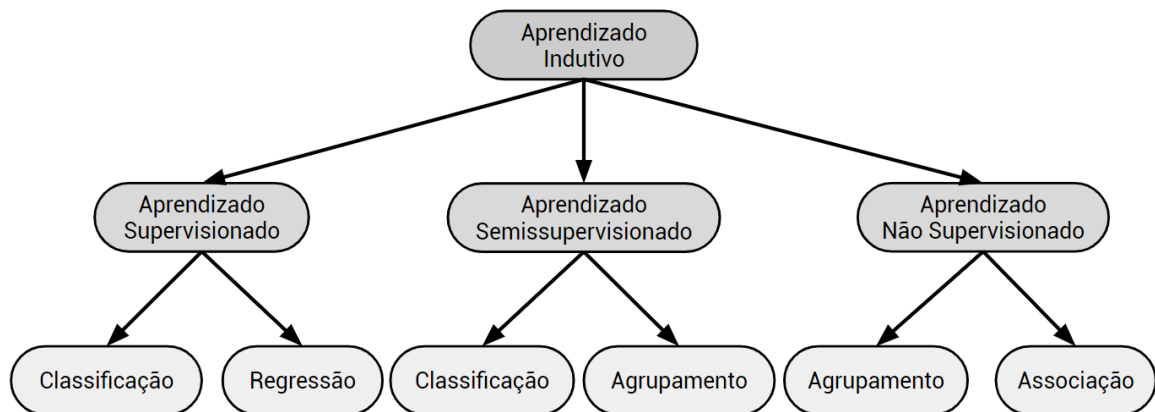


Fonte: O autor.

Ainda, há o aprendizado semissupervisionado definido como um intermédio entre o aprendizado supervisionado e não-supervisionado (CHAPELLE; SCHOLKOPF; ZIEN, 2009). Aqui o algoritmo de aprendizado trabalha com um conjunto de treinamento formado por exemplos rotulados e um conjunto (geralmente grande) de dados não rotulados.

Na Figura 2, o diagrama apresenta um resumo das categorias do aprendizado indutivo de AM, e suas principais aplicações.

Figura 2: Categorias do Aprendizado Indutivo.



Fonte: O autor.

2.1.1 Conceitos de Aprendizado de Máquina

A seguir são apresentados os principais conceitos relacionados com AM, focado em Aprendizado Supervisionado (MONARD; BARANAUSKAS, 2003).

- **indutor:** é o algoritmo ou programa de aprendizado que usa indução a fim de extrair e definir um classificador a partir de um conjunto de exemplos. Esse indutor pode então ser utilizado para classificar novos exemplos com o objetivo de prever corretamente o rótulo de cada um;
- **exemplo:** é um dado, registro ou modelo, comumente representado por uma tupla de valores, que vão ser usadas e/ou observadas pelos algoritmos;
- **atributo:** é descrito como uma característica ou aspecto do exemplo, como cor, tamanho, *etc.* Esses atributos podem ser nominais ou contínuos, mas em alguns casos pode haver dados faltantes, que podem ser representados por “?”;
- **classe:** é um atributo especial que rótula/categoriza os exemplos. Geralmente descreve o fenômeno de interesse, que é a meta que se deseja aprender e fazer previsões a respeito. Para problemas de classificação, esses rótulos são discretos e para problemas de regressão, valores reais;
- **conjunto de exemplos:** representa um conjunto de dados com seus atributos e respectivos valores. Em problemas de classificação ou regressão há a presença do rótulo com sua classe ou valor, geralmente como último atributo da tupla.

Sendo assim, definindo os atributos de um exemplo como um conjunto $A = \{A_1, A_2, \dots, A_N\}$ e um exemplo como um conjunto $X_i = \{x_{i1}, x_{i2}, \dots, x_{im}, y_i\}$, em que cada X_{ij} cor-

responde a um valor de atributo A_j , podemos definir um conjunto de exemplos como $X = \{X_1, X_2, \dots, X_n\}$, que também pode ser visualizado na Tabela 1.

Tabela 1: Conjunto de Exemplos.

	$\mathbf{A}_1$	$\mathbf{A}_2$	$\dots$	$\mathbf{A}_m$	$\mathbf{Y}$
X_1	x_{11}	x_{12}	$\dots$	x_{1m}	y_1
X_2	x_{21}	x_{22}	$\dots$	x_{2m}	y_2
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
X_n	x_{n1}	x_{n2}	$\dots$	x_{nm}	y_n

Fonte: Adaptado de (MONARD; BARANAUSKAS, 2003).

Ainda, esse conjunto de exemplos geralmente é dividido entre dois conjuntos disjuntos: o conjunto de treinamento e o conjunto de teste. O conjunto de treinamento geralmente é utilizado para o aprendizado de conceitos e o conjunto de testes serve para medir o grau de efetividade dos conceitos aprendidos;

- modo de aprendizado: o modo de aprendizado pode ser dividido entre aprendizado não incremental e incremental;
 - não incremental: também conhecido como modo *batch*, nesse modo todo o conjunto de treinamento deve estar presente para o aprendizado;
 - incremental: nesse modo, é possível adicionar novos exemplos ao conjunto de treinamento (por meio de *batches*), assim não é necessário construir uma hipótese a partir do início, apenas atualiza-se a hipótese antiga quando forem adicionados novos exemplos ao conjunto de treinamento;
- distribuição de classes: em algumas aplicações é importante saber quantos exemplos pertencem a tal classe em um conjunto de exemplos. Assim, dada uma classe C_j , a distribuição é dada pela soma de vezes que essa classe faz parte dos exemplos, dividido pelo número total de exemplos do conjunto, sendo também definida pela equação a seguir.

$$\text{distr}(C_j) = \frac{1}{n} \sum_{i=1}^n \|y_i = C_j\| \quad (2.1)$$

Onde o operador $\|E\|$ retorna 1 se a expressão E for verdadeira e 0 caso contrário. E a partir desse cálculo, há dois novos conceitos de prevalência de classes:

- classe majoritária: é a classe que mais aparece no conjunto de dados;
- classe minoritária: é a classe que menos aparece no conjunto de dados;

Em aplicações onde todas as classes possuem mesma distribuição, o conjunto de dados é dito ser balanceado. Conjunto de dados balanceados são preferíveis em muitos algoritmos de AM, embora em muitas aplicações, isso não seja possível;

- *bias*: é definido como qualquer preferência de uma hipótese sobre outra. Como existe sempre um número grande de hipóteses consistentes, todos os indutores possuem alguma forma de *bias*;
- *overfitting*: acontece quando um algoritmo não é capaz de generalizar para amostras diferentes, sendo muito específico e ajustado somente para o conjunto de exemplos apresentado para geração do indutor;
- *underfitting*: quando após a geração do modelo, a taxa de erro ainda é alta. Isto é, o algoritmo não conseguiu aprender e é comum em conjuntos de exemplos pouco representativos ou que seja muito pequeno;
- ruídos: são inconsistências de dados que podem fazer parte do conjunto de treinamento. Por exemplo, um e-mail de *spam* que estava registrado no conjunto de dados como não *spam*;
- *outlier*: é quando um dado possui valores/descrição muito discrepante dos outros dados do conjunto de exemplos. Por exemplo, um conjunto de dados de pessoas jovens com um atributo de idade, em que um exemplo tem valor de idade 78.

2.1.2 Algoritmos Preditivos

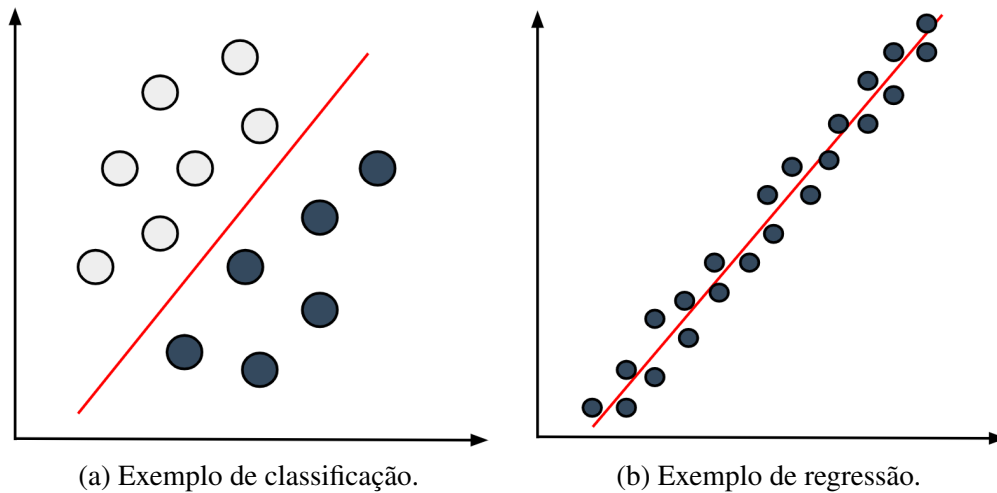
Uma das maiores aplicações do AM, é a predição. Um algoritmo de AM preditivo é uma função que, dados um conjunto de exemplos rotulados, constrói um estimador. Assim, dizemos que temos uma tarefa de classificação quando os rótulos são nominais e então o estimador gerado é um classificador. Um classificador por sua vez é definido como uma função que, dado um exemplo ainda não rotulado, atribui esse exemplo a uma das possíveis classes. No caso de rótulos contínuos o estimador gerado é um regressor (FACELI et al., 2011).

Com isso, a classificação e a regressão possuem diversas aplicações na vida real. A classificação pode ser utilizada em problemas que podem ser separados entre categorias, por exemplo: classificar e-mails que são *spam* ou não são *spam*. A regressão pode ser aplicada em problemas em que deve-se estimar um valor, como exemplo, predizer o preço de uma casa.

Na Figura 3a há uma exemplificação de um cenário de classificação, onde objetos simples são separados por duas classes diferentes. A meta de um classificador é gerar uma fronteira de decisão que separe os dados entre suas classes corretamente. O exemplo da Figura 3a é um caso simples, sendo um problema linearmente separável e com isso uma reta foi suficiente como fronteira de decisão. Caso um problema não seja linearmente separável, é necessário uma

combinação de retas. Se os exemplos possuírem mais de dois atributos de entrada, em vez de retas, serão gerados hiperplanos de separação. Além disso, algoritmos diferentes podem encontrar fronteiras de decisão diferentes para um mesmo conjunto (FACELI et al., 2011). Na Figura 3b é exemplificado um cenário de regressão simples, onde os pontos (rótulos contínuos) foram aproximados por uma reta.

Figura 3: Exemplos de problemas de predição.



Fonte: O autor.

Para isso, os algoritmos trabalham com os conjuntos de dados. Os conjuntos de dados são constituídos de n tuplas representando exemplos com suas características ou atributos (que podem ser nominais ou numéricos, por exemplos), e no caso do aprendizado supervisionado, um atributo-meta y que é o atributo de interesse da predição. Esse conjunto é separado entre conjunto de treino e teste. O conjunto de treino então é apresentado ao indutor, levando em consideração os rótulos dos exemplos. Após a geração do modelo, o indutor pode rotular os exemplos do conjunto de teste e então pode-se avaliar o desempenho do classificador gerado.

Há diversos algoritmos para classificação e regressão. Esses algoritmos são usualmente organizados entre métodos baseados em distância, métodos probabilísticos, métodos baseados em procura ou métodos baseados em otimização (FACELI et al., 2011). Neste trabalho serão utilizados métodos baseados em otimização como o algoritmo *Support Vector Machine* e as Redes Neurais Artificiais.

2.1.2.1 Classificação de Imagens

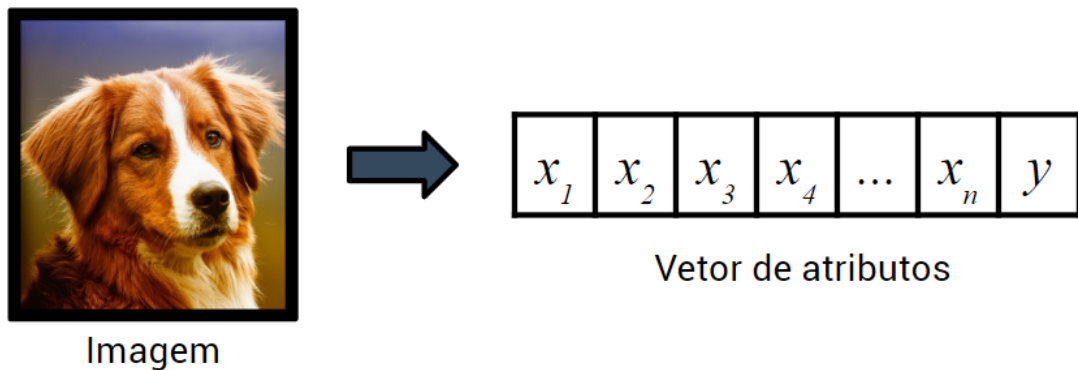
A classificação de imagens pode ser vista como um caso especial da classificação. Neste tipo de classificação o objetivo é prever corretamente a qual classe a imagem pertence. Assim, classificar imagens pode ser uma tarefa muito útil em algumas aplicações, como por exemplo, detecção de doenças (VEMURI et al., 2008), reconhecimento facial (VARGAS; PAES; VAS-

CONCELOS, 2016), análise de dados de satélite (PENATTI K. NOGUEIRA, 2015), dentre outras aplicações.

2.1.2.1.1 Extração de Características

Mas na classificação, comumente, os conjuntos de dados são compostos pelas características dos exemplos, a fim de identificar padrões, regras e *etc*, para aprender e poder rotular exemplos ainda não vistos. No entanto, com imagens não é direta. Geralmente, para se trabalhar com imagens é necessário utilizar descritores de imagens para extrair características. Assim as imagens poderão ser representadas como exemplos munidos de n atributos e podem formar um conjunto de dados. Esse processo é exemplificado na Figura 4.

Figura 4: Exemplo prático da extração de características.



Fonte: O autor.

Diversos algoritmos para descrever imagens foram propostos. O Algoritmo *Global Color Histogram* - GCH (SWAIN; BALLARD, 1991) é um dos descritores mais famosos e considera o histograma da imagem, a partir da contagem de ocorrência das cores. Um outro algoritmo muito utilizado na literatura é o *Border Interior Pixel Classification* - BIC (STEHLING; NASCIMENTO; FALCÃO, 2002), classificando cada *pixel* como borda ou interior e em seguida, cria histogramas para os dois tipos de *pixels*. Atualmente, com o uso das *Convolutional Neural Networks*, a extração de características está acoplada e alcançando ótimos resultados (Lecun et al., 1998) devido ao fato da própria rede ter uma fase de extração de características e usar diversos filtros detectores de padrões, seguido pela fase de classificação (opcional).

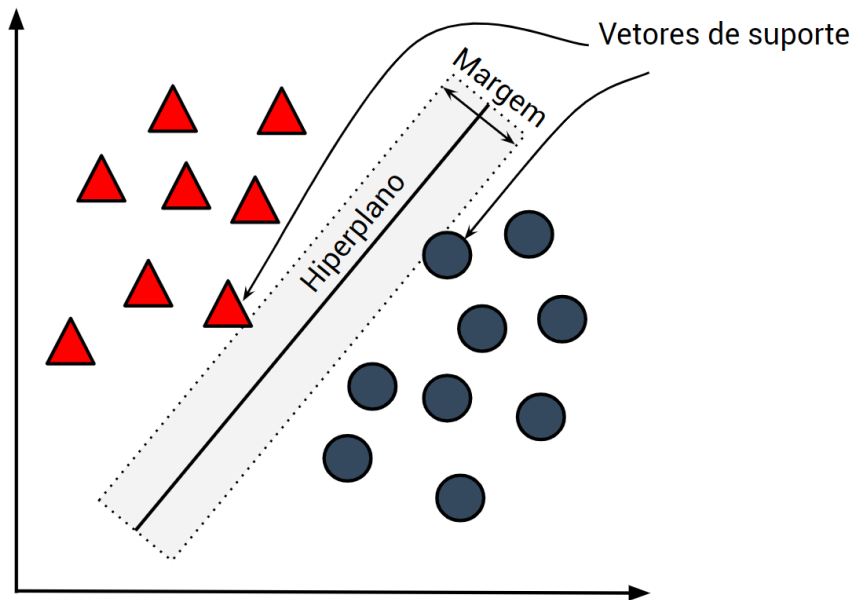
2.1.2.2 Support Vector Machine (SVM)

O *Support Vector Machine* é um método proposto por (CORTES; VAPNIK, 1995) para classificação binária. (CORTES; VAPNIK, 1995) resumem a ideia do método como: vetores de entrada não-lineares mapeados para o espaço de atributos de alta dimensão, onde é construída uma superfície de decisão com uma alta capacidade de generalização.

Em outras palavras, deve-se encontrar um hiperplano ótimo, onde teremos uma fronteira de decisão que possua menor erro, mas maior margem de separação entre os exemplos das diferentes classes. Para isso, são usados uma pequena quantidade de dados de treinamento chamados *Support Vectors* (SVs), que ajudarão na determinação desta margem (LORENA; CARVALHO, 2007).

Na Figura 5 há um exemplo visual do fundamento teórico do SVM, onde temos um hiperplano separando objetos diferentes, com uma determinada margem. No SVM linear com margens rígidas queremos encontrar um hiperplano que separe corretamente os exemplos com a maior margem possível. Na Figura 5 também é possível visualizar os SVs (vetores de suporte, em português), que são os exemplos mais próximos da fronteira de decisão.

Figura 5: Exemplo do cenário de um SVM.



Fonte: O autor.

2.1.2.2.1 SVM com Margens Rígidas

Seja então um conjunto com n exemplos m -dimensionais $\mathbf{x}_i = (x_1, x_2, \dots, x_m)$ com $\mathbf{x}_i \in \mathbb{R}^m$ com classes $y_i \in \{+1, -1\}$. Um hiperplano que separe os dados será definido por:

$$\mathbf{w} \cdot \mathbf{x} + b = 0 \quad (2.2)$$

Onde $\mathbf{w} \cdot \mathbf{x}$ é o produto escalar entre $\mathbf{w}$ e $\mathbf{x}$, sendo $\mathbf{w}$ um vetor normal ao hiperplano e b é uma constante. Essa equação divide o espaço em duas regiões: $\mathbf{w} \cdot \mathbf{x} + b > 0$ e $\mathbf{w} \cdot \mathbf{x} + b < 0$ e

com isso, temos:

$$y(\mathbf{x}_i) = \begin{cases} +1 & \text{se } \mathbf{w} \cdot \mathbf{x}_i + b > 0 \\ -1 & \text{se } \mathbf{w} \cdot \mathbf{x}_i + b < 0 \end{cases} \quad (2.3)$$

E então, pode-se definir uma função sinal $g(x) = \text{sin}al(f(\mathbf{x})) = \text{sin}al(\mathbf{w} \cdot \mathbf{x} + b)$, o que leva à classificação +1 se $f(\mathbf{x}) > 0$ e -1 se $f(\mathbf{x}) < 0$. A partir disso, se existir um par $(\mathbf{w}, b)$ tal que a função sinal $g(\mathbf{x})$ consiga classificar corretamente todos os exemplos $\mathbf{x}_i$, então o conjunto é dito ser linearmente separável (LORENA; CARVALHO, 2007).

Por definição, um hiperplano é ótimo se possui grande margem ρ . Essa margem ρ é calculada por $\rho = \frac{2}{\|\mathbf{w}\|}$ e sendo assim, teremos uma maior margem tendo um $\|\mathbf{w}\|$ menor. Dessa forma, é formulado um problema de otimização com a restrição de não haver exemplos na margem de separação encontrada, definindo assim uma SVM com margens rígidas (LORENA; CARVALHO, 2007). O processo de otimização é dado pelas Equações 2.4 e 2.5.

$$\text{Minimizar}_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad (2.4)$$

$$\text{Com as restrições : } y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 \geq 0, \forall i = 1, \dots, n \quad (2.5)$$

Como nesse problema a função objetivo a ser minimizada é convexa e os pontos que satisfazem as restrições formam um conjunto convexo, o problema possui um único mínimo global. Para solucionar esse problema é introduzida a função Lagrangiana, que vai englobar as restrições à função objetivo, associadas a parâmetros denominados multiplicadores de Lagrange α_i como na Equação 2.6 (LORENA; CARVALHO, 2007).

$$L(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i (y_i (\mathbf{w} \cdot \mathbf{x}_i + b) - 1) \quad (2.6)$$

Dessa forma, deve-se minimizar a função Lagrangiana, o que implica maximizar α_i e minimizar $\mathbf{w}$ e b . Resolvendo algumas equações chegamos a duas novas expressões, dadas pelas Equações 2.7 e 2.8.

$$\sum_{i=1}^n \alpha_i y_i = 0 \quad (2.7)$$

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \quad (2.8)$$

Fazendo as devidas substituições, o problema de otimização é reformulado como:

$$\text{Maximizar}_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) \quad (2.9)$$

$$\text{Com as restrições: } \begin{cases} \alpha_i \geq 0, & \forall i = 1, \dots, n \\ \sum_{i=1}^n \alpha_i y_i = 0 \end{cases} \quad (2.10)$$

O problema original é dito ser da forma primal, já a nova formulação é da forma dual. A forma dual se destaca por ser mais simples, permite representar problemas por produtos internos e utiliza apenas os dados de treinamento e seus rótulos (LORENA; CARVALHO, 2007). Assim, sendo α^* a solução do problema dual e $\mathbf{w}^*$ e b^* para a forma primal, depois de encontrar o valor de α^* é possível encontrar o valor de $\mathbf{w}^*$, enquanto que o valor de b^* é obtido pelo valor de α^* e pelas condições de Kühn-Tucker. Portanto, para o problema dual formulado, temos agora (LORENA; CARVALHO, 2007):

$$\alpha_i^* (y_i (\mathbf{w}^* \cdot \mathbf{x}_i + b^*) - 1) = 0, \forall i = 1, \dots, n \quad (2.11)$$

Nesta equação α_i^* é diferente de 0 para os exemplos que estão mais próximos do hiperplano separador, isto é, sobre as margens. Esses exemplos são os SVs, que são considerados os dados que trazem mais informação do conjunto e são somente eles que participam na determinação do hiperplano ótimo (LORENA; CARVALHO, 2007).

Fazendo as novas devidas substituições é obtido uma nova equação para determinar b^* , conforme Equação 2.12, em que SV determina o conjunto de *Support Vectors* e n_{SV} o número de *Support Vectors*.

$$b^* = \frac{1}{n_{SV}} \sum_{\mathbf{x}_j \in SV} \frac{1}{y_j} - \mathbf{w}^* \cdot \mathbf{x}_j \quad (2.12)$$

Novamente, fazendo as devidas substituições em relação a $\mathbf{w}^*$, obtemos um novo cálculo de b^* descrito pela Equação 2.13.

$$b^* = \frac{1}{n_{SV}} \sum_{\mathbf{x}_j \in SV} \left(\frac{1}{y_j} - \sum_{\mathbf{x}_i \in SV} \alpha_i^* y_i \mathbf{x}_i \cdot \mathbf{x}_j \right) \quad (2.13)$$

Por fim, temos como resultado o classificador $g(x)$ dado pela Equação 2.14, em que sinál é a função sinal, $\mathbf{w}^*$ é dado pela Equação 2.8, obtida anteriormente, e b^* pela Equação 2.13.

$$g(\mathbf{x}) = \text{sinál}(f(\mathbf{x})) = \text{sinál} \left(\sum_{\mathbf{x}_i \in SV} y_i \alpha_i^* \mathbf{x}_i \cdot \mathbf{x} + b^* \right) \quad (2.14)$$

2.1.2.2.2 SVM com Margens Suaves

No entanto, esse é um método ideal para problemas linearmente separáveis, mas aplicações reais podem conter ruídos ou *outliers*. Para contornar essa situação, deve-se admitir a ocorrência de erros de classificação e são introduzidas as variáveis de folga ξ , originando as SVMs com margens suaves. Todo o processo de formulação do problema de otimização é modificado para considerar ξ , com isso as restrições impostas ao problema de otimização primal é dado pela Equação 2.15 (LORENA; CARVALHO, 2007).

$$y_i (\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \forall i = 1, \dots, n \quad (2.15)$$

Então, o objetivo de minimização é reformulado como descrito pela Equação 2.16, em que C é um termo de regularização que considera um peso à minimização dos erros no conjunto de dados em relação à minimização da complexidade do modelo (LORENA; CARVALHO, 2007).

$$\text{Minimizar}_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \left(\sum_{i=1}^n \xi_i \right) \quad (2.16)$$

Ao final, o classificador obtido é igual ao obtido pelo SVM de margens rígidas, porém o cálculo de α_i^* é dado pela Equação 2.17 e as restrições da Equação 2.18.

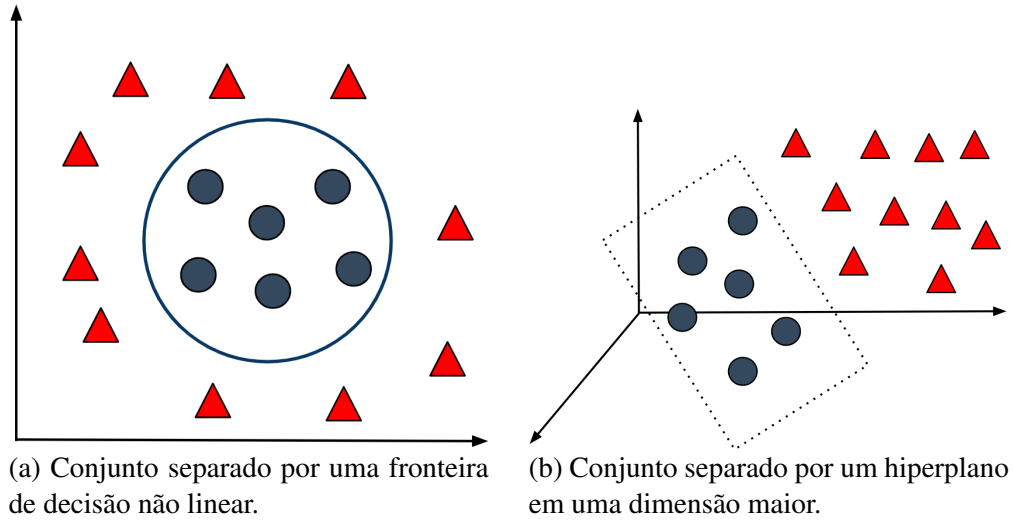
$$\text{Maximizar}_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) \quad (2.17)$$

$$\text{Com as restrições: } \begin{cases} 0 \leq \alpha_i \leq C, & \forall i = 1, \dots, n \\ \sum_{i=1}^n \alpha_i y_i = 0 \end{cases} \quad (2.18)$$

2.1.2.2.3 SVM Não-linear

Em outros casos não é possível dividir os dados de forma satisfatória por um hiperplano como exemplificado na Figura 6a, onde o problema não é de natureza linear. Porém, uma SVM pode ser generalizada para esta tarefa. Dessa forma os dados de treinamento devem ser levados para um espaço de maior dimensão e então, os dados serão linearmente separáveis como pode ser visualizado na Figura 6b. Isto é, se os dados pertencem a um espaço de entradas do $\mathbb{R}^m$, esses dados serão mapeados para um espaço de características do $\mathbb{R}^{m+1}$ por meio de um determinado mapeamento, e assim poderá ser aplicado uma SVM Linear com Margens Suaves (LORENA; CARVALHO, 2007).

Figura 6: Exemplo de problema não linear.



Fonte: O autor.

Seja então $\Phi : X \rightarrow \mathfrak{S}$ um mapeamento, em que X é o espaço de entradas e $\mathfrak{S}$ é o espaço de características, existem diversos mapeamentos que podem ser aplicados. Por exemplo:

$$\Phi(\mathbf{x}) = \Phi(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2) \quad (2.19)$$

Sendo assim, pode-se definir uma função *kernel* K , como uma função que recebe dois pontos $\mathbf{x}_i$ e $\mathbf{x}_j$ do espaço de atributos e calcula o produto escalar desses dados no espaço de características. Alguns exemplos das funções mais utilizadas, são apresentadas na Tabela 2, em que $K(\mathbf{x}_i, \mathbf{x}_j) = \Phi(\mathbf{x}_i) \cdot \Phi(\mathbf{x}_j)$ (LORENA; CARVALHO, 2007).

Tabela 2: Tabela de funções de *kernel*.

<i>Kernel</i>	Função $K(\mathbf{x}_i, \mathbf{x}_j)$
Polinomial	$(\delta(\mathbf{x}_i \cdot \mathbf{x}_j) + \kappa)^d$
Gaussiano (RBF)	$\exp(-\sigma \ \mathbf{x}_i - \mathbf{x}_j\ ^2)$
Sigmoidal	$\tanh(\delta(\mathbf{x}_i \cdot \mathbf{x}_j) + \kappa)$

Fonte: Adaptado de (LORENA; CARVALHO, 2007).

Sendo assim, agora o problema de otimização para obter o classificador de uma SVM Não-linear é reformulado novamente, considerando os mapeamento das entradas com as funções de *kernel* K , conforme as Equações 2.20 e 2.21.

$$\text{Maximizar}_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (\Phi(\mathbf{x}_i) \cdot \Phi(\mathbf{x}_j)) \quad (2.20)$$

$$\text{Com as restrições: } \begin{cases} 0 \leq \alpha_i \leq C, & \forall i = 1, \dots, n \\ \sum_{i=1}^n \alpha_i y_i = 0 \end{cases} \quad (2.21)$$

E portanto, o classificador agora será dado pela Equação 2.22, em que b^* é calculado pela Equação 2.23.

$$g(\mathbf{x}) = \text{sin}(\mathbf{f}(\mathbf{x})) = \text{sin} \left(\sum_{\mathbf{x}_i \in \text{SV}} \alpha_i^* y_i \Phi(\mathbf{x}_i) \cdot \Phi(\mathbf{x}) + b^* \right) \quad (2.22)$$

$$b^* = \frac{1}{n_{\text{SV}: \alpha^* < C}} \sum_{\mathbf{x}_j \in \text{SV}: \alpha_j^* < C} \left(\frac{1}{y_j} - \sum_{\mathbf{x}_i \in \text{SV}} \alpha_i^* y_i \Phi(\mathbf{x}_i) \cdot \Phi(\mathbf{x}_j) \right) \quad (2.23)$$

2.1.2.2.4 SVM para Problemas Multi-classes

A SVM foi desenvolvida para problemas de classificação binária, mas pode ser adaptada para problemas multi-classes. Existem duas abordagens comumente utilizadas:

- a primeira é a “um-contra-todos”, neste caso dado a existência de k classes, serão gerados k classificadores, cada um separa uma classe i das $k - 1$ restantes e a classe de um novo exemplo é o índice da SVM que produzir a maior saída;
- na segunda abordagem, “todos-contra-todos”, são produzidos classificadores para separação de cada classe i de outra j , em que $i, j = 1, \dots, k$ e $i \neq j$ e neste caso, ocorre uma espécie de votação, onde a classe de um novo exemplo é dado pelo voto de maioria entre as SVMs geradas (LORENA; CARVALHO et al., 2003).

2.1.2.2.5 Vantagens e Desvantagens das SVMs

Entre as vantagens das SVMs estão sua boa capacidade de generalização, robustez para dados de grandes dimensões, conexidade na função objetivo (na SVM há a otimização de uma função quadrática e portanto, possui apenas um mínimo global) e o tempo de treinamento é geralmente menor que outros algoritmos de AM. Como desvantagem, está sua complexidade computacional (ordem n^2 a n^3), a velocidade de classificação pode ser um pouco demorada e não há como interpretar o modelo gerado (LORENA; CARVALHO et al., 2003).

2.1.2.3 Redes Neurais Artificiais (RNAs)

Segundo (HAYKIN, 2011) as Redes Neurais Artificiais foram motivadas pelo reconhecimento de que o cérebro humano processa informações da forma que um computador digital

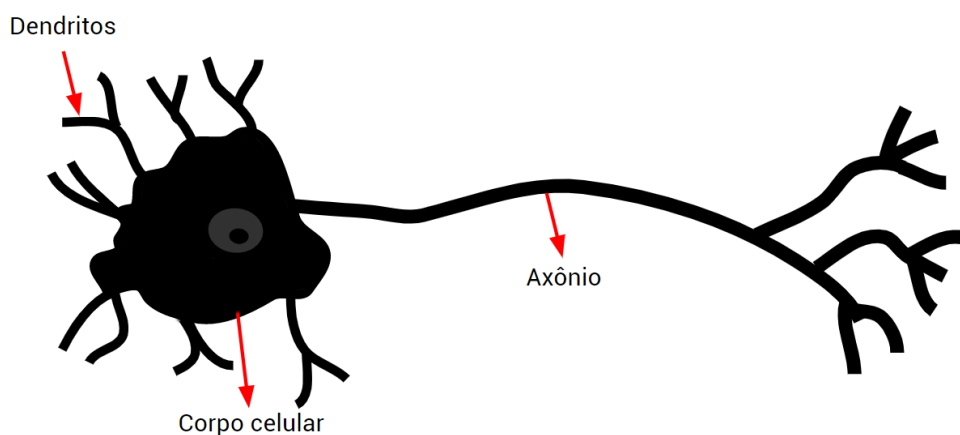
faz. (HAYKIN, 2011) também define que o cérebro é um computador, ou sistema de processamento de informação, altamente complexo, não-linear, paralelo e que possui bilhões de neurônios. Dessa forma, o cérebro possui a capacidade de organizar seus neurônios para realizar tarefas como reconhecer padrões, percepção *etc*, muito mais rapidamente que os computadores atuais.

2.1.2.3.1 Neurônio Biológico

Biologicamente, esses neurônios são formados por três partes: dendrito, corpo celular e axônio (HAYKIN, 2011). Na Figura 7, a estrutura de um neurônio pode ser visualizada, onde um exemplo, muito simples é apresentado. A seguir, seguem os detalhes dessas três partes:

- dendritos: recebem os estímulos transmitidos pelos outros neurônios;
- corpo celular: coleta e combina informações vindas dos outros neurônios;
- axônio: transmite os estímulos para outras células.

Figura 7: Neurônio biológico.



Fonte: O autor.

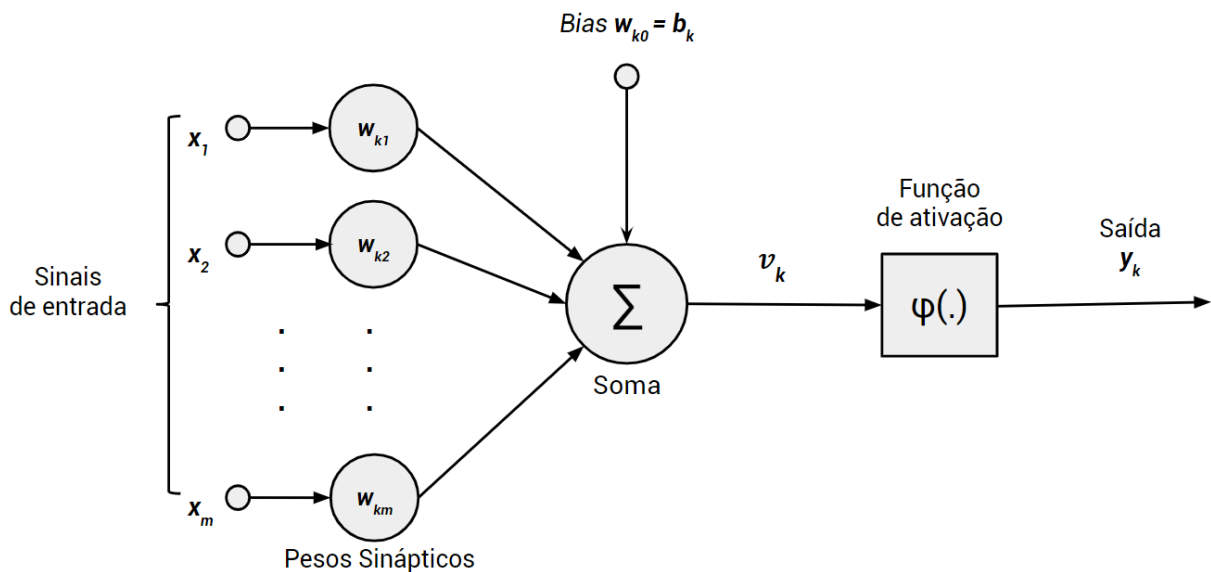
Assim, a comunicação entre os neurônios é feita por meio de sinapses de seus dendritos. Isto é, os dendritos captam os estímulos recebidos em um determinado período de tempo e os transmitem ao corpo do neurônio, onde são processados. Quando esses estímulos atingem um limiar, um sinal é propagado no axônio e esse sinal é transmitido para outros neurônios (HAYKIN, 2011).

2.1.2.3.2 Neurônio Artificial

Uma rede neural pode ser vista como um processador maciçamente paralelamente distribuído, constituído de processamento simples que têm a capacidade de armazenar conhecimento adquirido pela experiência e após isso, tornar esse conhecimento disponível para que possa ser usado. Sua semelhança ao cérebro está atrelado ao fato de que o conhecimento da rede é obtido por um processo de aprendizagem e o conhecimento adquirido é armazenado pela conexão dos neurônios ou pesos sinápticos. Esse procedimento de aprendizagem é chamado de algoritmo de aprendizagem, cuja função é modificar os pesos sinápticos da rede de uma forma ordenada para alcançar um objetivo (HAYKIN, 2011).

Dessa forma, um neurônio é uma unidade de processamento de informação que é fundamental para a operação de uma rede neural e o modelo neuronal de McCulloch–Pitts (MCCULLOCH; PITTS, 1943) pode ser visualizado na Figura 8. Esse modelo de neurônio é formado por três elementos básicos: um conjunto de sinapses, um somador e uma função de ativação. Cada elemento do conjunto de sinapses representa um peso ou força, isto é, um sinal x_j na entrada da sinapse j conectada ao neurônio k é multiplicado pelo peso sináptico w_{kj} . O somador é responsável por somar os sinais de entrada ponderado pelas respectivas sinapses do neurônio. Já a função de ativação serve para restringir a amplitude da saída de um neurônio. Além disso, esse modelo inclui um *bias*, que tem como objetivo aumentar ou diminuir a entrada líquida da função de ativação, dependendo se ele é positivo ou negativo, respectivamente (HAYKIN, 2011).

Figura 8: Modelo de um neurônio.



Fonte: O autor.

Matematicamente, sendo $W = \{w_{k0}, w_{k1}, \dots, w_{km}\}$ os pesos sinápticos do neurônio k ,

$X = \{x_0, x_1, \dots, x_m\}$ as entradas apresentadas ao neurônio e b um *bias* (HAYKIN, 2011). A entrada do limitador abrupto ou o campo local induzido do neurônio é:

$$v_k = \sum_{i=0}^m w_{ki} x_i \quad (2.24)$$

2.1.2.3.3 Funções de Ativação

A partir de $v = v_k$, temos as funções de ativação denotadas por $\varphi(v)$ (HAYKIN, 2011). Por exemplo:

- degrau: assume o valor 1, se a entrada do limitador abrupto for maior que 0 ou assume 0, caso contrário. O gráfico dessa função pode ser visualizado na Figura 9a;

$$\varphi(v) = \begin{cases} 1 & \text{se } v \geq 0 \\ 0 & \text{se } v < 0 \end{cases} \quad (2.25)$$

- sigmóide: é uma função estritamente crescente com um balanceamento entre um comportamento linear e não-linear no intervalo $[0, 1]$ e um exemplo é a função logística definida pela Equação 2.26, sendo α uma constante. O gráfico dessa função pode ser visualizado na Figura 9b;

$$\varphi(v) = \frac{1}{1 + e^{-\alpha v}} \quad (2.26)$$

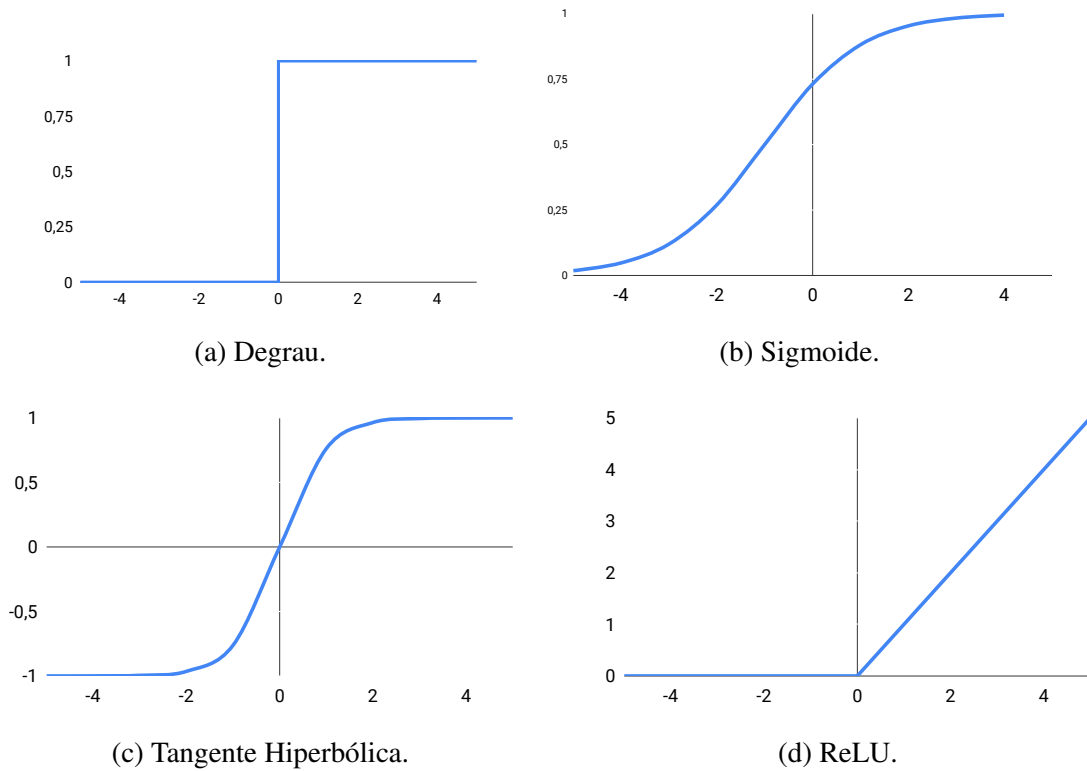
- tangente hiperbólica: neste caso, a função assume valores positivos e negativos no intervalo $[-1, 1]$ e o gráfico dessa função pode ser visualizado na Figura 9c;

$$\varphi(v) = \tanh(v) = \frac{1 - e^{-v}}{1 + e^{-v}} \quad (2.27)$$

- ReLU: a ReLU ou *Rectified Linear Units* (NAIR; HINTON, 2010) é uma das funções de ativação mais utilizadas no contexto de *deep learning*. Essa função é simplesmente o máximo entre 0 e o valor de v . Assim, o gráfico dessa função pode ser visualizado na Figura 9d;

$$\varphi(v) = \max(0, v) \quad (2.28)$$

Figura 9: Gráficos das funções de ativação.



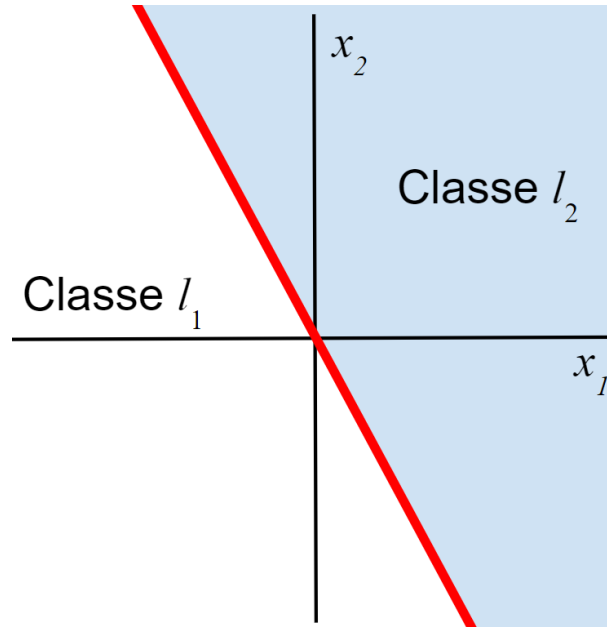
Fonte: O autor.

2.1.2.3.4 Perceptron

O Perceptron de Rosenblatt ([ROSENBLATT, 1958](#)) ou simplesmente Perceptron, é um modelo de neurônio não-linear construído em torno do modelo de McCulloch–Pitts. Assim, o objetivo do Perceptron é classificar corretamente o conjunto de estímulos aplicados externamente $x_1, x_2, \dots, x_n$ em uma das classes l_1 e l_2 . A regra da decisão para a classificação é atribuir o ponto x_i à classe l_1 se a saída do perceptron y_i for +1 e à classe l_2 se for -1 ([HAYKIN, 2011](#)).

A equação $\sum_{i=1}^n w_i x_i + b$ gera um hiperplano, o que na forma simples do perceptron, gera duas regiões separadas. No caso de duas variáveis x_1 e x_2 , a fronteira de decisão será uma reta e assim, um ponto (x_1, x_2) que está acima da fronteira de decisão será atribuído à classe l_1 , se estiver abaixo da fronteira de decisão será atribuído à classe l_2 , conforme visto na Figura 10 ([HAYKIN, 2011](#)).

Figura 10: Hiperplano gerado por um perceptron para UM problema de duas variáveis.



Fonte: O autor.

Mas isso é idealmente para dados linearmente separáveis. Para tentar contornar a situação, os pesos sinápticos podem ser adaptados a cada iteração com um algoritmo de correção de erro (HAYKIN, 2011). Sendo assim, seja o *bias* como $b(n) = 1$ e n como o identificador de iteração do algoritmo. Vamos definir o vetor de entrada como:

$$\mathbf{x}(n) = [+1, x_1(n), x_2(n), \dots, x_m(n)]^T \quad (2.29)$$

Também, vamos definir o vetor de peso como:

$$\mathbf{w}(n) = [b, w_1(n), w_2(n), \dots, w_m(n)]^T \quad (2.30)$$

Assim, a combinação linear da saída pode ser escrita na forma compacta como:

$$v(n) = \sum_{i=0}^m w_i(n)x_i(n) = \mathbf{w}^T(n)\mathbf{x}(n) \quad (2.31)$$

Além disso, seja d , a resposta desejada quantificada definida por:

$$d(n) = \begin{cases} +1 & \text{se } x(n) \in l_1 \\ -1 & \text{se } x(n) \in l_2 \end{cases} \quad (2.32)$$

Então, a atualização do vetor de peso $\mathbf{w}(n)$ de acordo com a regra de aprendizagem por correção de erro será dado pela equação 2.33, onde η é a taxa de aprendizado, com $0 < \eta \leq 1$

e a parcela $d(n) - y(n)$ é a diferença da classe desejada e a classe prevista, representando um sinal de erro (HAYKIN, 2011).

$$\mathbf{w}(n + 1) = \mathbf{w}(n) + \eta[d(n) - y(n)]x(n) \quad (2.33)$$

2.1.2.3.5 Treinamento de uma RNA

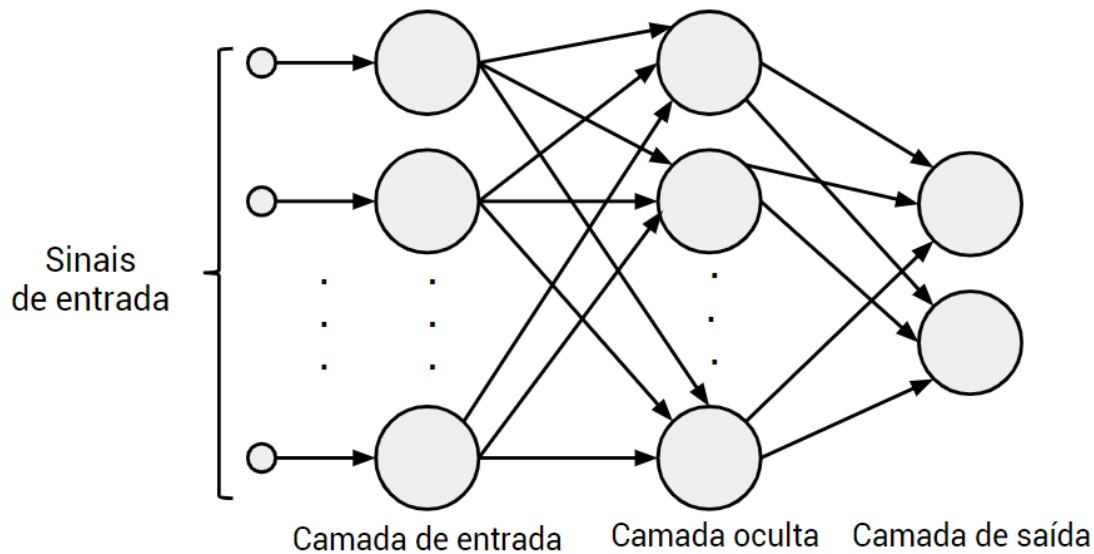
Outros algoritmos de treinamento para RNAs foram propostos além do algoritmo de correção do erro (FACELI et al., 2011). Esses algoritmos de treinamento seguem paradigmas diferentes e são divididos entre quatro grupos:

- correção de erro: comum no aprendizado supervisionado, os pesos da rede são ajustados de forma a diminuir os erros cometidos pela RNA.
- hebbiano: comum no aprendizado não supervisionado e se baseia na regra de Hebb, em que se dois neurônios estão simultaneamente ativos, a conexão entre eles deve ser reforçada.
- competitivo: também comum no aprendizado não supervisionado, onde ocorre uma espécie de competição entre os neurônios para definir quais neurônios terão que ajustar seus pesos.
- termodinâmico (Boltzmann): se baseia em algoritmos estocásticos baseados em princípios observados na metalurgia.

2.1.2.3.6 *Multilayer Perceptron*

Como visto anteriormente, o Perceptron possui limitações como a premissa dos dados serem linearmente separáveis para que funcione corretamente. Na vida real, problemas assim são quase inexistentes e portanto, uma solução criada são os perceptrons de múltiplas camadas ou *Multilayer Perceptron* (MLP). Nesse modelo de rede neural cada neurônio inclui uma função de ativação não-linear que é diferenciável, como a Sigmóide. Além disso essa rede dispõe de uma ou mais camadas ocultas e, geralmente, possui alto nível de conectividade com as camadas totalmente conectadas, isto é, cada neurônio de uma camada está conectado a todos os neurônios da camada anterior. Na Figura 11 temos a representação da arquitetura de uma rede MLP com uma camada de entrada, uma camada oculta e uma camada de saída (HAYKIN, 2011).

Figura 11: Modelo de uma rede MLP.



Fonte: O autor.

Nas redes MLP, cada neurônio realiza uma função específica. Isto é, a função de um neurônio de uma determinada camada é uma combinação das funções realizadas pelos neurônios da camada anterior que estão conectados a ele. Por exemplo, na camada de entrada, cada neurônio aprende uma função que define um hiperplano para separar os dados. Na camada seguinte, cada neurônio combina um grupo de hiperplanos definidos pelos neurônios da camada anterior, formando regiões convexas. E os neurônios da próxima camada combinam um subconjunto das regiões convexas em regiões de formato arbitrário (FACELI et al., 2011).

Ainda, na rede MLP, cada neurônio da camada de saída é associado a uma das classes definidas pelo problema. Assim, pode-se representar os valores de saída por um vetor $Y = [y_1, y_2, \dots, y_k]^T$, em que k é o número de classes. Portanto, o vetor de resposta desejado para um exemplo com classe y_i é um vetor Y com o valor 1 na posição associada a classe y_i e 0 nas outras posições. Com isso, os acertos e erros são verificados comparando o vetor de saída da camada de saída e o vetor de saída desejada para o exemplo. Em alguns casos, em vez do valor 1, é considerado o valor mais alto entre os outros valores do vetor.

Comumente, o treinamento dessa rede é baseada no algoritmo de retropropagação do erro, conhecido também como *backpropagation* (LINNAINMAA, 1970). A aprendizagem por retropropagação consiste em dois passos por meio das diferentes camadas da rede: um passo para frente (*forward*) e um passo para trás, a retropropagação (*backward*). Na fase *forward*, os pesos da rede são mantidos fixos e os valores de saída são propagados, camada por camada, até chegar na saída e então compara-se o valor de saída e o valor de saída desejado para esse neurônio, a diferença será o erro cometido pela rede para o exemplo. No passo para trás, o erro da saída é propagado pela rede, camada por camada, mas dessa vez no sentido contrário (camada de saída até a primeira camada oculta) e são feitos ajustes nos pesos sinápticos da rede.

([FACELI et al., 2011](#)).

2.1.2.3.7 Vantagens e Desvantagens das RNAs

Dentre as vantagens das RNAs estão sua alta capacidade de generalização, auto-aprendizado, sua capacidade de adaptação e em geral, bons resultados. Como desvantagem, tem-se a dificuldade de interpretação do modelo gerado, a escolha dos parâmetros e o treinamento, que muitas vezes, pode ser demorado em relação a outros algoritmos ([HAYKIN, 2011](#)).

2.1.3 Deep Learning

Com o crescimento do poder de processamento dos computadores atuais e a quantidade de dados disponíveis, houve a ascensão do *deep learning*, também conhecido como aprendizado profundo. O *deep learning* permite que modelos computacionais compostos por múltiplas camadas de processamento possam aprender muitos níveis de abstração dos problemas, indo além na imitação do ser humano em sua capacidade de aprender ([LECUN; BENGIO; HINTON, 2015](#)).

Dessa forma, por si só, o *deep learning* é considerado uma subárea do Aprendizado de Máquina, sendo uma área de estudo crescente entre pesquisadores. Recentemente, com o *deep learning* houveram muitos avanços em reconhecimento de faces ([PARKHI et al., 2015](#)), objetos ([EITEL et al., 2015](#)) e fala ([HINTON et al., 2012](#)), processamento de linguagem natural ([COLLOBERT; WESTON, 2008](#)), dentre outras tarefas.

No *deep learning* utiliza-se o algoritmo de retropropagação do erro para descobrir estruturas complexas em grandes conjuntos de dados, assim as redes alteram seus parâmetros internos para computar a representação em cada camada para a representação da camada anterior ([LECUN; BENGIO; HINTON, 2015](#)).

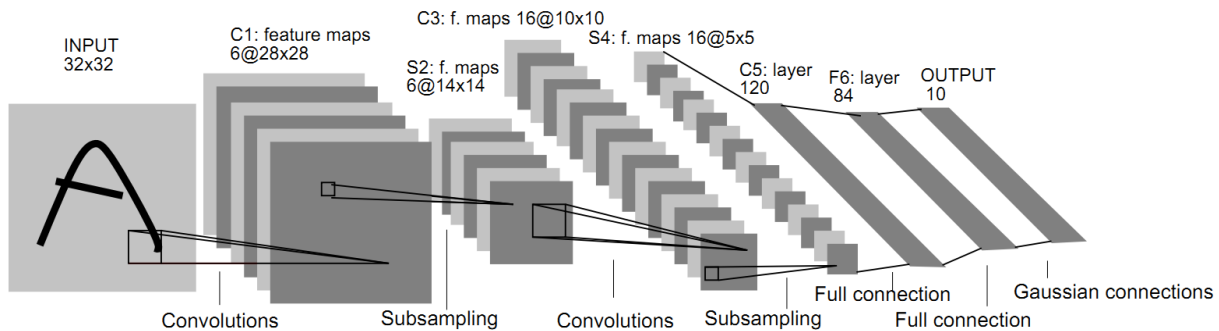
Nas aplicações de *deep learning* como o reconhecimento de faces e objetos, geralmente, são utilizadas *Convolutional Neural Networks* que serão detalhadas na Seção 2.1.3.1. Já para reconhecimento de fala são utilizadas as Redes Neurais Recorrentes ([HINTON et al., 2012](#)).

2.1.3.1 Convolutional Neural Networks (CNNs)

As *Convolutional Neural Networks* foram introduzidas por ([Lecun et al., 1998](#)) e diferente das redes neurais comuns, as CNNs se inspiram no processo biológico do processamento de dados visuais. Essas redes aplicam filtros em dados visuais, mas mantendo a relação de vizinhança entre os *pixels* da imagem ao longo do processamento da rede ([VARGAS; PAES; VASCONCELOS, 2016](#)).

Além disso, essas redes possuem múltiplas partes com funções diferentes, possuindo uma fase de extração de características das imagens e uma para classificação. Em geral, a fase de extração de características é composta por camadas de convolução e de *pooling*, e na de classificação, uma camada de neurônios completamente conectados (MLP) (VARGAS; PAES; VASCONCELOS, 2016). Na Figura 12, temos um exemplo de arquitetura da CNN LeNet-5, que possui 3 camadas de convolução, 2 de *pooling* (ou *subsampling*), 1 camada totalmente conectada, seguida por uma camada de saída.

Figura 12: Exemplo de arquitetura da CNN LeNet-5.



Fonte: (Lecun et al., 1998).

2.1.3.1.1 Convolução

Na camada de convolução, as imagens representadas por uma matriz de *pixels*, passam pelo processo de convolução (discreta), que é uma transformação linear que preserva a noção de ordem, esparsa (algumas unidades de entradas contribuem para uma unidade de saída) e reusa parâmetros (os mesmos pesos são aplicados em várias localizações na entrada) (DUMOULIN; VISIN, 2016).

Neste processo, a matriz de *pixels* da imagem original é chamada de *input feature maps*. Além disso, existe a matriz de filtro (ou *kernel*) e a matriz resultante do processo de convolução, chamada matriz de *output feature maps* (DUMOULIN; VISIN, 2016).

Dessa forma, para aplicar a convolução, a matriz de filtro e a matriz *input feature maps* são sobrepostas por meio de passos denominados *strides* e em cada passo, o produto entre cada elemento das matrizes sobrepostas são somados e esse resultado é adicionado em uma posição correspondente na matriz de *output feature maps*. Existem diversos filtros, com tamanhos diferentes, alguns exemplos são os filtros detectores de borda, ou os que aplicam efeitos como *blur*, *etc* (DUMOULIN; VISIN, 2016).

Para apresentar esse processo, considere um exemplo simples com uma matriz de *pixels* 5×5 representando uma imagem com 1 canal de cor e seus *pixels* como 0 ou 1, uma matriz de

filtro 3×3 , $stride = 1$ e uma matriz de saída 3×3 , definidas como na Equação 2.34.

$$\begin{array}{ccc}
 \text{Matriz de Pixels} & & \text{Matriz de Filtro} \quad \text{Matriz de Saída} \\
 \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} & \begin{bmatrix} - & - & - \\ - & - & - \\ - & - & - \end{bmatrix}
 \end{array} \quad (2.34)$$

Assim, na primeira iteração de processamento da convolução, é calculado $soma(0 \cdot 1, 0 \cdot 0, 1 \cdot 1, 0 \cdot 0, 0 \cdot 0, 1 \cdot 1, 0 \cdot 0, 0 \cdot 1, 1 \cdot 0) = 2$ e portanto esse é o primeiro resultado da matriz de saída, como pode ser observado na Equação 2.35. E então, o processo se repete até percorrermos toda a matriz de *pixels*, de acordo com os *strides*.

$$\begin{array}{ccc}
 \text{Matriz de Pixels} & & \text{Matriz de Filtro} \quad \text{Matriz de Saída} \\
 \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} & \begin{bmatrix} 2 & - & - \\ - & - & - \\ - & - & - \end{bmatrix}
 \end{array} \quad (2.35)$$

2.1.3.1.2 Pooling

A operação de *Pooling* é responsável por reduzir o tamanho do mapa de *features* usando alguma medida para sumarizar regiões das imagens. Algumas medidas comuns são o *AVG Pooling* e o *Max Pooling* (DUMOULIN; VISIN, 2016).

- *AVG Pooling*: nesse tipo de *pooling*, o valor de saída é a média dos valores de uma vizinhança definido pela grade de *pooling* (DUMOULIN; VISIN, 2016);

Por exemplo, seja uma matriz de *pixels* 5×5 , uma operação de *pooling* 3×3 , $stride = 1$ e uma matriz de saída. A primeira iteração será como a Equação 2.36, onde o valor do primeiro elemento da matriz de saída é dado pela média de todos os elementos da grade de *pooling* 3×3 na matriz original isto é $media(3, 2, 1, 0, 4, 2, 3, 1, 1) = 1.9$.

$$\begin{array}{ccc}
 \text{Matriz de Pixels} & & \text{Matriz de AVG Pooling} \\
 \begin{bmatrix} 3 & 2 & 1 & 0 & 1 \\ 0 & 4 & 2 & 3 & 2 \\ 3 & 1 & 1 & 0 & 0 \\ 4 & 4 & 1 & 2 & 0 \\ 1 & 3 & 1 & 0 & 1 \end{bmatrix} & & \begin{bmatrix} 1.9 & - & - \\ - & - & - \\ - & - & - \end{bmatrix}
 \end{array} \quad (2.36)$$

- *Max Pooling*: já neste tipo de *pooling*, o valor de saída é simplesmente o valor máximo dos valores presentes em uma dada vizinhança na grade de *pooling* (DUMOULIN; VISIN, 2016);

Por exemplo, seja também uma matriz de *pixels* 5×5 , uma operação de *pooling* 3×3 , *stride* = 1 e uma matriz de saída 3×3 . A primeira iteração será como a Equação 2.37, onde o valor do primeiro elemento da matriz de saída é dado pelo valor máximo de todos os elementos da grade de *pooling* 3×3 na matriz original, isto é $\max(3, 2, 1, 0, 4, 2, 3, 1, 1) = 4$.

$$\begin{array}{cc}
 \text{Matriz de Pixels} & \text{Matriz de Max Pooling} \\
 \begin{bmatrix} 3 & 2 & 1 & 0 & 1 \\ 0 & 4 & 2 & 3 & 2 \\ 3 & 1 & 1 & 0 & 0 \\ 4 & 4 & 1 & 2 & 0 \\ 1 & 3 & 1 & 0 & 1 \end{bmatrix} & \begin{bmatrix} 4 & - & - \\ - & - & - \\ - & - & - \end{bmatrix}
 \end{array} \tag{2.37}$$

2.1.3.1.3 Camada Totalmente Conectada

Essa camada se encontra ao final do processamento das convoluções e *pooling* em uma CNN e serve para classificação. Assim, essa camada é, geralmente, formada por uma rede MLP e usa como entrada as *features* de saída da CNN após a aplicação das convoluções e *pooling*. Isso se deve ao fato dessas saídas representarem as *features* de mais alta ordem da imagem de entrada. E por fim, a MLP fará o processo de classificação do exemplo passado.

2.1.3.2 Transferência de Aprendizado e *Finetuning*

Muitas vezes o treinamento de uma rede CNN pode ser demorado aliado ao fato de demandar uma grande quantidade de dados. Para facilitar o processo de utilizar uma CNN, pode-se carregar os pesos (e outros parâmetros) de uma rede já treinada com uma grande quantidade de dados em uma rede. Esse processo é denominado transferência de aprendizado (TORREY; SHAVLIK, 2010), sendo possível utilizar diretamente a rede carregada com esses parâmetros ou ainda, utilizá-la e ao final, aplicar um *finetuning*. O processo de *finetuning* é onde a CNN realiza um pequeno treinamento para adaptar a rede com o novo conjunto de dados.

2.1.3.3 Modelos de *Convolutional Neural Networks*

Com o surgimento das CNNs, diversas redes com técnicas, métodos e número de camadas diferentes foram propostas. Alguns modelos além da LeNet-5 são a VGGNet, ResNet e Xception, que serão as CNNs detalhadas nas próximas subseções.

2.1.3.3.1 VGGNet

A VGGNet foi proposta por (SIMONYAN; ZISSERMAN, 2014), fruto da participação dos autores na competição ImageNet Challenge 2014, onde eles investigaram o efeito da profundidade das CNNs em relação a acurácia no reconhecimento de imagens de larga escala. A partir disso, realizaram diversos testes aumentando o número de camadas das CNNs até 19 camadas e isso era possível devido ao fato das matrizes de filtro terem dimensão 3×3 (a menor possível para manter a noção de cima/baixo, lados esquerdo/direito). Com isso, mostraram a efetividade do aumento de camadas, criando as redes VGG16 e VGG19 que possuem 16 e 19 *weight layers*, respectivamente.

Essas redes recebem como entrada uma imagem RGB de dimensão 224×224 e possuem 13 e 16 camadas de convolução (13 para a VGG16 e 16 para a VGG19) com filtros 3×3 e *stride* = 1, além de 5 camadas de *pooling* do tipo *max pooling* 2×2 com *stride* = 2. Após isso, possui 3 camadas totalmente conectadas, sendo as duas primeiras com 4096 neurônios cada e a terceira com 1000 neurônios para classificação. Essas camadas ocultas são equipadas com a ativação ReLU. Por fim, a camada final é uma camada de *softmax*.

Na Figura 13 é possível visualizar uma tabela com as arquiteturas testadas com a VGGNet, sendo a VGG16 e VGG19 representadas nas colunas D e E, respectivamente.

Figura 13: Visualização da Arquitetura das VGGs.

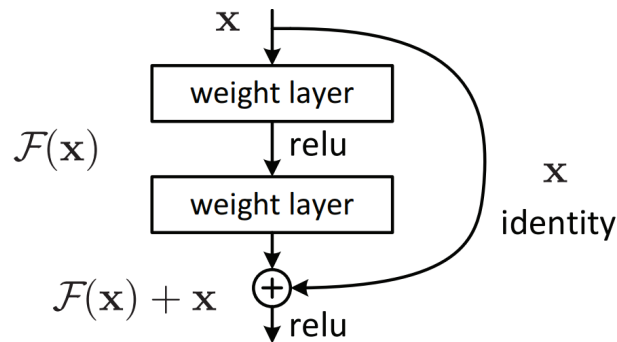
ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Fonte: (SIMONYAN; ZISSERMAN, 2014).

2.1.3.3.2 ResNet

Com a VGGNet é notável a influência do número de camadas na acurácia. Mas simplesmente adicionar mais camadas pode não representar um ganho real. Isto porque quanto maior o número de camadas, mais difícil o treinamento de uma CNN e além disso, pode haver problemas de saturação e degradação da acurácia. Esse foi o problema resolvido pelos autores (HE et al., 2016) com a criação da ResNet, uma CNN também advinda de uma competição, onde atingiu o 1º lugar em 2015.

Para resolver o problema de degradação, a ResNet se baseia em um *framework* de *deep residual learning*. Nesta técnica a saída de uma camada i é adicionada na saída da camada $i + 1$, como pode ser visualizado pelo esquema da Figura 14.

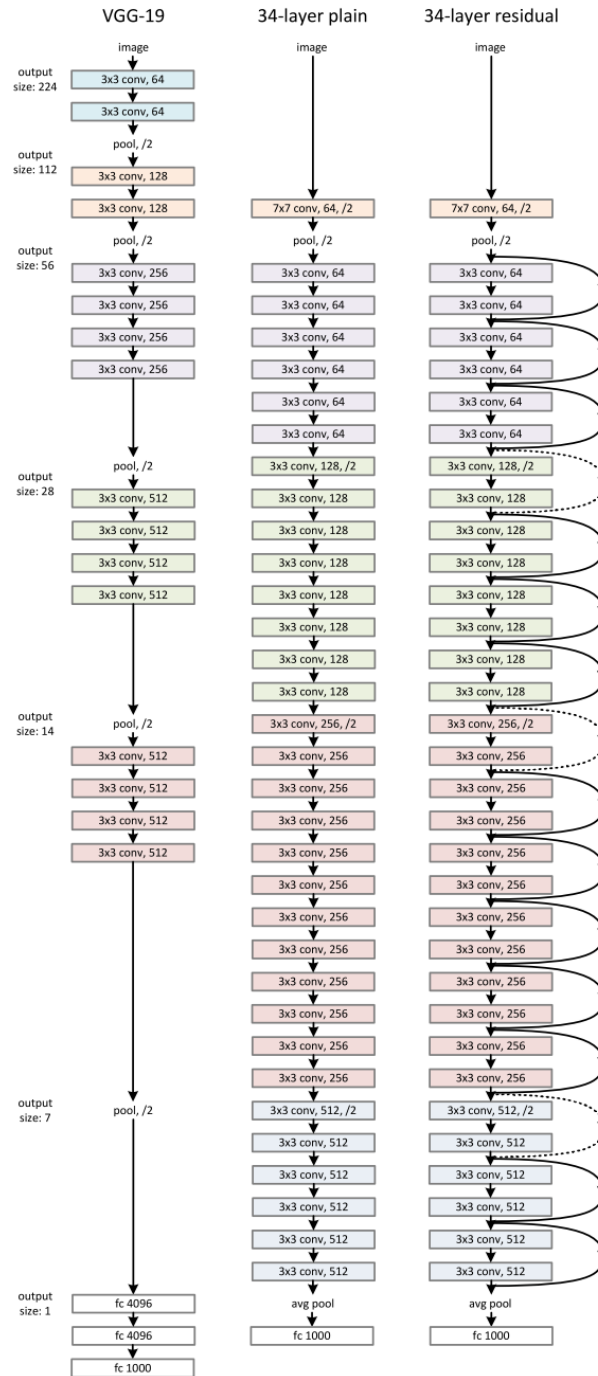
Figura 14: Esquema da técnica de *residual learning*.

Fonte: (HE et al., 2016).

A arquitetura da ResNet é composta por diversas camadas de convolução, 50 camadas no caso da ResNet50 e utiliza filtros 1×1 e 3×3 . Após a primeira camada de convolução é aplicado um *pooling* com *stride* = 2 e em seguida, são aplicadas as 48 camadas de convoluções restantes. Ao final dessas camadas, a rede possui uma camada de *global average pooling*, seguido por uma camada totalmente conectada com 1000 conexões e uma camada *softmax*.

Com isso, a ResNet possui mais camadas que a VGG16 e VGG19, porém possui menos filtros e menor complexidade. Na Figura 15 é apresentada a arquitetura da VGG19, em seguida uma CNN com 34 camadas qualquer e por fim, essa rede com a técnica da ResNet, denominado-a ResNet34.

Figura 15: Visualização da Arquitetura da VGG19, uma CNN com 34 camadas e a ResNet34.



Fonte: (HE et al., 2016).

2.1.3.3.3 Xception

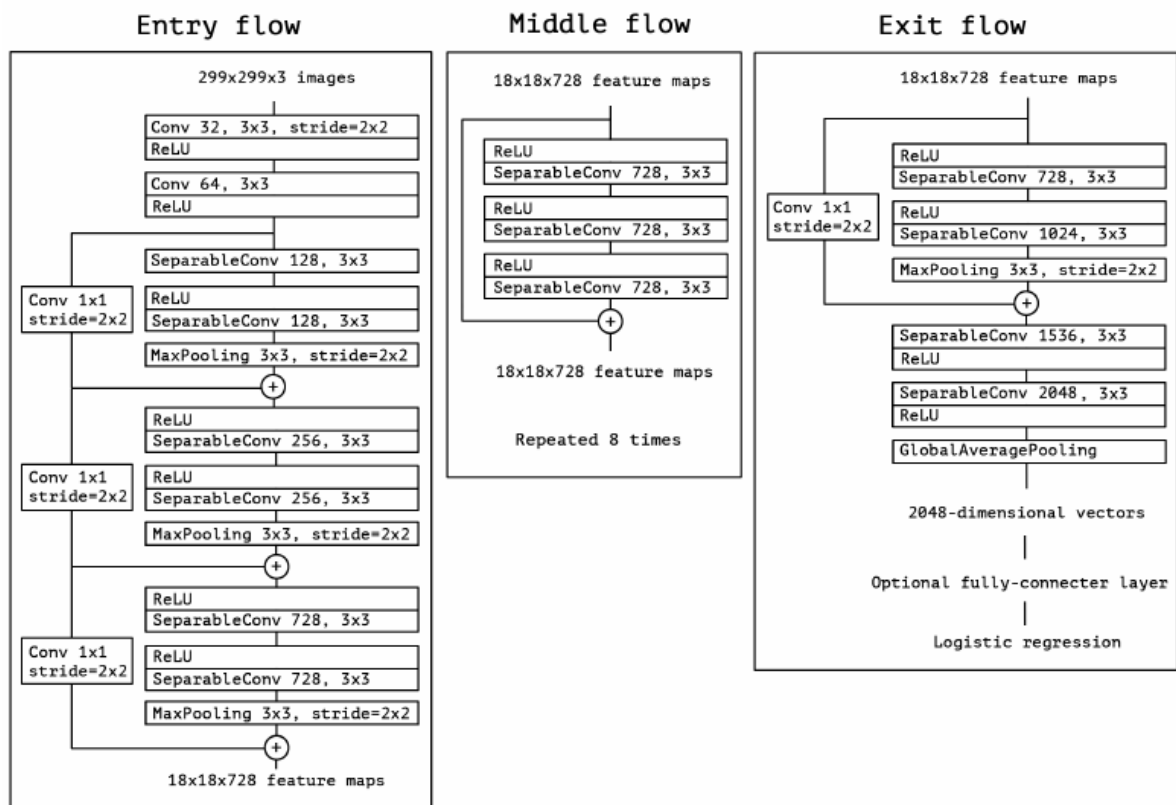
A Xception (CHOLLET, 2017) ou ainda, Extreme Inception se baseia na CNN Inception (SZEGEDY et al., 2015), diferenciando-se pelo fato de possuir *depthwise separable convolution* ao invés dos *inception modules* como na Inception. Assim, a Xception possui uma hipótese mais forte em relação a Inception. Esta hipótese se baseia em que o mapeamento de

correlações entre canais e correlações espaciais nos mapas de características de CNNs pode ser totalmente desacoplado.

A fase de *depthwise separable convolution* é composta por duas operações: *depthwise convolution* seguida por uma *pointwise convolution*. A primeira é definida como uma convolução espacial aplicada independentemente sobre cada canal da entrada. A segunda é definida como uma convolução 1×1 , projetando os canais de saída pela *depthwise convolution* para um novo espaço de canal.

A arquitetura da rede é baseada em 36 camadas de convolução estruturadas em 14 módulos e cada um desses módulos possuem conexões residuais ao seu redor (exceto pelo primeiro e último módulo). Além disso, essa rede possui camadas de *pooling* do tipo *max pooling*, *global average pooling*, ReLU e camadas totalmente conectadas (opcionais). Essa arquitetura pode ser visualizada na Figura 16.

Figura 16: Visualização da Arquitetura da Xception.



Fonte: (CHOLLET, 2017).

2.1.3.4 Generative Adversarial Network (GANs)

As *Generative Adversarial Networks* (GANs), ou Redes Gerativas Adversárias, em português, são uma proposta de (GOODFELLOW et al., 2014) para a criação de um *framework*

baseado na geração de dados sintéticos, por um processo adversarial de duas redes neurais.

Neste processo há dois modelos: um modelo gerador G , que irá gerar novos dados, e o modelo discriminador D que será um avaliador, que determina se o exemplo foi gerado pela rede G ou é um dado real do conjunto de treino. Analogamente, este processo pode ser visto como a rede geradora G sendo uma espécie de falsificadora tentando produzir moedas falsas e a rede discriminadora D , a polícia que tenta detectar se a moeda é falsa ou não. Assim, ambas redes tentam melhorar suas funções até que as falsificações da rede G sejam indistinguíveis do real (GOODFELLOW et al., 2014).

Dessa forma, esse modelo pode ser implementado por redes neurais. Nos modelos mais simples, ambas redes são representadas por MLPs. Sendo assim, sejam G e D , funções diferenciáveis representadas por MLPs com parâmetros θ_g e θ_d , respectivamente. Para que G aprenda a distribuição p_g sobre um dado x , primeiro são definidos variáveis ruído de entrada $p_z(z)$ que representam um mapeamento para o espaço de dados como $G(z; \theta_g)$. Já $D(x; \theta_d)$ retorna um escalar e $D(x)$ representa a probabilidade de x ter vindo de p_g (GOODFELLOW et al., 2014).

Portanto, G é treinado de forma a minimizar $\log(1 - D(G(z)))$ e D é treinado para maximizar a probabilidade de atribuir corretamente as categorias dos dados gerados e originais (GOODFELLOW et al., 2014). Este processo também pode ser visto como o problema de *minimax* (DU; PARDALOS, 2013) com dois jogadores por meio da função $V(G, D)$, dada por:

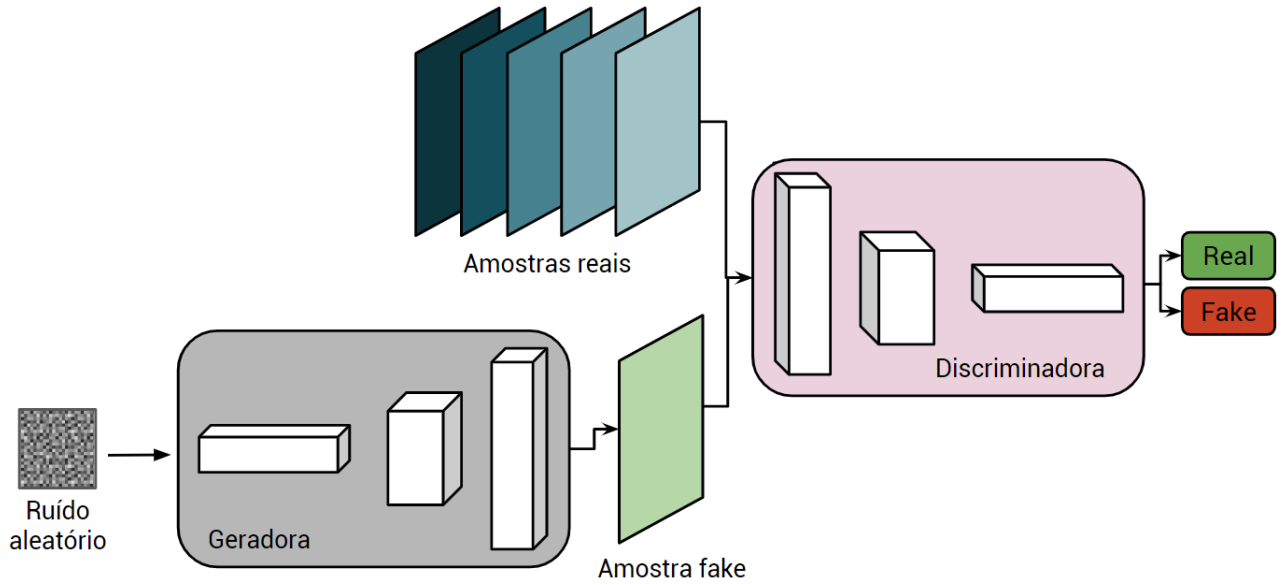
$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (2.38)$$

A organização e esquema de funcionamento das GANs pode ser visualizada na Figura 17. Em suma:

- a rede G considera números aleatórios e retorna um vetor, que após um *upsampling* retorna uma imagem;
- a imagem é inserida junta com as amostras reais para a rede D ;
- D irá retornar probabilidades 0 ou 1, representando se a imagem é falsa ou verdadeira;
- o processo é repetido, de forma que G consiga produzir resultados cada vez mais parecidos com o original.

Como visto anteriormente, a ideia é aprender uma distribuição de uma variável complexa, que é representada por um vetor. Para imagens, o vetor passa por um processo de *upsampling*. Esse processo é na verdade, um processo de transformar o vetor em uma imagem, isso pode ser feito dividindo o vetor em partes iguais que representarão uma linha de uma imagem e ao final, formarão uma imagem completa.

Figura 17: Exemplo de funcionamento das GANs.



Fonte: O autor.

2.1.4 Medidas de Avaliação de Algoritmos de AM

Depois de gerar modelos de indutores com técnicas de AM é importante medir e estimar o desempenho futuro dos algoritmos. Sendo assim, nas próximas seções serão detalhadas medidas de avaliação, amostragem e comparação de algoritmos de AM.

2.1.4.1 Taxa de Arro e Acurácia

Para avaliar os algoritmos de AM existem diversas medidas. Para problemas de classificação, uma medida comumente usada é a taxa de erro (MONARD; BARANAUSKAS, 2003). A taxa de erro de um classificador h é dada pela média de classificações incorretas. Assim essa medida pode ser calculada por:

$$err(h) = \frac{1}{n} \sum_{i=1}^n \|y_i \neq h(x_i)\| \quad (2.39)$$

Neste caso, $\|E\|$ é o operador que retorna 1 se a expressão E é verdadeira e 0 caso contrário, sendo y_i o rótulo do exemplo, $h(x_i)$ o rótulo predito pelo indutor e n o número de exemplos.

Uma medida complementar à taxa de erro é a acurácia. A acurácia leva em conta a

média dos elementos classificados corretamente e pode ser calculada como:

$$acc(h) = 1 - err(h) = \frac{1}{n} \sum_{i=1}^n \|y_i = h(x_i)\| \quad (2.40)$$

2.1.4.2 Matriz de Confusão

Uma outra forma de avaliar algoritmos de AM é por meio das matrizes de confusão. A matriz de confusão mostra o número de classificações corretas e as classificações preditas para cada classe. Portanto, o número de acertos de cada classe está localizado na diagonal da matriz e os demais elementos denotam os erros de classificação (MONARD; BARANAUSKAS, 2003).

Por simplicidade, considere um problema de duas classes, a classe positiva denotada por C_+ e a classe negativa, denotada por C_- . Com isso, podemos definir a seguinte matriz de confusão da Tabela 3.

Tabela 3: Exemplo de Matriz de Confusão.

	Predita C_+	Predita C_-
Verdadeiro C_+	VP	FN
Verdadeiro C_-	FP	VN

Fonte: O autor. Adaptado de (MONARD; BARANAUSKAS, 2003)

Com isso, novas definições são introduzidas, os acertos serão denotados como verdadeiros positivos (VP) e verdadeiros negativos (VN) e os erros por falsos positivos (FP) e falsos negativos (FN) (FACELI et al., 2011). Esses conceitos são definidos por:

- verdadeiro positivo: corresponde ao número de exemplos da classe positiva classificados corretamente.
- verdadeiro negativo: corresponde ao número de exemplos da classe negativa classificados corretamente.
- falso positivo: corresponde ao número de exemplos da classe negativa que foram classificados incorretamente como se fossem da classe positiva.
- falso negativo: corresponde ao número de exemplos da classe positiva que foram classificados incorretamente como se fossem da classe negativa.

Além disso, note que se o número de exemplos de um conjunto é n , temos:

$$n = VP + VN + FP + FN \quad (2.41)$$

2.1.4.3 Medidas de Desempenho de Algoritmos de AM

A partir de uma matriz de confusão, outras medidas podem ser extraídas além da taxa de erro e acurácia (FACELI et al., 2011). Sendo assim, considerando um classificador h e um matriz de confusão para duas classes, podemos definir:

- taxa de erro: pode ser reescrita por:

$$err(h) = \frac{FP + FN}{n} \quad (2.42)$$

- acurácia: pode ser reescrita por:

$$acc(h) = \frac{VP + VN}{n} \quad (2.43)$$

- precisão: mede a proporção de exemplos positivos classificados corretamente entre os verdadeiros e falsos positivos;

$$prec(h) = \frac{VP}{VP + FP} \quad (2.44)$$

- sensibilidade ou revocação: mede a taxa de acerto na classe positiva;

$$sens(h) = revoc(h) = \frac{VP}{VP + FN} \quad (2.45)$$

- especificidade: mede a taxa de acertos na classe negativa;

$$spec(h) = \frac{VN}{VN + FP} \quad (2.46)$$

- medida F1: a precisão e revocação não levam em consideração o que acontece com a outra classe. Assim, geralmente elas são combinadas na Medida F1, que dá o mesmo grau de importância à revocação e à precisão, sendo calculada por:

$$F1 = \frac{2 * prec(h) * revoc(h)}{prec(h) + revoc(h)} \quad (2.47)$$

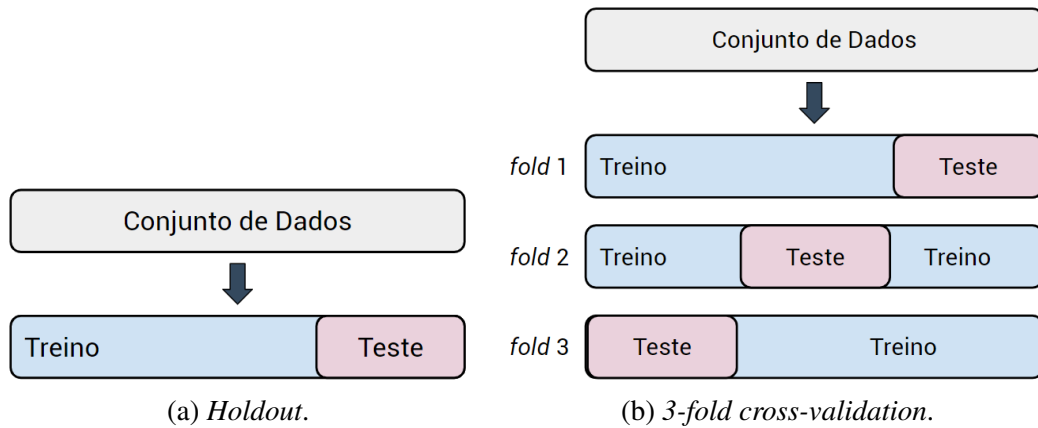
2.1.4.4 Métodos de Amostragem para Algoritmos de AM

Como visto anteriormente, após a geração de um classificador a partir de um conjunto de exemplos de tamanho finito, é importante estimar o desempenho futuro desse modelo (MONARD; BARANAUSKAS, 2003). Porém, estimar o erro com o próprio conjunto de exemplos (denominado método de ressubstituição) usado para geração do modelo pode gerar estimativas muito otimistas e com isso deve-se utilizar técnicas de amostragem (FACELI et al., 2011). Essas técnicas serão detalhadas a seguir conforme definições de (MONARD; BARANAUSKAS, 2003).

- *holdout*: o estimador divide o conjunto de dados em uma porcentagem p para exemplos de treino e $(1-p)$ para teste. Normalmente $p > \frac{1}{2}$ e um valor muito utilizado é $p = \frac{2}{3}$;
- amostragem aleatória: esse estimador considera partições aleatórias e independentes gerando L hipóteses, $L \ll n$ e o erro será dado pela média dos erros de cada hipótese induzida. Como as amostras são aleatórias, geralmente, produz melhores estimativas de erro que o estimador *holdout*;
- *cross-validation*: no *r-fold cross-validation* o conjunto de dados é aleatoriamente dividido em r partições mutuamente exclusivas denominado *folds* com tamanho aproximadamente $\frac{n}{r}$. Assim, os $(r-1)$ *folds* são utilizados para treinamento e o *fold* restante é utilizado para teste. Este mesmo processo é repetido n vezes e o erro final será dado pela média dos erros calculado para cada *fold*;
- *stratified cross-validation*: similar ao *cross-validation*, porém considera a distribuição de classes ao gerar os *folds* e portanto cada *fold* terá a mesma proporção de classes que o conjunto original.
- *Leave-one-out*: Esse estimador pode ser visto como uma versão mais extrema do *cross-validation* e é usado quando o conjunto de dados é muito pequeno. Sendo assim, dado um conjunto de dados de tamanho n , será considerado para treino $(n-1)$ exemplos e o único exemplo restante será utilizado para teste. Este processo também será repetido n vezes e o erro final será dado pela soma de erro de cada teste dividido por n ;
- *bootstrap*: este estimador consiste na ideia de repetir o processo de classificação um grande número de vezes. Há diversos estimadores deste tipo, sendo mais comum o *bootstrap* e0. Então, um conjunto de treino *bootstrap* é composto de n exemplos amostrados com reposição a partir do conjunto original de exemplos. Isto é, alguns exemplos podem não aparecer neste conjunto de treino, enquanto que alguns exemplos podem aparecer mais de uma vez. Esses exemplos que não aparecem no conjunto de treinamento *bootstrap* são utilizados como conjunto de teste. O erro é estimado como a média dos erros sobre o número de repetições do método.

Neste trabalho será considerado o uso da técnica de *holdout*, para poder comparar com as bases geradas por DA. Na Figura 18a é apresentando uma ilustração sobre o método de amostragem *holdout* e na Figura 18b, o *3-fold cross-validation*.

Figura 18: Exemplos de métodos de amostragem.



2.1.4.5 Medidas de Comparação de Algoritmos de AM

Comumente algoritmos e técnicas são comparadas em AM. Para concluir se tal técnica ou algoritmo é melhor que outro, deve-se utilizar testes estatísticos e verificar se a diferença é significativa ou não. Assim, para comparar algoritmos é comum utilizar a técnica de *cross-validation*, calculando a média e desvio padrão dos algoritmos. Porém, outras técnicas podem ser utilizadas, exceto pelo método de ressubstituição (MONARD; BARANAUSKAS, 2003).

Portanto, dado um algoritmo A , um conjunto de exemplos T e assumindo que T seja dividido por r partições. Cada partição terá uma hipótese h_i induzida e um erro $err(h_i)$ com $i = 1, \dots, r$. Dessa forma, a média, variância e desvio padrão para todas partições serão dados, respectivamente, pelas equações a seguir:

$$media(A) = \frac{1}{r} \sum_{i=1}^r err(h_i) \quad (2.48)$$

$$var(A) = \frac{1}{r} \left[\frac{1}{r-1} \sum_{i=1}^r (err(h_i) - media(A))^2 \right] \quad (2.49)$$

$$desvio(A) = \sqrt{var(A)} \quad (2.50)$$

Porém, comparar dois algoritmos apenas pelo erro pode não ser suficiente para concluir se algum algoritmo é melhor que o outro. Quando comparados dois algoritmos em um mesmo domínio T , o desvio padrão pode ilustrar a robustez do algoritmo, pois se o erro muda muito de um experimento para outro, então o indutor não é robusto a mudanças nesse conjunto que segue uma mesma distribuição (MONARD; BARANAUSKAS, 2003).

Portanto, quando não é possível concluir diretamente qual algoritmo foi superior (por exemplo, um algoritmo se saiu melhor na média, mas possui um desvio padrão maior que o obtido pelo outro algoritmo), para poder concluir com 95% de confiança qual foi melhor, podemos

primeiro assumir que os valores seguem uma distribuição normal. Com isso, dado um algoritmo A_1 e A_2 o outro algoritmo a ser comparado, temos novas medidas a serem calculadas como a média combinada, desvio padrão combinado e a diferença absoluta, dados, respectivamente, pelas equações a seguir (MONARD; BARANAUSKAS, 2003).

$$media(A_1 - A_2) = media(A_1) - media(A_2) \quad (2.51)$$

$$desvio(A_1 - A_2) = \sqrt{\frac{desvio(A_1)^2 + desvio(A_2)^2}{2}} \quad (2.52)$$

$$difabs(A_1 - A_2) = \frac{media(A_1 - A_2)}{desvio(A_1 - A_2)} \quad (2.53)$$

Sendo assim, se $difabs(A_1 - A_2) > 0$, então A_2 supera A_1 e ainda, $difabs(A_1 - A_2) > 2$ desvios padrões então A_2 supera A_1 com grau de confiança de 95%. No entanto, se $difabs(A_1 - A_2) \geq 0$, então A_1 supera A_2 e se $difabs(A_1 - A_2) > -2$ então A_1 supera A_2 com grau de confiança de 95% (MONARD; BARANAUSKAS, 2003).

2.2 Data Augmentation

Em AM, o desempenho dos algoritmos pode ser estendido de acordo com o aumento no número de instâncias numa base de dados. No entanto, em situações da vida real, nem sempre temos muitos exemplos disponíveis ou muitas vezes, a maioria destes não são rotulados, o que é necessário para algumas aplicações.

Dessa forma, pode-se utilizar técnicas de *data augmentation* (DA), que em tradução direta, significa aumento de dados. O objetivo é utilizar técnicas computacionais para aumentar o conjunto de treinamento supervisionado para se obter algoritmos com maiores capacidade de predição, e consequentemente, melhores resultados.

Isto é, uma técnica de DA é um método capaz de aumentar o conjunto de dados de treinamento preservando seu rótulo original e que pode ser representada como o mapeamento da Equação 2.54. O fato de DA se basear em técnicas que preservam os rótulos originais de um exemplo, significa que se um dado x tem rótulo y , então $\phi(x)$ também terá rótulo y (TAYLOR; NITSCHKE, 2018)

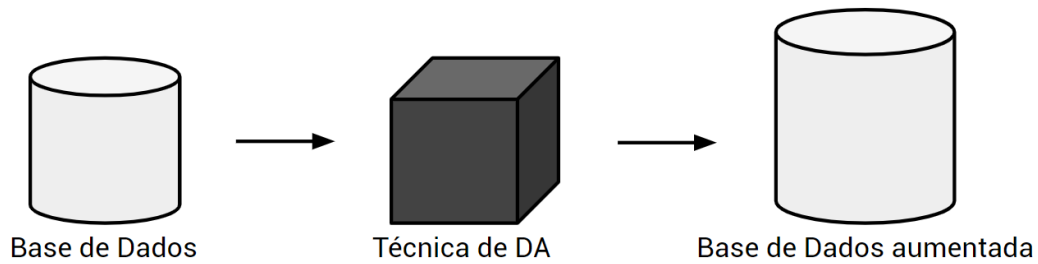
$$\phi : S \mapsto T \quad (2.54)$$

Onde S é o conjunto de treino original e T é o conjunto aumentado a partir de S . E com isso, o conjunto de dados aumentado S' , contendo os dados originais e os dados aumentado pela técnica ϕ é representado por:

$$S' = S \cup T \quad (2.55)$$

Dessa forma, existem diversas abordagens para se obter mais dados ao conjunto de treino. Uma técnica comum é aplicar efeitos de transformação de imagens no conjunto de dados para gerar novos exemplos. Outras, como as *Generative Adversarial Networks*, geram dados sintéticos a partir do conjunto de dados. Quando se tem um conjunto de dados rotulados e não rotulados, pode-se utilizar algoritmos para rotular esses dados não rotulados, gerando um conjunto maior de treinamento de exemplos rotulados (MRKŠIĆ, 2013). Na Figura 19, temos um esquema prático do funcionamento do conceito de DA. Neste trabalho, serão utilizadas duas técnicas de DA: Transformações de Imagens e o uso das *Generative Adversarial Networks*.

Figura 19: Exemplo de funcionamento de DA.



Fonte: O autor.

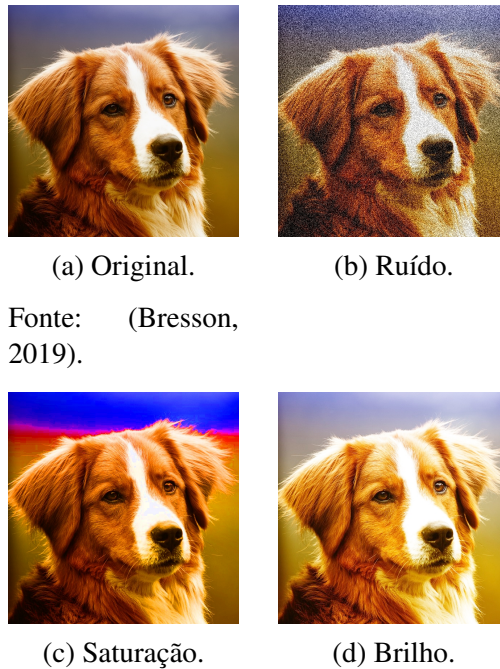
2.2.1 Transformação de Imagens

O método de Transformação de Imagens é uma ideia simples e muito utilizada quando o objetivo é obter mais dados de imagens. Nesta abordagem, dado um conjunto de imagens, aplica-se efeitos de transformação de imagens nos exemplos deste conjunto, gerando mais imagens. Ao final, o conjunto aumentado será composto pelas imagens originais e as imagens com aplicação dos efeitos.

Essas transformações de imagens são usualmente divididas entre duas categorias: técnicas fotométricas e técnicas geométricas (TAYLOR; NITSCHKE, 2017). As fotométricas aplicam efeitos como ruído, *blur*, efeitos de coloração (ex. brilho, saturação, *color jittering*), e *etc.* As geométricas usam, por exemplo, rotações, translações, escalas, *flip's* e *etc.*

Na Figura 20 é apresentado um exemplo prático da aplicação de efeitos fotométricos, onde na Figura 20a temos a imagem original, na Figura 20b a mesma imagem com aplicação de ruído, 20c mudanças na saturação e na Figura 20d, alteração no brilho.

Figura 20: Exemplo de funcionamento das transformações de imagens fotométricas.

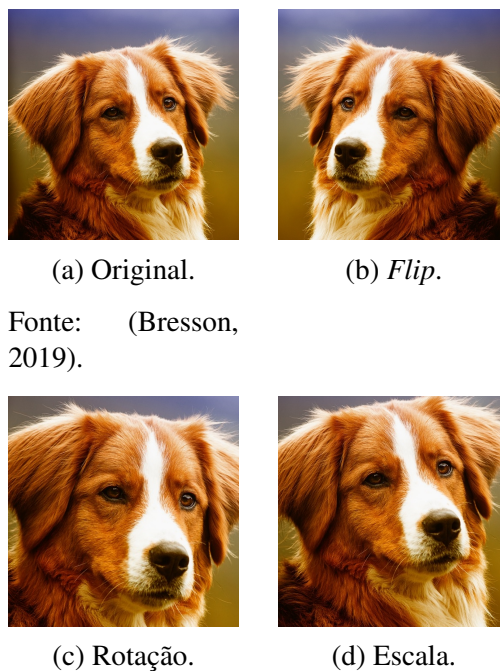


Fonte: (Bresson, 2019).

Fonte: O autor.

Na Figura 21 são apresentados os efeitos geométricos. Na Figura 20a, a imagem original, Figura 21b, *flip* horizontal, Figura 21c rotação e Figura 21d, uma escala.

Figura 21: Exemplo de funcionamento das transformações de imagens geométricas.



Fonte: (Bresson, 2019).

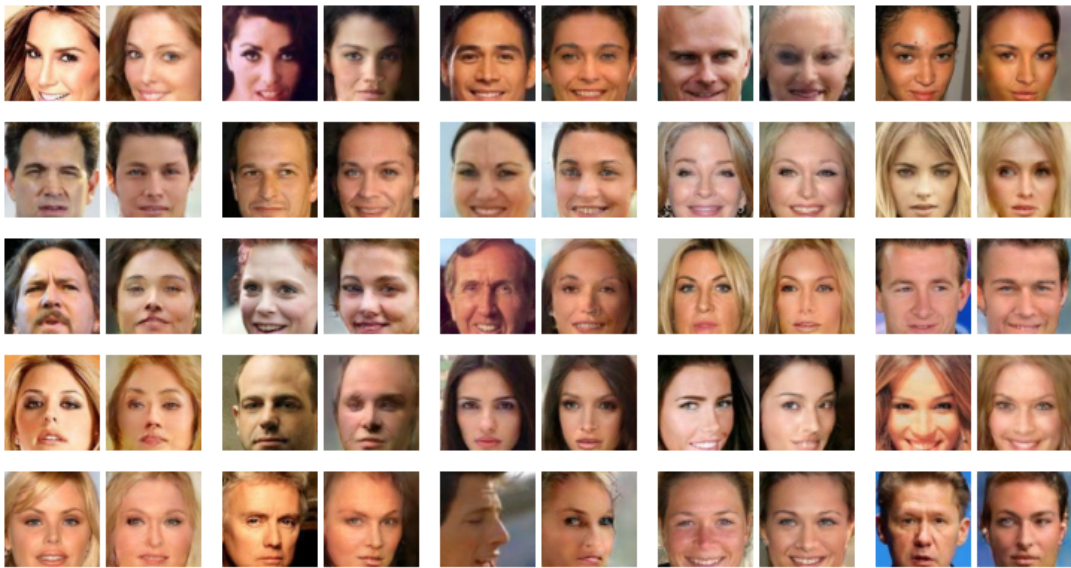
Fonte: O autor.

2.2.2 Generative Adversarial Network (GANs) Como Técnica de DA

Como fora visto anteriormente, as GANs geram novos dados a partir de um conjunto de treino e geralmente, obtém bons resultados na geração de dados artificiais. Dessa forma, pode-se utilizar essas redes para criar um conjunto de dados maior, sendo o novo conjunto formado pelas imagens originais e as imagens criadas pelas GANs.

Na Figura 22, há exemplos do funcionamento das GANs na reconstrução de rostos. Cada par de fotos é composto pela foto original e sua versão reconstruída (com algumas alterações) por uma GAN.

Figura 22: Exemplo de aplicação das GANs na reconstrução de rostos.



Fonte: Greg Heinrich (2017).

3 Revisão Bibliográfica

Diversas técnicas de DA foram propostas na literatura, pois obter grandes quantidades de dados é uma tarefa difícil e de extrema importância para se obter bons resultados com os algoritmos de AM. Portanto, alguns trabalhos relacionados serão analisados e detalhados nas próximas seções.

3.1 Trabalhos Relacionados

As abordagens mais utilizadas são as que envolvem técnicas de transformação de imagens tradicionais. No entanto, também foram propostas várias variações desse métodos e as novas abordagens utilizadas são os métodos gerativos, como as GANs.

3.1.1 Aplicação de DA

Várias aplicações podem ser beneficiadas com as técnicas de DA. A visão computacional, por exemplo, se baseia em detectar objetos em imagens e é melhor utilizada com uma grande quantidade de dados. Dessa forma, diversos trabalhos utilizaram técnicas de DA em seus conjuntos de dados, como os trabalhos apresentados a seguir.

(ZHANG et al., 2019) utilizaram DA com três tipos de métodos: rotação de imagem, correção *gamma* e *noise injection*. Nesse trabalho foi utilizado uma CNN como classificador e o *dataset* utilizado era composto por imagens para classificação de frutas. O melhor resultado encontrado foi 94,94%, sendo aproximadamente 5% a mais que os resultados encontrados na literatura.

Outra área de aplicação beneficiada por DA é a classificação de imagens aéreas, pois esses dados são geralmente limitados. Os autores (YU et al., 2017) utilizaram três técnicas de transformação de imagens, como o *flip*, translação e rotação para gerar dados aumentados com três base de dados de imagens de sensoriamento remoto. Os experimentos confirmaram que o uso dessas técnicas de DA afetaram positivamente os resultados obtidos na classificação dessas imagens.

A área médica é uma das aplicações que mais sofre com o problema de dados limitados, então técnicas de DA podem ser muito úteis. (HUSSAIN et al., 2017) estudaram diferentes tipos de estratégias de transformação de imagens no contexto de imagens médicas, como *flips* horizontais, cortes aleatórios, PCA, etc. A base de dados utilizada é composta por imagens de massa e não-massa (normal) da *Digital Database for Screening Mammography (DDSM)*. O

trabalho também comparou diferentes estratégias de DA e mostrou que a performance aumentou diante o aumento do conjunto de treino (retendo as propriedades da imagem médica original). As abordagens de *flips* e filtros gaussianos levaram a acurácia de validação de 84% para 88%. Mas algumas estratégias se mostraram menos efetivas como a adição do *noise* nas imagens, o que piorou a validação de acurácia pra 66%.

Um outro exemplo de aplicação médica que foi beneficiada por DA foi o trabalho apresentado por (WANG et al., 2018), onde os autores estudaram a detecção de alcoolismo. O alcoolismo está ligado a uma doença no cérebro, alterando sua estrutura e portanto, a visão computacional pode ser utilizada para detecção da doença. A base de dados utilizada possuía 235 imagens do cérebro de pessoas alcoólatras e não-alcoólatras, onde 100 exemplos foram utilizados para treino e houve a aplicação da técnica de DA. Os resultados encontrados foram uma sensibilidade de 96,88%, especificidade de 97,18% e acurácia de 97,04%, o que foi melhor que três outras abordagens do estado da arte.

3.1.2 Avaliação e Propostas de Novas Técnicas de DA Baseada em Transformação de Imagens

Por outro lado, além das técnicas tradicionais de Transformação de Imagens, diversos autores propuseram variações desses métodos, enquanto outros autores fizeram avaliações das abordagens existentes para uma determinada aplicação.

A ideia dos autores (OKAFOR et al., 2017) foi aplicar técnicas de transformação de imagens para criar uma nova imagem a partir de diversos ângulos de rotação de uma só imagem. Isto é, monta-se uma espécie de grade $n \times n$ com a mesma imagem em vários ângulos de rotação e tenta-se gerar um fundo natural na imagem criada. O domínio era um problema de classificação binária, onde as imagens poderiam conter a presença de vaca ou ser uma imagem sem presença de vaca. O classificador utilizado foi uma CNN e os autores constataram uma melhora significativa com o conjunto aumentado por essa nova abordagem.

Dentre as técnicas de Transformação de Imagens, existe uma longa lista de diferentes abordagens, mas algumas podem se sair melhores que outras ou serem mais indicadas em algumas aplicações. Sendo assim, em (PAULIN et al., 2014) os autores propuseram um algoritmo chamado *Image Transformation Pursuit* (ITP), para seleção automática de um menor conjunto de técnicas de Transformação de Imagens e que consiga melhores resultados, representando ganhos reais. O ITP se apoia em uma estratégia gulosa, onde a cada iteração seleciona a técnica com melhor ganho na acurácia. Ainda, o algoritmo pode ser usado para explorar transformações complexas, combinando outras transformações. Em *datasets* como o ImageNet, os autores relataram um ganho de aproximadamente 5% na acurácia top-5.

Em (FAWZI et al., 2016) os autores também propuseram uma ferramenta para escolher

as melhores transformações de imagens a serem utilizadas em DA. A ideia dos autores foi para cada exemplo buscar uma pequena transformação que produza um *maximal loss* na classificação nas amostras transformadas. Além disso, esse esquema possui uma versão do algoritmo do *Stochastic Gradient Descent* (SGD) para integrar o algoritmo proposto ao treinamento de redes neurais profundas. Os experimentos foram realizados com as bases MNIST e Small-NORB, e os autores obtiveram resultados superiores aos encontrados pelas abordagens existentes.

(INOUE, 2018) propuseram um técnica chamada *SamplePairing* onde novas amostras são sintetizadas a partir do *overlaying* de uma imagem com uma outra imagem escolhida aleatoriamente, isto é, cada *pixel* da nova amostra será dado pela média de *pixels* dessas duas imagens. Com o método podem ser geradas até N^2 imagens dado um conjunto de tamanho N e os resultados mostraram uma melhora significativa na acurácia dos *datasets* utilizados. Com a base ILSVR o erro top-1 foi reduzido de 33,5% pra 29,0%, enquanto que para CIFAR-10 o erro top-1 foi reduzido de 8,22% para 6,93%.

Os autores (TAYLOR; NITSCHKE, 2017) propuseram um estudo comparativo entre os esquemas populares de DA para indicar qual é a técnica mais indicada para tal *dataset*. Foram testadas técnicas geométricas (ex. corte, rotações) e fotométricas (ex. *color jittering*, *fancy PCA*) sendo avaliadas por uma CNN. Os resultados utilizando *4-fold cross validation* foram reportados com as acurácia top-1 e top-5, indicando que a técnica de corte aumentou os resultados obtidos pela CNN.

No algoritmo *Random Erasing* proposto por (ZHONG et al., 2017) uma região retangular da imagem é escolhida aleatoriamente e esses *pixels* são preenchidos com valores aleatórios. Com essa técnica, é esperado menos risco de *overfitting* e faz com que o modelo seja robusto à oclusão. Esse algoritmo pode ser utilizado em conjunto à outras técnicas tradicionais de DA e os resultados relatados pelos autores indicaram melhorias de desempenho nas tarefas testadas, como a detecção de objetos e re-identificação de pessoas.

3.1.3 Métodos Gerativos como DA

Alguns métodos não fazem uso das técnicas tradicionais como as ferramentas de Transformação de Imagens, utilizando outras técnicas para gerar novos dados. Atualmente, diversos trabalhos estão fazendo o uso de métodos gerativos como as GANs para gerar dados sintéticos.

Como exemplo, (ANTONIOU; STORKEY; EDWARDS, 2017) propuseram uma ferramenta chamada *Data Augmentation Generative Adversarial Network* (DAGAN) para criar dados sintéticos com as GANs. Uma das bases de dados utilizadas foi a Omniglot, onde conseguiram uma aumento de acurácia de 69% para 82% e com as *Matching Networks* houve um aumento de 96,9% para 97,5%.

A proposta dos autores (SHIJIE et al., 2017) era aplicar diversas técnicas de DA e após

a geração da base aumentada, classificar os dados utilizando a CNN Alexnet e nesse caso, considerou *subsets* das bases de dados CIFAR-10 e da ImageNet com 10 classes. As técnicas de DA utilizadas foram: GAN/WGAN, corte, rotação *flipping*, *shifting*, *PCA jittering*, *color jittering*, *noise*, e combinações desses métodos. Os autores concluíram que as técnicas de corte, *flipping*, WGAN e rotação obtiveram melhores resultados em relação às outras técnicas e que combinações de métodos de DA podem ajudar a se obter melhores resultados que os individuais.

Em (PEREZ; WANG, 2017), os autores propuseram o estudo de outras técnicas de DA para expandir o conjunto de imagens. Nesse caso, além das técnicas tradicionais como corte, girar, *etc*, também consideraram as redes GANs com filtros de estilização das imagens. O domínio do problema foi baseado em imagens de animais, como peixes, cachorros, gatos e também, dígitos.

Já em (FRID-ADAR et al., 2018), os autores propuseram a utilização de métodos de DA tradicionais e a aplicação das GANs para gerar dados sintéticos. Os testes foram realizados com uma base de dados limitadas de tomografia computacional composta por 182 imagens de lesões no fígado. Os resultados mostraram uma melhora significativa. Com os métodos clássicos de DA obtiveram 78,6% de sensibilidade e 88,4% de especificidade. Com a adição dos dados gerados pelas GANs, foram obtidos 85,7% de sensibilidade e 92,4% de especificidade.

Métodos gerativos como as GANs também podem ser aplicados em casos em que a entrada não é uma imagem, podendo ser gerados também novos dados numéricos. Os autores (TANAKA; ARANHA, 2019) utilizaram as GANs e as técnicas de SMOTE e ADASYN para gerar novos dados. Os três *datasets* utilizados não eram balanceados e o objetivo era torná-los balanceados com a esperança de melhoras na acurácia e outras medidas. Os experimentos de classificação foram realizados com uma árvore de decisão e os resultados de acurácia e precisão encontrados com o classificador utilizando os dados sintéticos balanceados das GANs foram superiores aos encontrados com o *dataset* original.

3.2 Resumo dos Trabalhos Relacionados

Na Tabela 4 é apresentado um resumo de todos os trabalhos relacionados revisados, contendo seu título, os autores e o ano de publicação, o *dataset* ou área de aplicação utilizada e os tipos de técnicas utilizadas. Os trabalhos 1-4 são aplicações de DA, trabalhos 5-10 são avaliações ou propostas ou avaliações de métodos de DA, já os trabalhos 11-15 utilizaram métodos gerativos como as GANs.

Tabela 4: Resumo dos Trabalhos Relacionados.

	Título	Autores e Ano	<i>Dataset</i>	Tipo/Ideia de Técnica de DA
1	Deep learning in remote sensing scene classification: a data augmentation enhanced convolutional neural network framework	(YU et al., 2017)	Imagens Aéreas	<i>Flip</i> ; Translação; Rotação
2	Differential Data Augmentation Techniques for Medical Imaging Classification Tasks	(HUSSAIN et al., 2017)	Imagens Mamografia	<i>Flips</i> horizontais; Cortes aleatórios; PCA; etc.
3	Image based fruit category classification by 13-layer deep convolutional neural network and data augmentation	(ZHANG et al., 2019)	Classificação de Frutas	Rotação; Correção <i>Gamma</i> ; <i>Noise Injection</i> .
4	Transformation Pursuit for Image Classification	(PAULIN et al., 2014)	ImageNet	ITP - Estratégia gulosa para definir melhor técnica de DA
5	ADAPTIVE DATA AUGMENTATION FOR IMAGE CLASSIFICATION	(FAWZI et al., 2016)	MNIST; Small-NORB	Buscar melhor técnica de DA pelo <i>maximal loss</i>
6	Improving Deep Learning using Generic Data Augmentation	(TAYLOR; NITSCHKE, 2017)	Caltech 101; Pascal VOC	Comparação manual de técnicas fotométricas e geométricas
7	Operational Data Augmentation in Classifying Single Aerial Images of Animals	(OKAFOR et al., 2017)	Deteção de Animal	Criar um <i>grid nxn</i> com várias imagens.
8	Random Erasing Data Augmentation	(ZHONG et al., 2017)	Deteção de Objetos; Re-identificação de pessoas	<i>Random Erasing</i> - Apagar e preencher <i>pixels</i> aleatórios
9	Data Augmentation by Pairing Samples for Images Classification	(INOUE, 2018)	ILSVR; CIFAR-10	SampleParing - Overlaying de Imagens
10	Alcoholism Detection by Data Augmentation and Convolutional Neural Network with Stochastic Pooling	(WANG et al., 2018)	Deteção Alcoolismo	Rotação; Correção <i>Gamma</i> ; <i>Noise Injection</i> ; Translação
11	DATA AUGMENTATION GENERATIVE ADVERSARIAL NETWORKS	(ANTONIOU; STORKEY; EDWARDS, 2017)	Omniglot; EMNIST; VGG-Face	DAGAN - DA + GAN
12	Research on Data Augmentation for Image Classification Based on Convolutional Neural Networks	(SHIJIE et al., 2017)	Subsets da CIFAR-10 e ImageNet	Comparação DA tradicional e GANs
13	Effectiveness of Data Augmentation in Image Classification using Deep Learning	(PEREZ; WANG, 2017)	Animais; Dígitos	Comparação DA tradicional e GANs com filtros de estilização
14	SYNTHETIC DATA AUGMENTATION USING GAN FOR IMPROVED LIVER LESION CLASSIFICATION	(FRID-ADAR et al., 2018)	Tomografia do Fígado	Comparação DA tradicional e GANs para balancear <i>datasets</i>
15	Data Augmentation Using GANs	(TANAKA; ARANHA, 2019)	Pima Indians Diabetes Dataset; Breast Cancer Wisconsin; Credit Card Fraud Detection	Uso de DA em dados numéricos e não balanceados com GANs

Fonte: O autor.

3.3 Conclusões

Com o levantamento bibliográfico sobre os trabalhos relacionados em DA, é possível ver as diversas aplicações do conceito. Também é possível notar que todos os artigos citados são relativamente recente, isso porque o uso do *deep learning* cresceu muito nesses últimos anos, levando à uma grande busca por técnicas artificiais para se obter mais dados e consequentemente, surgiram diversas aplicações e novas propostas de abordagens. Portanto, é evidente a importância de DA nos problemas de AM.

4 Proposta de Trabalho

Neste capítulo está centrada a proposta do trabalho, sendo detalhadas as bases de dados de imagens utilizadas, os softwares e bibliotecas, apresentação da fase de experimentação e avaliação dos resultados.

4.1 Bases de Dados

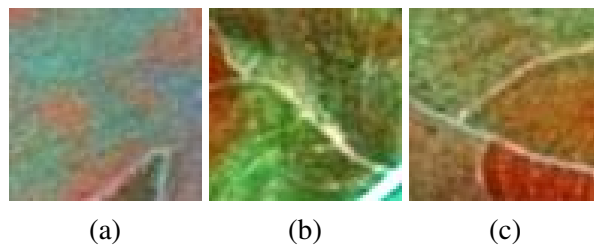
As bases de dados utilizadas são de imagens, disponíveis publicamente na Internet e serão detalhadas a seguir.

4.1.1 *Brazilian Coffee Scenes Dataset*

A base de dados *Brazilian Coffee Scenes Dataset* ([PENATTI K. NOGUEIRA, 2015](#)) é composta por imagens de satélite de plantações de café em cidades de Minas Gerais, pelo sensor SPOT em 2005. Cada imagem possui dimensão 64×64 e foi classificada manualmente por pesquisadores agrícolas como *coffee* se pelo menos 85% dos *pixels* totais da imagens são de uma plantação de café e *non coffee*, caso contrário.

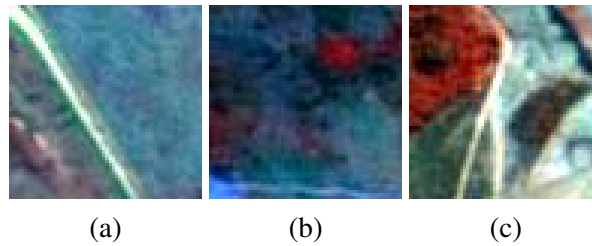
Assim, o conjunto de dados é composto por 2.876 exemplos, sendo 50% exemplos da classe *coffee* e 50% da classe *non coffee*. E a seguir nas Figuras 23 e 24, alguns exemplos das imagens capturadas pelo sensor.

Figura 23: Exemplos de imagens rotuladas como *coffee*.



Fonte: ([PENATTI K. NOGUEIRA, 2015](#)).

Figura 24: Exemplos de imagens rotuladas como *non-coffee*.



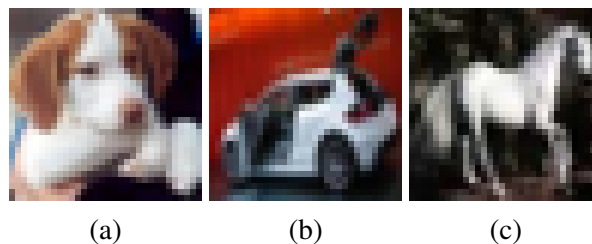
Fonte: (PENATTI K. NOGUEIRA, 2015).

4.1.2 CIFAR-10

A CIFAR-10 (KRIZHEVSKY; HINTON, 2009) é uma base de dados amplamente utilizada em experimentos de AM, sendo composta por 60.000 imagens de dimensão 32×32 , dividida em 10 classes, com 6.000 imagens por classe. Além disso, são 50.000 exemplos para treino e 10.000 para teste.

As 10 classes são: *airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck*. E alguns exemplos de imagens contidas nesse *dataset* são apresentados na Figura 25. Para este trabalho, foi selecionado um *subset* de imagens da classe “Dog”, “Automobile” e “Horse”. Como essa base é bem diversa, não há classes que possuem muita probabilidade de confusão, as classes do *subset* foram escolhidas aleatoriamente.

Figura 25: Exemplos de imagens da base CIFAR-10.



Fonte: (KRIZHEVSKY; HINTON, 2009).

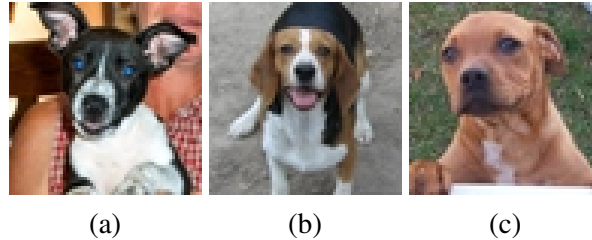
4.1.3 Dogs vs Cats

A base de dados *Dogs vs Cats* é uma base contendo imagens de cachorros e gatos, disponibilizada no *Kaggle*. Esta base é composta por 13.000 exemplos para treino, 12.500 para teste e possuem dimensão variada (KAGGLE, 2015).

Por fins de simplicidade, será usado um subconjunto da base completa. Sendo 10.000 exemplos para treino (com 5.000 exemplos de cada classe) e 2.000 exemplos de teste (com

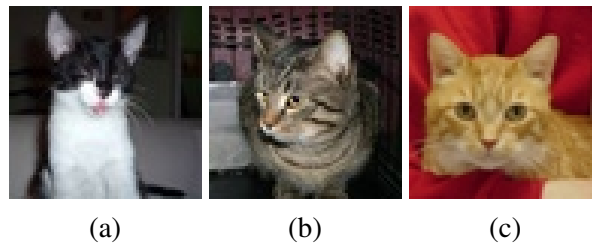
1.000 exemplos de cada classe). Nas Figuras 26 e 27, temos exemplos das imagens contidas no conjunto.

Figura 26: Exemplos de imagens rotuladas como *Dog*.



Fonte: (KAGGLE, 2015).

Figura 27: Exemplos de imagens rotuladas como *Cat*.



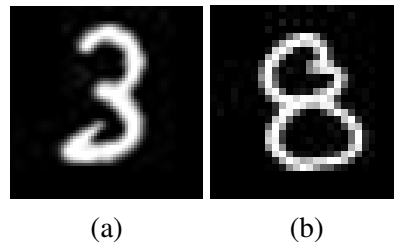
Fonte: (KAGGLE, 2015).

4.1.4 MNIST DATABASE of handwritten digits

A MNIST (LECUN et al., 1998) é uma base amplamente utilizada em experimentações de AM. Esta base é composta por imagens de dígitos escritos a mão e possui 60.000 exemplos para treino, 10.000 para teste e cada imagem possui dimensão 28×28 .

Neste caso, também foi selecionado apenas um *subset*, contendo dígitos “3” e “8”, formando ao final 8.000 exemplos, sendo 4.000 de cada classe. Na Figura 28, temos alguns exemplos das imagens contidas nesse conjunto de dados. Para escolher o *subset*, levou-se em conta classes que possuem certa semelhança e seriam um desafio para os classificadores, como os dígitos “1” e “7” e “3” e “8” (que foram os escolhidos).

Figura 28: Exemplos de imagens contidas na MNIST.



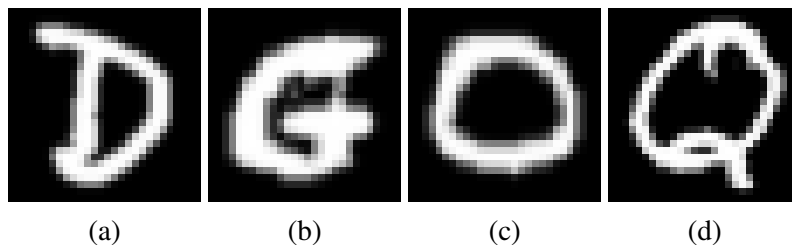
Fonte: (LECUN et al., 1998).

4.1.5 EMNIST

A Extend MNIST - EMNIST (COHEN et al., 2017) é um *dataset* de imagens 28×28 formados por dígitos e letras escritos a mão. Essa base possui diversos *splits* e a versão utilizada será a de letras, que é composta por 145.600 caracteres e 26 classes balanceadas. Algumas imagens contidas nesse *dataset* são apresentados na Figura 29.

Por fins de simplicidade, usou-se apenas exemplos de letra “D”, “G”, “O” e “Q” (que possuem certa semelhança e seriam desafios para o classificador), sendo 5.600 exemplos de cada, que ao final, formaram uma base de 22.400 exemplos.

Figura 29: Exemplos de imagens contidas na EMNIST.



Fonte: Base de dados EMNIST (COHEN et al., 2017).

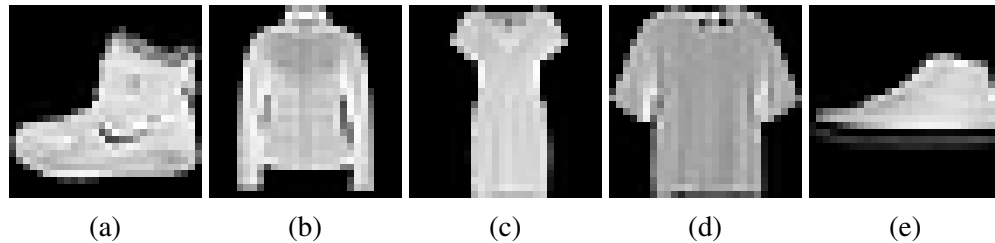
4.1.6 MNIST-Fashion

O *dataset* MNIST-Fashion (XIAO; RASUL; VOLLGRAF, 2017) foi criado com a intenção de gerar novos desafios para os algoritmos de AM, pois podem representar problemas de visão computacional como a detecção de objetos. Essa base de dados de imagens é composta por 60.000 exemplos para treino e 10.000 exemplos para teste.

Cada exemplo possui dimensão 28×28 em níveis de cinza e é associada a uma das 10 classes. Essas 10 classes são: *T-shirt/top*, *Trouser*, *Pullover*, *Dress*, *Coat*, *Sandal*, *Shirt*, *Sneaker*, *Bag*, *Ankle boot*. Alguns imagens contidas nesse *dataset* são apresentados na Figura 30.

Por fins de simplicidade, usou-se apenas exemplos de 5 classes com 7.000 exemplos de cada, sendo elas *Ankle boot*, *Coat*, *Dress*, *Shirt* e *Sneaker*, representas na Figura 30, respectivamente. Com isso, o *subset* possui 35.000 exemplos.

Figura 30: Exemplos de imagens da base MNIST-Fashion.



Fonte: Base de dados MNIST-Fashion. (XIAO; RASUL; VOLLGRAF, 2017)

4.2 Softwares e Bibliotecas

Para realização deste projeto foram realizados experimentos computacionais, sendo utilizadas diversas ferramentas e bibliotecas.

Em resumo, o Python 3.6 (ROSSUM, 1995) foi a principal ferramenta para a realização deste projeto. Sendo utilizado as bibliotecas TensorFlow¹ e Keras² para a extração de características, aplicar Transformação de Imagens e uso de redes neurais. scikit-learn³ para leitura das bases, pré-processamento e uso do classificador SVM, por exemplo. Já o NumPy⁴ será utilizado para *arrays* e matrizes, gerar números aleatórios, entre outras operações.

4.2.1 Python

O Python (ROSSUM, 1995) é uma linguagem de programação interpretada e de alto nível criada por Guido van Rossum em 1991. Entre as principais vantagens da linguagem é sua simplicidade e a existência de vários módulos e bibliotecas para ela. Dessa forma, o Python hoje em dia é uma das linguagens mais utilizadas em uso geral e para computação científica.

4.2.2 TensorFlow

TensorFlow (ABADI et al., 2015) é uma plataforma *open-source* para Aprendizado de Máquina desenvolvido pelo time Google Brain. Essa plataforma é definida pelo Google como

¹ <https://www.tensorflow.org/>

² <https://keras.io/>

³ <https://scikit-learn.org/stable/>

⁴ <https://www.numpy.org/>

um ecossistema compreensível e flexível de ferramentas, bibliotecas e recursos para que pesquisadores desfrutem do estado da arte em AM e que desenvolvedores possam facilmente criar e implementar aplicações baseados em AM.

4.2.3 Keras

Keras ([CHOLLET et al., 2015](#)) é uma API de alto nível para redes neurais e *deep learning* escrita em linguagem Python e pode ser utilizado em conjunto com o TensorFlow. Essa API permite uma fácil e rápida prototipagem por meio de sua facilidade, modularidade e extensibilidade. Além disso, pode ser utilizada tanto em CPUs como GPUs.

4.2.4 scikit-learn

scikit-learn ([PEDREGOSA et al., 2011](#)) é uma biblioteca *open-source* escrita em Python para AM, oferecendo ferramentas simples e eficientes para análise de dados e mineração de dados, como algoritmos de pré-processamento de dados, classificação, regressão, agrupamento e *etc.*

4.2.5 NumPy

O Numpy ([OLIPHANT, 2006](#)) é uma biblioteca fundamental para computação científica com o Python. Essa biblioteca oferece poderosos objetos de *arrays* de dimensão- N , onde pode-se defini-lo como um contêiner de dados de vários tipos, o que faz o Numpy se integrar fácil e rapidamente a uma ampla variedade de bancos de dados. Além disso, possui diversas ferramentas de Álgebra Linear, Transformadas de Fourier, números aleatórios, também oferece funções sofisticadas, ferramentas para integrar código de C/C++ e Fortran e *etc.*

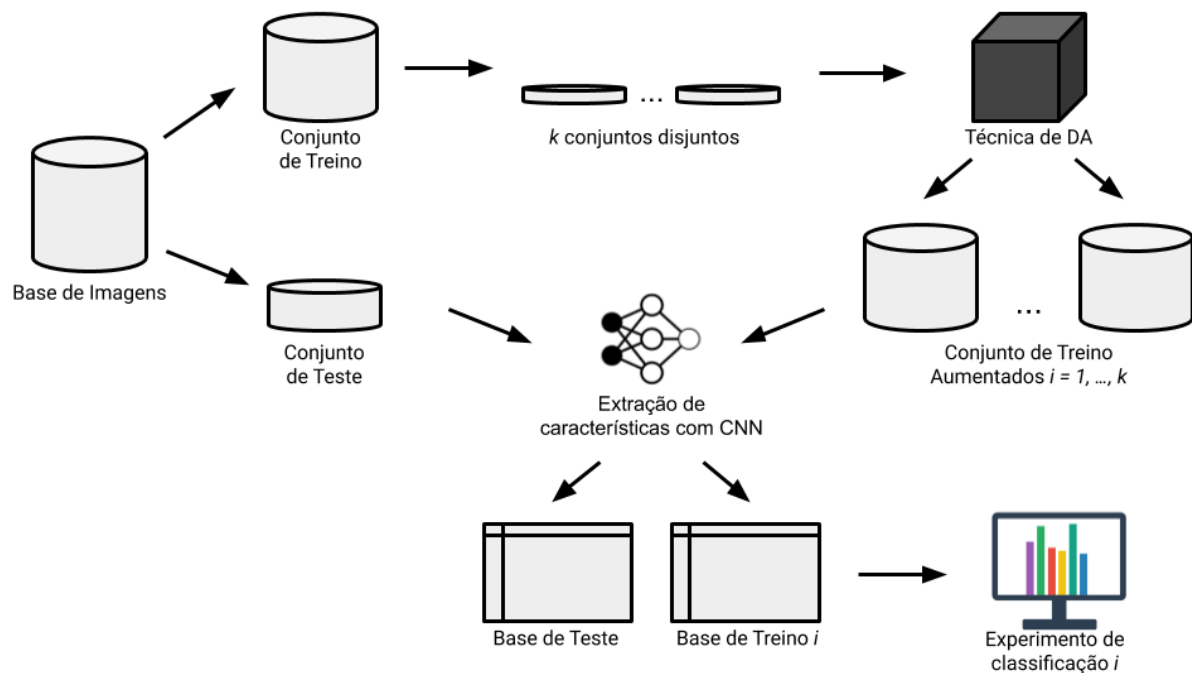
4.3 Método e Análise Proposta

A parte de experimentação envolvida neste projeto, pode ser dividido em 4 fases principais, descritas a seguir. Na Figura 31 é demonstrado um esquema prático das 4 fases da metodologia deste projeto.

- fase 1: cada base de dados será separada aleatoriamente entre conjunto de treino e de teste, com o método de amostragem *holdout* com p a depender do tamanho da base. Sendo assim, o conjunto de treino terá tamanho p e o conjunto de teste terá $(1 - p)$.

- fase 2: o conjunto de treino será dividido entre $k = 5$ conjuntos de dados disjuntos, cada um considerando apenas 10% dos dados originais (em relação ao tamanho da base de treino).
- fase 3: por fim, cada um desses k conjuntos passará pela aplicação das técnicas de DA escolhidas, gerando novos dados até o conjunto atingir o tamanho original da base de treino.
- fase 4: após a geração destas bases aumentadas, cada base passará por experimentos de classificação, com a base de teste definida ao início. Como serão geradas k bases de treino diferentes para cada experimento, este tipo de amostragem pode ser considerado também uma amostragem aleatória, exceto pelo fato da base de teste ser fixa.

Figura 31: Esquema da metodologia empregada.



Fonte: O autor.

4.3.1 Aplicação das Técnicas de DA

Na fase de aplicação das técnicas de DA, o conjunto de dados passa pelas duas técnicas de DA: Transformação de Imagens e pelas *Generative Adversarial Networks*. Estas técnicas serão aplicadas pelas implementações disponíveis com as bibliotecas TensorFlow e Keras.

Os efeitos escolhidos para TIs foram:

- rotação;

- corte;
- translação;
- zoom;
- *flip*;
- preenchimento.

A implementação da GAN utilizada foi a HyperGAN⁵. E alguns dos principais parâmetros padrão da implementação são os seguintes, sendo alterado apenas o número de iterações, *batch size* e dimensão das imagens:

- discriminadora D
 - 4 camadas;
 - *tanh* como ativação final;
 - 1 camada totalmente conectada;
- geradora G
 - 64 de profundidade final;
 - *tanh* como ativação final;
- treinamento
 - taxa de Aprendizado de D (ηD) = 0.0001;
 - taxa de Aprendizado de G (ηG) = 0.0001;
 - 5.000 iterações;
 - *batchsize* = variável com o tamanho das imagens;
 - dimensão das imagens = mesma do *dataset*;

4.3.2 Extração de Características das Imagens

Para a etapa de extração de características das imagens, os conjuntos aumentados passam por uma rede neural convolucional e salva-se os vetores de *features* da última camada, obtidos por elas. Nesta rede neural é utilizado a técnica de transferência de aprendizado, onde serão carregados os pesos da rede já treinada com uma grande base de dados de imagens, a ImageNet (DENG et al., 2009). Com isso, a base no formato atributo-valor (tabelas) formam os conjuntos de treino para os classificadores.

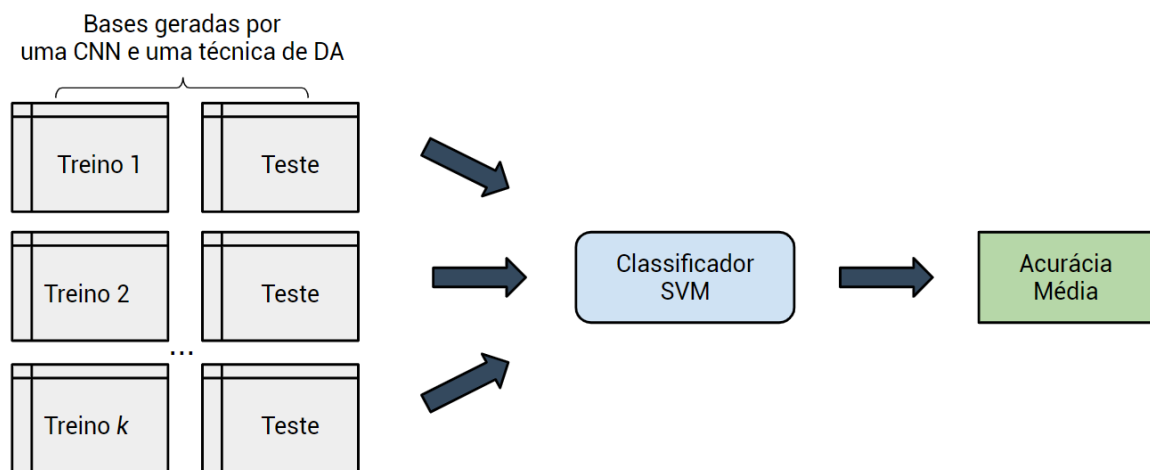
⁵ <https://github.com/HyperGAN/HyperGAN>

4.3.3 Avaliação dos Resultados

As bases geradas pela extração das características das imagens são utilizadas para os experimentos de classificação com a base de teste, usando o algoritmo SVM. Para avaliar esses resultados, foi utilizado a métrica de acurácia e os resultados dados pela acurácia média. Pois cada resultado considera a média dos k conjuntos de dados gerados por cada técnica de DA e para cada CNN, dado uma base de dados.

Em seguida, com os resultados, foram calculadas medidas estatísticas resumo para uma maior comparação e avaliação entre todas abordagens. A Figura 32 ilustra essa fase de avaliação, onde dado um conjunto de dados, o processo se repete para cada CNN e para técnica de DA. Assim, a base de treino é variada entre as k bases aumentada, enquanto que a base de testes é fixa, gerando k acurácias pelo classificador SVM e o resultado final é dado pela média das acurácias obtidas.

Figura 32: Esquema da Avaliação dos Resultados.



Fonte: O autor.

4.3.4 Ambiente Computacional

Os experimentos foram executados em um notebook com processador Intel I5 7^a geração 2.5GHz, 1 TB de HD, 8 GB de RAM e GPU NVIDIA GEFORCE 920MX com Windows 10 e Python 3.7.

5 Resultados

Neste capítulo é detalhado os resultados para as duas técnicas de DA consideradas: Transformação de Imagens e *Generative Adversarial Network*. Assim as seções 5.1 à 5.6 apresentam os resultados da aplicação de DA, enquanto na seção 5.7, o uso e resultados das CNNs são discutidas e por fim, na seção 5.8, uma discussão geral sobre todos resultados é apresentada.

5.1 Base 1: MNIST

Nesta seção será detalhado todos os experimentos que envolvem a base MNIST. Isto é, será demonstrado as imagens geradas pelas técnicas, os experimentos sem DA, com TIs e com GANs e por fim, um resumo dos resultados obtidos.

Para a MNIST, os 8.000 exemplos foram divididos em treino e teste com $p = 70\%$. Dessa forma, a base de treino possui 5.600 exemplos e a de teste, 1.400 exemplos. Vale lembrar, que os 5 conjuntos da simulação possuem então 560 exemplos (280 de cada classe) e passaram pela técnica de TIs e GANs, gerando novos exemplos, até atingir 5.600.

5.1.1 Aplicação de DA

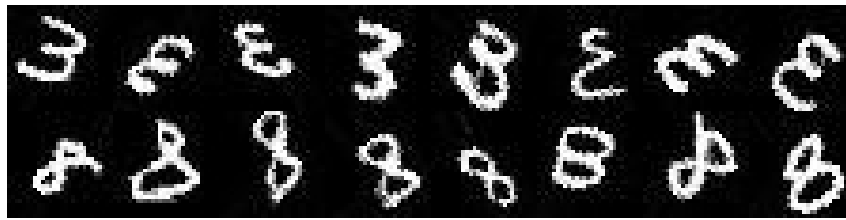
A Figura 33 representa um compilado de exemplos das imagens originais contidas nesse *dataset*, enquanto as Figuras 34 e 35 representam algumas imagens geradas pelas técnicas de TIs e GANs, respectivamente. Pode-se observar bons resultados, ambas técnicas seguem o padrão das imagens originais, com alguns ruídos.

Figura 33: Exemplos de imagens contidas no *dataset* original.



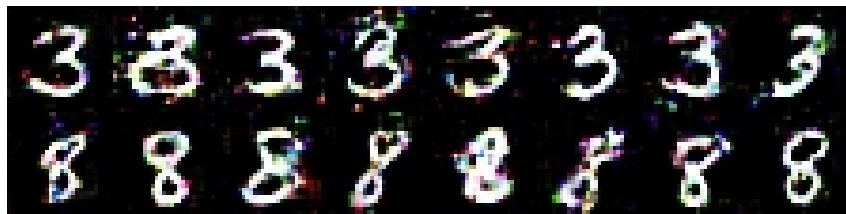
Fonte: (LECUN et al., 1998).

Figura 34: Exemplos de imagens geradas por Transformação de Imagens.



Fonte: O autor.

Figura 35: Exemplos de imagens geradas por *Generative Adversarial Networks*.



Fonte: O autor.

5.1.2 Resultados

As Tabelas 5, 6 e 7, apresentam os resultados obtidos com a Base 1 sem aplicação de DA, com TIs e GANs, respectivamente. Por fim, a Tabela 8 apresenta um resumo desses resultados obtidos.

Tabela 5: Resultados para Base 1 sem DA.

	Conjunto 1	Conjunto 2	Conjunto 3	Conjunto 4	Conjunto 5	Média
ResNet50	97,46	98,04	97,88	97,33	98,29	97,80
VGG16	97,04	96,71	97,29	97,46	96,92	97,08
VGG19	97,12	97,71	97,00	96,92	97,42	97,23
Xception	91,58	91,83	92,17	91,50	92,62	91,94

Fonte: O autor.

Tabela 6: Resultados para Base 1 com TIs.

	Conjunto 1	Conjunto 2	Conjunto 3	Conjunto 4	Conjunto 5	Média
ResNet50	98,12	98,00	98,04	97,29	97,96	97,88
VGG16	94,96	95,00	95,54	95,25	95,46	95,24
VGG19	95,62	95,58	95,54	94,79	96,04	95,51
Xception	87,12	88,29	89,17	88,79	88,62	88,40

Fonte: O autor.

Tabela 7: Resultados para Base 1 com GANs.

	Conjunto 1	Conjunto 2	Conjunto 3	Conjunto 4	Conjunto 5	Média
ResNet50	98,29	98,21	98,33	98,08	97,54	98,09
VGG16	97,00	96,67	97,38	97,21	96,62	96,98
VGG19	97,29	97,71	97,58	97,79	97,54	97,58
Xception	92,46	93,25	93,46	92,75	93,25	93,03

Fonte: O autor.

Tabela 8: Resumo dos resultados para Base 1.

	ResNet50	VGG16	VGG19	Xception	Média	Desvio
10%	97,80	97,08	97,23	91,94	96,01	2,73
100%	99,04	98,38	98,50	96,17	98,02	1,27
TIs	97,88	95,24	95,51	88,40	94,26	4,08
GANs	98,09	96,98	97,58	93,03	96,42	2,30

Fonte: O autor.

5.1.3 Discussão

Com apenas 10% dos dados originais, obtivemos em média 96,01% de acurácia e com a base original (100% dos dados), obtivemos 98,02% de acurácia. Simulando o aumento de 10% para 100% por TIs, obtivemos 94,26% e por GANs, 96,42% de acurácia. Ou seja, nesse caso a abordagem de TI não foi efetiva, piorando a acurácia dos 10%, enquanto por GANs, houve um pequeno aumento na acurácia.

Em relação às CNNs, a ResNet50 foi a que retornou melhores resultados, seguida pela VGG19, VGG16 e por último, a Xception, que apresentou os piores resultados.

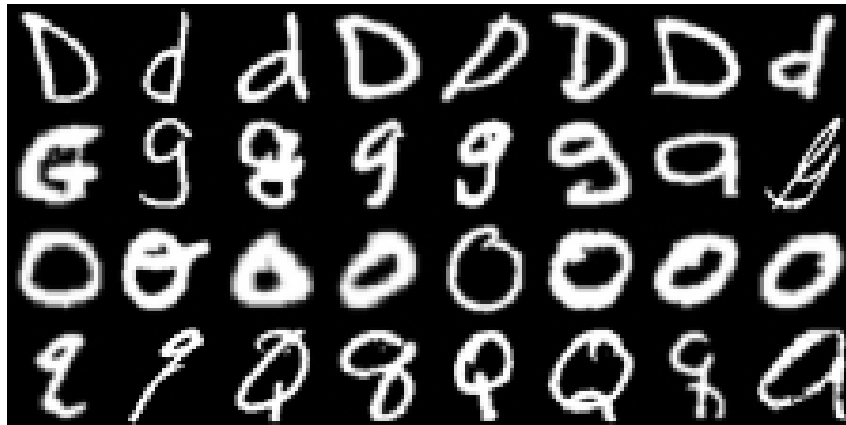
5.2 Base 2: EMNIST

Nesta seção será detalhado todos os experimentos que envolvem a base EMNIST. Para ela, os 22.400 exemplos foram divididos em treino e teste com aproximadamente $p = 85\%$ (já dividido pelo próprio *dataset*). Dessa forma, a base de treino possui 19.200 exemplos e a de teste, 3.200 exemplos. Vale lembrar, que os 5 conjuntos da simulação possuem então 1.920 exemplos (480 de cada classe) e passaram pela técnica de TIs e GANs, gerando novos exemplos, até atingir 19.200.

5.2.1 Aplicação de DA

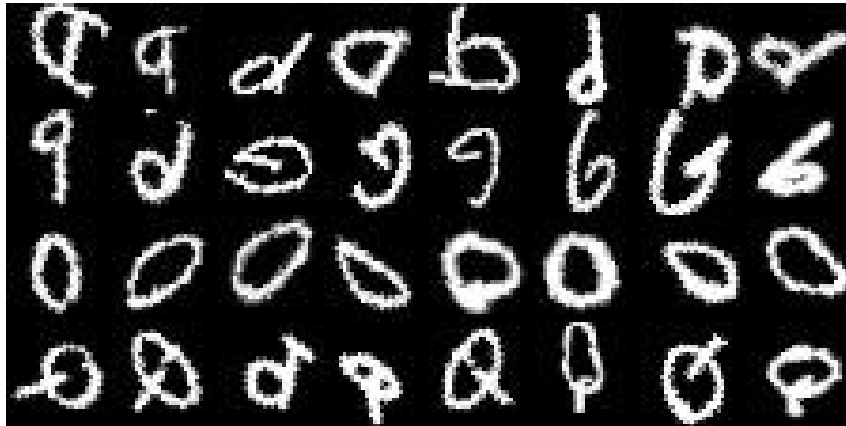
A Figura 36 representa um compilado de exemplos das imagens originais contidas nesse *dataset*, enquanto as Figuras 37 e 38 representam algumas imagens geradas pelas técnicas de TIs e GANs, respectivamente.

Figura 36: Exemplos de imagens contidas no *dataset* original.



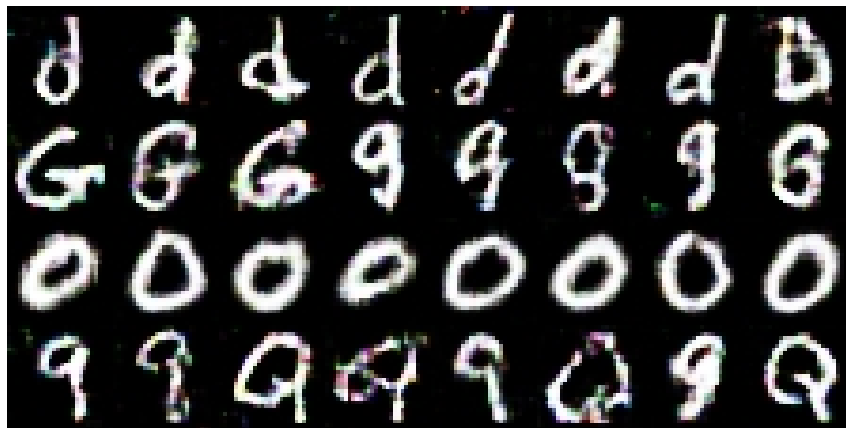
Fonte: O autor.

Figura 37: Exemplos de imagens geradas por Transformação de Imagens.



Fonte: O autor.

Figura 38: Exemplos de imagens geradas por *Generative Adversarial Networks*



Fonte: O autor.

5.2.2 Resultados

As Tabelas 9, 10 e 11, apresentam os resultados obtidos com a Base 2 sem aplicação de DA, com TIs e GANs, respectivamente. Por fim, a Tabela 8 apresenta um resumo dos resultados obtidos.

Tabela 9: Resultados para Base 2 sem DA.

	Conjunto 1	Conjunto 2	Conjunto 3	Conjunto 4	Conjunto 5	Média
ResNet50	82,97	84,81	84,12	84,12	84,22	84,05
VGG16	84,75	84,38	84,91	84,69	85,25	84,80
VGG19	84,25	84,28	85,31	84,97	85,31	84,82
Xception	76,47	76,56	75,59	76,94	76,06	76,32

Fonte: O autor.

Tabela 10: Resultados para Base 2 com TIs.

	Conjunto 1	Conjunto 2	Conjunto 3	Conjunto 4	Conjunto 5	Média
ResNet50	84,84	83,53	83,12	83,12	82,78	83,48
VGG16	83,75	83,81	84,22	83,94	83,81	83,91
VGG19	83,22	83,88	84,28	83,97	83,88	83,85
Xception	74,66	73,94	73,28	75,19	74,34	74,28

Fonte: O autor.

Tabela 11: Resultados para Base 2 com GANs.

	Conjunto 1	Conjunto 2	Conjunto 3	Conjunto 4	Conjunto 5	Média
ResNet50	82,50	83,91	83,88	84,28	84,00	83,71
VGG16	84,06	83,53	84,41	84,88	84,62	84,30
VGG19	84,28	84,09	83,94	84,91	85,00	84,44
Xception	74,66	74,94	75,59	75,44	75,91	75,31

Fonte: O autor.

Tabela 12: Resumo dos resultados para Base 2.

	ResNet50	VGG16	VGG19	Xception	Média	Desvio
10%	84,05	84,80	84,82	76,32	82,50	4,13
100%	88,78	88,50	89,22	82,72	87,31	3,07
TIs	83,48	83,91	83,85	74,28	81,38	4,73
GANs	83,71	84,30	84,44	75,31	81,94	4,43

Fonte: O autor.

5.2.3 Discussão

Com os 10% dos dados originais, obtivemos em média 82,50% de acurácia e com a base original (100% dos dados), obtivemos 87,31% de acurácia. Simulando o aumento de 10% para 100% por TIs, obtivemos 81,38% e por GANs, 81,94% de acurácia.

Embora a abordagem de GANs tenha se saído melhor em relação às TIs, nenhuma das abordagens foi efetiva, pois acabaram piorando a acurácia dos 10%. Esse comportamento pode ser explicado pelo fato das letras serem muito parecidas e possuírem versões minúsculas e maiúsculas. No caso das GANs, isso dificultou o treinamento, em alguns casos a rede aprendeu a reproduzir só uma versão da letra (minúsculo ou maiúsculo). Já com as TIs, há casos de uma letra rotacionada parecer mais com um exemplo de outra classe. Em relação às CNNs, os melhores resultados variaram entre a VGG16 e VGG19, seguida pela ResNet50 e por fim, a Xception, que apresentou os piores resultados.

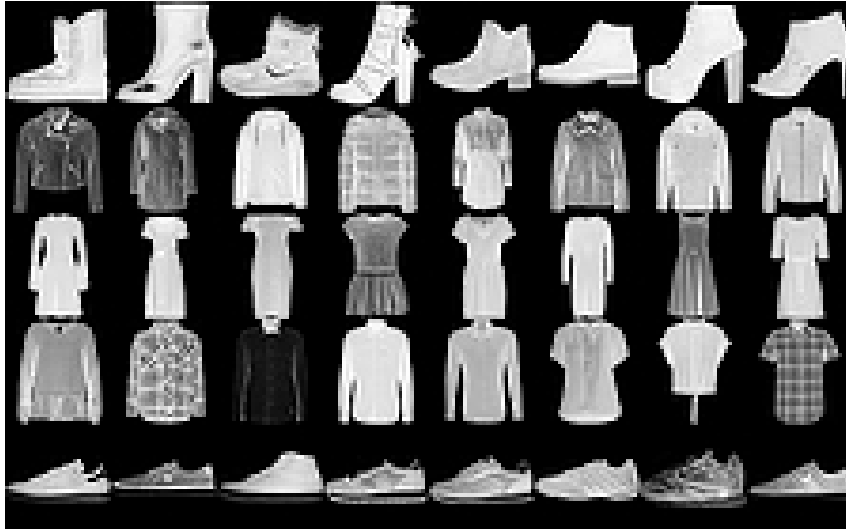
5.3 Base 3: MNIST-Fashion

Nesta seção será detalhado todos os experimentos que envolvem a base MNIST-Fashion. Já para essa base, os 35.000 exemplos foram divididos em treino e teste com aproximadamente $p = 85\%$ (já dividido pelo próprio *dataset*). Dessa forma, a base de treino possui 30.000 exemplos e a de teste, 5.000 exemplos. Vale lembrar, que os 5 conjuntos da simulação possuem então 3.000 exemplos (600 de cada classe) e passaram pela técnica de TIs e GANs, gerando novos exemplos, até atingir 30.000.

5.3.1 Aplicação de DA

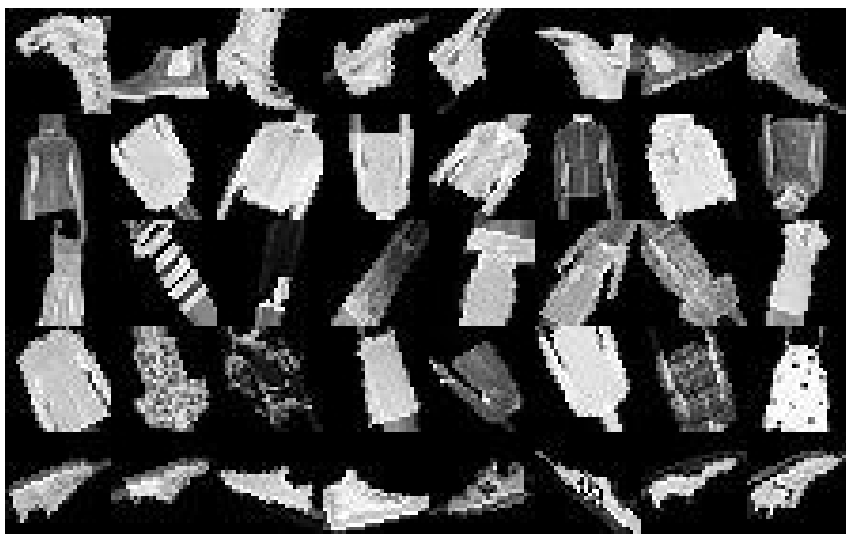
A Figura 39 representa um compilado de exemplos das imagens originais contidas nesse *dataset*, enquanto as Figuras 40 e 41 representam algumas imagens geradas pelas técnicas de TIs e GANs, respectivamente.

Figura 39: Exemplos de imagens contidas no *dataset* original.



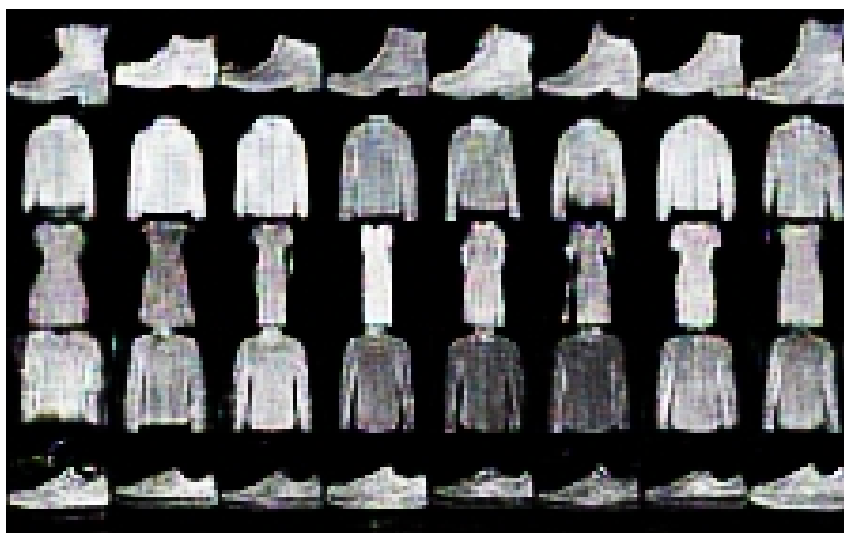
Fonte: O autor.

Figura 40: Exemplos de imagens geradas por Transformação de Imagens.



Fonte: O autor.

Figura 41: Exemplos de imagens geradas por *Generative Adversarial Networks*.



Fonte: O autor.

5.3.2 Resultados

As Tabelas 13, 14 e 15, apresentam os resultados obtidos com a Base 3 sem aplicação de DA, com TIs e GANs, respectivamente. Por fim, a Tabela 8 apresenta um resumo dos resultados obtidos.

Tabela 13: Resultados para Base 3 sem DA.

	Conjunto 1	Conjunto 2	Conjunto 3	Conjunto 4	Conjunto 5	Média
ResNet50	87,56	87,64	87,92	87,84	88,60	87,91
VGG16	86,50	86,66	86,12	86,00	86,78	86,41
VGG19	86,80	86,86	86,66	86,68	87,46	86,89
Xception	81,08	81,22	81,12	80,68	81,02	81,02

Fonte: O autor.

Tabela 14: Resultados para Base 3 com TIs.

	Conjunto 1	Conjunto 2	Conjunto 3	Conjunto 4	Conjunto 5	Média
ResNet50	87,46	87,36	87,58	87,64	87,84	87,58
VGG16	85,64	85,68	85,54	85,76	85,60	85,64
VGG19	87,08	86,92	86,58	86,88	87,34	86,96
Xception	81,02	81,06	81,04	80,86	81,62	81,12

Fonte: O autor.

Tabela 15: Resultados para Base 3 com GANs.

	Conjunto 1	Conjunto 2	Conjunto 3	Conjunto 4	Conjunto 5	Média
ResNet50	87,30	87,92	87,56	87,58	88,04	87,68
VGG16	86,86	87,08	87,02	86,24	87,04	86,85
VGG19	86,60	86,72	86,02	86,32	87,34	86,60
Xception	82,34	82,64	83,06	83,32	82,48	82,77

Fonte: O autor.

Tabela 16: Resumo dos resultados para Base 3.

	ResNet50	VGG16	VGG19	Xception	Média	Desvio
10%	87,91	86,41	86,89	81,02	85,56	3,09
100%	91,36	90,28	90,04	87,32	89,75	1,72
TIs	87,58	85,64	86,96	81,12	85,33	2,92
GANs	87,68	86,85	86,60	82,77	85,97	2,19

Fonte: O autor.

5.3.3 Discussão

Com os 10% dos dados originais, obtivemos em média 85,56% de acurácia e com a base original (100% dos dados), obtivemos 89,75% de acurácia. Simulando o aumento de 10% para 100% por TIs, obtivemos 85,33% e por GANs, 85,97% de acurácia.

Nesse caso, a aplicação de TIs também não foi efetiva, não superando a acurácia dos 10% originais. Com GANs, houve um pequeno aumento. As imagens geradas pelas GANs se destacaram dessa vez, seguindo bem o padrão da base original, com pouco ruído, apenas carecendo de detalhes.

Em relação às CNNs, a ResNet50 foi responsável pelos melhores resultados, em média. Seguida por uma variação entre VGG16 e VGG19, e por último, a Xception, que obteve os piores resultados.

5.4 Base 4: Brazilian Coffee Scenes Dataset

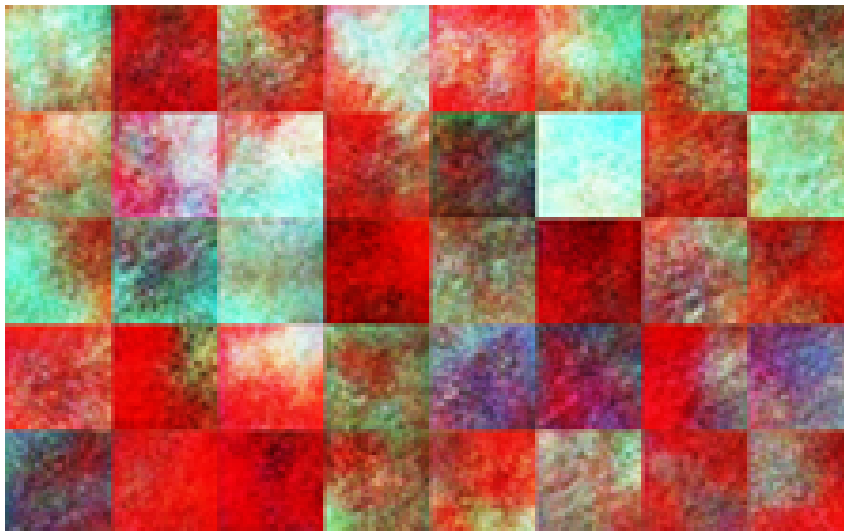
Com essa base de dados, os resultados não foram bons, como pode ser observado com a Figura 42, que representa algumas imagens originais do conjunto, e a Figura 43, que representa algumas imagens geradas pelas GANs.

Figura 42: Exemplos de imagens contidas no *dataset* original.



Fonte: O autor.

Figura 43: Exemplos de imagens geradas por *Generative Adversarial Networks*.



Fonte: O autor.

Esses resultados são explicáveis por serem uma tarefa bem mais complexa e o fato de termos um processamento limitado. Pode ser que, com mais iterações (e mais dias), a rede conseguisse aprender a representar os dados. Por enquanto, decidiu-se por descartar a base para os experimentos de classificação.

5.5 Base 5: CIFAR-10

Com a base de dados CIFAR-10, os resultados obtidos também não foram bons, como pode ser visto pela Figura 44, que representa um resumo das imagens originais contidas no *dataset*, e a Figura 45, que apresenta algumas das imagens geradas pela técnica de GANs.

Em alguns casos, dá pra se perceber, bem vagamente, a silhueta de um cavalo (a classe escolhida para os testes). Porém, estão muito fora do padrão das imagens originais. Nesse caso, é bem possível que com mais iterações, a rede GAN conseguisse aprender a representar os dados. Mas por enquanto, decidiu-se não utilizar essa base para os experimentos de classificação.

Figura 44: Exemplos de imagens contidas no *dataset* original.



Fonte: O autor.

Figura 45: Exemplos de imagens geradas por *Generative Adversarial Networks*.



Fonte: O autor.

5.6 Base 6: Dogs vs Cats

Os resultados obtidos com a base Dogs vs Cats podem ser observados pela Figura 46, que contém exemplos de imagens originais do conjunto, e na Figura 47, imagens geradas pelas GANs.

Esses resultados foram os piores resultados obtidos, as imagens não possuem qualquer semelhança com as imagens base original. Isso é justificado por essa base de dados ser bem diversa, as imagens não possuem nenhum padrão, seja de estilo da imagem ou dimensão, o que dificulta bastante o trabalho da GAN.

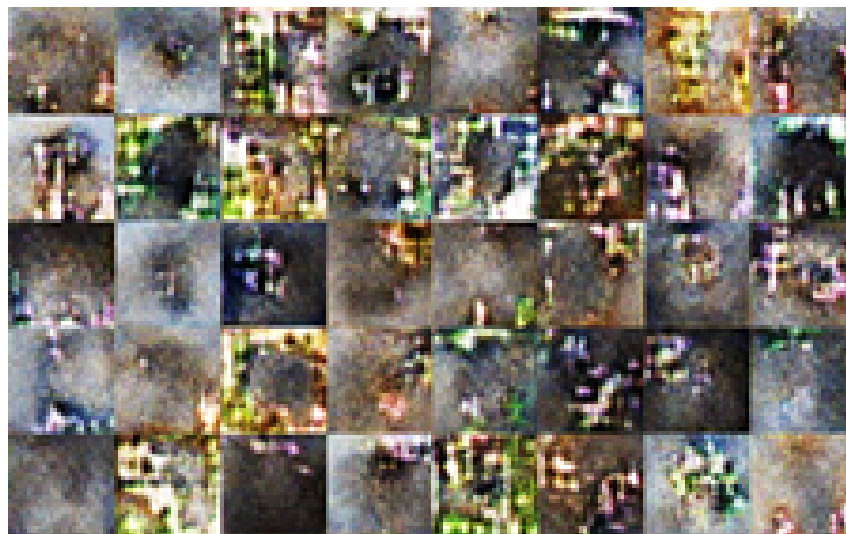
Nesse caso, talvez seja necessário muito mais processamento e semanas de treinamento. Dessa forma, dados os resultados obtidos, decidiu-se por não utilizar mais essa base para os experimentos de classificação.

Figura 46: Exemplos de imagens contidas no *dataset* original.



Fonte: O autor.

Figura 47: Exemplos de imagens geradas por *Generative Adversarial Networks*.



Fonte: O autor.

5.7 Avaliação das CNNs

O segundo objetivo deste trabalho foi estudar e avaliar o uso de CNNs como extratores de características. Esse processo envolve, não somente os resultados de acurácia, mas também seu tamanho, quantidade de parâmetros, o tamanho do armazenamento das *features* e o tempo gasto.

A Tabela 17 apresenta o resultado médio das CNNs para os três *datasets* considerando a média de todos os resultados de classificação com a base original, TIs e GANs. Em geral, a Res-

Net50 obteve melhores resultados. Em seguida, a VGG16 e VGG19 e por último, a Xception, que obteve os piores resultados.

Tabela 17: Resultado médios de acurácia das CNNs para os *datasets*

	ResNet50	VGG16	VGG19	Xception
MNIST	98,20	96,92	97,21	92,39
EMNIST	85,01	85,38	85,58	77,16
MNIST-Fashion	88,63	87,30	87,62	83,06
Média	90,61	89,86	90,14	84,20

Fonte: O autor.

Por outro lado, a Tabela 18 apresenta uma comparação de detalhes de arquitetura da CNN (tamanho da rede, quantidade de parâmetros e o número de atributos que gera), também o tamanho do arquivo de *features* gerado e o tempo gasto. Nesse caso, os dois últimos fatores foram avaliados com a base MNIST-Fashion, pois é a maior base utilizada neste trabalho (30.000 exemplos).

Tabela 18: Comparação das CNNs.

	ResNet50	VGG16	VGG19	Xception
Tamanho da Rede (em MB)	98	528	549	88
Parâmetros	25.636.712	138.357.544	143.667.240	22.910.480
Atributos	2048	4096	4096	4096
Armazenamento (em GB)	0,977247	1,255253	1,204083	0,509851
Tempo (Minutos)	75	122	124	111

Fonte: O autor.

Analisando a tabela, pode-se perceber que a Xception é a rede com menor tamanho, menor número de parâmetros e suas *features* ocupam menor espaço, em relação às outras. A ResNet50, por sua vez, gera menos atributos, levou menos tempo e possui o 2º menor tamanho e número de parâmetros. Já a VGG16 e VGG19 são as redes com maior tamanho, maior quantidade de parâmetros (quase 6x mais), ocupam maior armazenamento e gastaram mais tempo.

Sendo assim, um projeto que faz uso de CNNs deve levar muitas questões em consideração antes de escolher o modelo a ser utilizado. Portanto, deve-se fazer uma análise e ponderar quais são os fatores mais importantes, seja tempo, armazenamento, processamento, ou acurácia. Em geral, a ResNet50 pode ser a mais indicada, já que possui, em média, os melhores resultados, gera menos atributos, levou menor tempo, tem consideravelmente menos parâmetros e também é leve (em tamanho da rede).

5.8 Discussão dos Resultados

A aplicação de DA não foi efetiva em todos os casos. Nas bases mais complexas como a CIFAR-10, Brazilian Coffee Scenes Dataset e Dogs vs Cats, a GAN não conseguiu convergir no tempo estipulado (5.000 épocas), e portanto, essas bases foram descartadas para os experimentos de classificação.

Com as outras bases as duas abordagens foram aplicadas com sucesso. Porém, em TIs percebe-se que alguns exemplos acabaram perdendo o sentido, principalmente com as letras da EMNIST. Em GANs, percebe-se alguns ruídos nas imagens, além do fato de cada experimento (cada classe de cada base) levar, aproximadamente, 1 hora para ser gerado.

Em relação aos experimentos de classificação, conclui-se que a abordagem de GANs foi mais efetiva que a de TIs. E que a abordagem de GANs foi capaz de conseguir aumentar a acurácia, embora, esse aumento seja pequeno, é possível que com mais treinamento, os resultados sejam ainda melhores. Esses resultados são explicáveis pelo fato da abordagem de TIs ser sensível ao tipo de método utilizado, sendo necessário uma análise e seleção de quais técnicas não trarão perda de sentido nas imagens, o que, por simplicidade, não foi realizado aqui. Enquanto as GANs conseguiram representar bem em quase todos os casos, com alguns ruídos.

Já em relação a avaliação das CNNs em resultados dos experimentos de classificação, em geral, a ResNet50 foi a rede que retornou melhores resultados, seguido pela VGG16, VGG19 e por último, a Xception. Em outros fatores, não há uma rede perfeita, e deve-se ponderar o que é mais importante para aplicação: tempo, processamento, armazenamento ou resultados. Porém, em geral, a ResNet50 pode ser indicada, por ter um bom balanço desses fatores.

Além disso, foi verificado que o aumento da acurácia a partir de DA, também foi influenciado pelas CNNs. Em alguns casos, avaliando o resultado da CNN (e não a média das CNNs), pode-se perceber que o ganho foi maior, ou menor, em relação aos 10% originais.

Por fim, foi realizada uma comparação estatística entre os resultados obtidos pelas abordagens de TIs e GANs. Para isso, foi calculada a média combinada, desvio padrão combinado e o desvio absoluto, que foram definidos anteriormente na seção 2.1.4.5, e o resultados podem ser visualizados na Tabela 19, em que $A_1 = \text{TIs}$ e $A_2 = \text{GANs}$.

Tabela 19: Comparação estatística das abordagens.

	Média Combinada	Desvio Padrão Combinado	Diferença Absoluta
MNIST	2,16	0,11	20,46
EMNIST	0,56	0,16	3,63
MNIST-Fashion	0,65	0,08	8,49

Fonte: O autor.

Analisando essa tabela, dado todos os valores de diferença absoluta positivo, concluí-se que o algoritmo A_2 (GAN) foi superior em relação ao algoritmo A_1 (TIs). Além disso, dado que todos valores de diferença absoluta são maiores que 2, conclui-se ainda, que GAN foi superior à TI, com 95% de grau de confiança.

6 Considerações Finais

Na primeira parte desse trabalho, foi realizado o levantamento teórico sobre Aprendizado de Máquina e também investigado um problema comum dos algoritmos de AM: a dependência de uma grande quantidade de dados e ao mesmo tempo, falta de dados. Com a revisão bibliográfica notou-se uma grande quantidade de trabalhos que aplicaram DA em problemas de classificação de imagens, o que corrobora a importância de DA nas pesquisas atuais. Também foi possível analisar diversas técnicas para tratar esse problema, entre elas, escolhemos duas: TIs e GANs. A primeira, expande o conjunto de imagens original aplicando efeitos como corte, rotação e *etc.* E a segunda gera as imagens por duas redes neurais que competem entre si.

Já na segunda parte do trabalho, foram realizados os experimentos de aplicação de DA e de classificação com bases de imagens em diversos contextos. Em geral, a aplicação de DA foi realizada com sucesso, isto é, foi possível gerar novas imagens para aumentar as bases. Porém, há casos específicos de TIs que geraram exemplos muito parecidos (entre classes diferentes), o que pode ter gerado uma confusão pro classificador, e casos em que as GANs não conseguiram convergir com o tempo determinado.

Com os experimentos de aplicação de DA que deram certo, isto é, foi possível gerar novos dados com qualidade aceitáveis, foram realizados os experimento de classificação com o algoritmo SVM. Em média, a abordagem de GANs foi melhor em relação às TIs e aos 10% originais. Além disso, um teste estatístico confirmou a significância dos resultados obtidos pelas GANs sobre TIs, com 95% de grau de confiança. Ainda, TIs em alguns casos, não superou a acurácia dos 10% originais.

Além disso, o objetivo do trabalho foi analisar o uso de CNNs como extratores de características. Como não existe uma CNN perfeita, diversos fatores devem ser levados em consideração, como processamento, tempo, armazenamento e resultados. Embora as redes tenham apresentado bons resultados, a ResNet50 foi a rede que mais balanceou esses fatores.

Portanto, podemos concluir que DA é importante e pode ser considerado quando é preciso obter mais dados e tentar melhorar os resultados. Para isso, é necessário antes uma análise minuciosa sobre quais técnicas utilizar e ponderar, segundo as vantagens e desvantagens de cada método. Por exemplo, TIs é muito rápida, mas é necessário analisar se tais efeitos não podem confundir o classificador. Já com as GANs, os resultados sintéticos podem ser excepcionais, mas necessita de muito processamento e tempo.

Como trabalho futuro, com TIs, pode ser utilizado algum dos algoritmos citados na revisão bibliográfica que conseguem estimar as melhores técnicas para um *dataset*. Com GANs, pode-se treinar a rede por mais tempo, tentar mais processamento computacional, formas de otimizar a rede ou testar outras variações do algoritmo.

Referências

- ABADI, M. et al. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. 2015. Software available from tensorflow.org. Disponível em: <<http://tensorflow.org/>>. Citado na página 83.
- ANTONIOU, A.; STORKEY, A.; EDWARDS, H. Data augmentation generative adversarial networks. *arXiv preprint arXiv:1711.04340*, 2017. Citado 2 vezes nas páginas 75 e 77.
- BAHNSEN, A. C. et al. Improving credit card fraud detection with calibrated probabilities. In: SIAM. *Proceedings of the 2014 SIAM international conference on data mining*. [S.l.], 2014. p. 677–685. Citado na página 34.
- CHAPELLE, O.; SCHOLKOPF, B.; ZIEN, A. Semi-supervised learning (chapelle, o. et al., eds.; 2006)[book reviews]. *IEEE Transactions on Neural Networks*, IEEE, v. 20, n. 3, p. 542–542, 2009. Citado na página 34.
- CHOLLET, F. Xception: Deep learning with depthwise separable convolutions. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. [S.l.: s.n.], 2017. p. 1251–1258. Citado 2 vezes nas páginas 60 e 61.
- CHOLLET, F. et al. *Keras*. 2015. <https://keras.io>. Citado na página 84.
- COHEN, G. et al. Emnist: an extension of mnist to handwritten letters. *arXiv preprint arXiv:1702.05373*, 2017. Citado na página 82.
- COLLOBERT, R.; WESTON, J. A unified architecture for natural language processing: Deep neural networks with multitask learning. In: ACM. *Proceedings of the 25th international conference on Machine learning*. [S.l.], 2008. p. 160–167. Citado na página 53.
- CORTES, C.; VAPNIK, V. Support-vector networks. *Machine learning*, Springer, v. 20, n. 3, p. 273–297, 1995. Citado na página 39.
- CRAWFORD, M. et al. Survey of review spam detection using machine learning techniques. *Journal of Big Data*, SpringerOpen, v. 2, n. 1, p. 23, 2015. Citado na página 33.
- DENG, J. et al. Imagenet: A large-scale hierarchical image database. In: IEEE. *2009 IEEE conference on computer vision and pattern recognition*. [S.l.], 2009. p. 248–255. Citado na página 86.
- DU, D.-Z.; PARDALOS, P. M. *Minimax and applications*. [S.l.]: Springer Science & Business Media, 2013. Citado na página 62.
- DUMOULIN, V.; VISIN, F. A guide to convolution arithmetic for deep learning. *arXiv preprint arXiv:1603.07285*, 2016. Citado 3 vezes nas páginas 54, 55 e 56.
- EITEL, A. et al. Multimodal deep learning for robust rgb-d object recognition. In: IEEE. *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. [S.l.], 2015. p. 681–687. Citado na página 53.

- FACELI, K. et al. Inteligência artificial: Uma abordagem de aprendizado de máquina. 2011. Citado 7 vezes nas páginas 37, 38, 51, 52, 53, 64 e 65.
- FAWZI, A. et al. Adaptive data augmentation for image classification. In: IEEE. *2016 IEEE International Conference on Image Processing (ICIP)*. [S.l.], 2016. p. 3688–3692. Citado 2 vezes nas páginas 74 e 77.
- FRID-ADAR, M. et al. Synthetic data augmentation using gan for improved liver lesion classification. In: IEEE. *2018 IEEE 15th International Symposium on Biomedical Imaging (ISBI 2018)*. [S.l.], 2018. p. 289–293. Citado 2 vezes nas páginas 76 e 77.
- GOODFELLOW, I. et al. Generative adversarial nets. In: *Advances in neural information processing systems*. [S.l.: s.n.], 2014. p. 2672–2680. Citado 3 vezes nas páginas 29, 61 e 62.
- HAYKIN, S. *Neural Networks and Learning Machines*. Pearson Education, 2011. ISBN 9780133002553. Disponível em: <<https://books.google.com.br/books?id=faouAAAAQBAJ>>. Citado 8 vezes nas páginas 45, 46, 47, 48, 49, 50, 51 e 53.
- HE, K. et al. Deep residual learning for image recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun 2016. Disponível em: <<http://dx.doi.org/10.1109/CVPR.2016.90>>. Citado 3 vezes nas páginas 58, 59 e 60.
- HINTON, G. et al. Deep neural networks for acoustic modeling in speech recognition. *IEEE Signal processing magazine*, v. 29, 2012. Citado na página 53.
- HUSSAIN, Z. et al. Differential data augmentation techniques for medical imaging classification tasks. In: AMERICAN MEDICAL INFORMATICS ASSOCIATION. *AMIA Annual Symposium Proceedings*. [S.l.], 2017. v. 2017, p. 979. Citado 3 vezes nas páginas 30, 73 e 77.
- INOUE, H. Data augmentation by pairing samples for images classification. *arXiv preprint arXiv:1801.02929*, 2018. Citado 2 vezes nas páginas 75 e 77.
- KAGGLE. *Dogs vs. Cats*. 2015. Disponível em: <<https://www.kaggle.com/c/dogs-vs-cats-/data>>. Citado 2 vezes nas páginas 80 e 81.
- KRIZHEVSKY, A.; HINTON, G. *Learning multiple layers of features from tiny images*. [S.l.], 2009. Citado na página 80.
- LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. *Nature*, v. 521, p. 436–44, 05 2015. Citado na página 53.
- Lecun, Y. et al. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, v. 86, n. 11, p. 2278–2324, Nov 1998. ISSN 0018-9219. Citado 3 vezes nas páginas 39, 53 e 54.
- LECUN, Y. et al. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, Taipei, Taiwan, v. 86, n. 11, p. 2278–2324, 1998. Citado 3 vezes nas páginas 81, 82 e 89.
- LINNAINMAA, S. The representation of the cumulative rounding error of an algorithm as a taylor expansion of the local rounding errors. *Master's Thesis (in Finnish)*, Univ. Helsinki, p. 6–7, 1970. Citado na página 52.

- LORENA, A. C.; CARVALHO, A. C. de. Uma introdução às support vector machines. *Revista de Informática Teórica e Aplicada*, v. 14, n. 2, p. 43–67, 2007. Citado 5 vezes nas páginas 40, 41, 42, 43 e 44.
- LORENA, A. C.; CARVALHO, A. C. P. d. L. et al. Introdução às máquinas de vetores suporte (support vector machines). 2003. Citado na página 45.
- MCCULLOCH, W. S.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, Springer, v. 5, n. 4, p. 115–133, 1943. Citado na página 47.
- MICHALSKI, R. S.; TECUCI, G. *Machine learning: A multistrategy approach*. [S.l.]: Morgan Kaufmann, 1994. Citado na página 33.
- MITCHELL, T. et al. Machine learning. *Annual review of computer science*, Annual Reviews 4139 El Camino Way, PO Box 10139, Palo Alto, CA 94303-0139, USA, v. 4, n. 1, p. 417–433, 1990. Citado na página 29.
- MONARD, M. C.; BARANAUSKAS, J. A. Conceitos sobre aprendizado de máquina. *Sistemas inteligentes-Fundamentos e aplicações*, v. 1, n. 1, p. 32, 2003. Citado 10 vezes nas páginas 29, 33, 34, 35, 36, 63, 64, 65, 67 e 68.
- MRKŠIĆ, N. Semi-supervised learning methods for data augmentation computer science tripos trinity college may 15, 2013. 2013. Citado na página 69.
- NAIR, V.; HINTON, G. E. Rectified linear units improve restricted boltzmann machines. In: *Proceedings of the 27th international conference on machine learning (ICML-10)*. [S.l.: s.n.], 2010. p. 807–814. Citado na página 48.
- OKAFOR, E. et al. Operational data augmentation in classifying single aerial images of animals. In: IEEE. *2017 IEEE International Conference on INnovations in Intelligent SysTems and Applications (INISTA)*. [S.l.], 2017. p. 354–360. Citado 2 vezes nas páginas 74 e 77.
- OLIPHANT, T. E. *A guide to NumPy*. [S.l.]: Trelgol Publishing USA, 2006. Citado na página 84.
- PARK, B.; BAE, J. K. Using machine learning algorithms for housing price prediction: The case of fairfax county, virginia housing data. *Expert Systems with Applications*, Elsevier, v. 42, n. 6, p. 2928–2934, 2015. Citado na página 34.
- PARKHI, O. M. et al. Deep face recognition. In: *bmvc*. [S.l.: s.n.], 2015. v. 1, n. 3, p. 6. Citado na página 53.
- PAULIN, M. et al. Transformation pursuit for image classification. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. [S.l.: s.n.], 2014. p. 3646–3653. Citado 2 vezes nas páginas 74 e 77.
- PEDREGOSA, F. et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011. Citado na página 84.
- PENATTI K. NOGUEIRA, J. A. d. S. O. A. B. Do deep features generalize from everyday objects to remote sensing and aerial scenes domains? 2015. Citado 4 vezes nas páginas 29, 39, 79 e 80.

- PEREZ, L.; WANG, J. The effectiveness of data augmentation in image classification using deep learning. *arXiv preprint arXiv:1712.04621*, 2017. Citado 2 vezes nas páginas 76 e 77.
- PUDARUTH, S. Predicting the price of used cars using machine learning techniques. *Int. J. Inf. Comput. Technol.*, v. 4, n. 7, p. 753–764, 2014. Citado na página 34.
- ROSENBLATT, F. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, American Psychological Association, v. 65, n. 6, p. 386, 1958. Citado na página 49.
- ROSSUM, G. van. *Python tutorial*. Amsterdam, 1995. Citado na página 83.
- SHIJIE, J. et al. Research on data augmentation for image classification based on convolution neural networks. In: IEEE. *2017 Chinese Automation Congress (CAC)*. [S.l.], 2017. p. 4165–4170. Citado 2 vezes nas páginas 75 e 77.
- SIMONYAN, K.; ZISSERMAN, A. *Very Deep Convolutional Networks for Large-Scale Image Recognition*. 2014. Citado 2 vezes nas páginas 57 e 58.
- STEHLLING, R. O.; NASCIMENTO, M. A.; FALCÃO, A. X. A compact and efficient image retrieval approach based on border/interior pixel classification. In: ACM. *Proceedings of the eleventh international conference on Information and knowledge management*. [S.l.], 2002. p. 102–109. Citado na página 39.
- SWAIN, M. J.; BALLARD, D. H. Color indexing. *International journal of computer vision*, Springer, v. 7, n. 1, p. 11–32, 1991. Citado na página 39.
- SZEGEDY, C. et al. Going deeper with convolutions. *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun 2015. Disponível em: <<http://dx.doi.org/10.1109/CVPR.2015.7298594>>. Citado na página 60.
- TANAKA, F. H. K. dos S.; ARANHA, C. *Data Augmentation Using GANs*. 2019. Citado 2 vezes nas páginas 76 e 77.
- TAYLOR, L.; NITSCHKE, G. Improving deep learning using generic data augmentation. *arXiv preprint arXiv:1708.06020*, 2017. Citado 3 vezes nas páginas 69, 75 e 77.
- TAYLOR, L.; NITSCHKE, G. Improving deep learning with generic data augmentation. In: IEEE. *2018 IEEE Symposium Series on Computational Intelligence (SSCI)*. [S.l.], 2018. p. 1542–1547. Citado 2 vezes nas páginas 29 e 68.
- TORREY, L.; SHAVLIK, J. Transfer learning. In: *Handbook of research on machine learning applications and trends: algorithms, methods, and techniques*. [S.l.]: IGI Global, 2010. p. 242–264. Citado na página 56.
- VARGAS, A. C. G.; PAES, A.; VASCONCELOS, C. N. Um estudo sobre redes neurais convolucionais e sua aplicação em detecção de pedestres. In: *Proceedings of the XXIX Conference on Graphics, Patterns and Images*. [S.l.: s.n.], 2016. p. 1–4. Citado 5 vezes nas páginas 29, 30, 39, 53 e 54.
- VEMURI, P. et al. Alzheimer’s disease diagnosis in individual subjects using structural mr images: validation studies. *Neuroimage*, Elsevier, v. 39, n. 3, p. 1186–1197, 2008. Citado 3 vezes nas páginas 29, 34 e 38.

- WANG, S.-H. et al. Alcoholism detection by data augmentation and convolutional neural network with stochastic pooling. *Journal of medical systems*, Springer, v. 42, n. 1, p. 2, 2018. Citado 3 vezes nas páginas 30, 74 e 77.
- XIAO, H.; RASUL, K.; VOLLGRAF, R. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. 2017. Citado 2 vezes nas páginas 82 e 83.
- YU, X. et al. Deep learning in remote sensing scene classification: a data augmentation enhanced convolutional neural network framework. *GIScience & Remote Sensing*, Taylor & Francis, v. 54, n. 5, p. 741–758, 2017. Citado 3 vezes nas páginas 30, 73 e 77.
- ZHANG, Y.-D. et al. Image based fruit category classification by 13-layer deep convolutional neural network and data augmentation. *Multimedia Tools and Applications*, Springer, v. 78, n. 3, p. 3613–3632, 2019. Citado 2 vezes nas páginas 73 e 77.
- ZHONG, Z. et al. Random erasing data augmentation. *arXiv preprint arXiv:1708.04896*, 2017. Citado 2 vezes nas páginas 75 e 77.